



Department of Computing
CS-212: Object Oriented Programming
Instructor : Mr. Jaudat Mamoon

PROJECT SUBMISSION

Submitted By: *M. Nehan*

Registration No: *541819*

Section: *BSCS-15C*



SUBJECT 47:

Code Files:

1. Main.java:

```
package subject47;

import com.jme3.app.SimpleApplication;
import com.jme3.light.AmbientLight;
import com.jme3.light.DirectionalLight;
import com.jme3.math.ColorRGBA;
import com.jme3.math.Vector3f;
import com.jme3.system.AppSettings;
import subject47.audio.AudioManager;
import subject47.events.EventManager;
import subject47.level.EditorManager;
import subject47.level.LevelManager;
import subject47.player.PlayerController;
import subject47.ui.IntroSequence;
import subject47.ui.MainMenu;
import subject47.ui.UIManager;
import subject47.world.WorldManager;

public class Main extends SimpleApplication {

    private PlayerController player;
    private WorldManager world;
    private UIManager ui;
    private AudioManager audio;
    private EventManager events;
    private LevelManager levelManager;
    private EditorManager editor;
    private MainMenu mainMenu;
    private IntroSequence introSequence;

    private boolean startedGame;
    private boolean gameplayActive;

    public static void main(String[] args) {
        Main app = new Main();
        AppSettings settings = new AppSettings(true);
        settings.setTitle("Subject 47");
        settings.setResolution(1280, 720);
        settings.setResizable(true);
        settings.setVSync(true);
        app.setSettings(settings);
        app.start();
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
@Override
public void simpleInitApp() {
    flyCam.setEnabled(false);
    cam.setFrustumPerspective(60f, (float) cam.getWidth() / Math.max(1f, cam.getHeight()), 0.03f, 250f);
    // Remove JME's built-in ESC-quits mapping so ESC can be used for in-game UI (keypad, menus).
    inputManager.deleteMapping(INPUT_MAPPING_EXIT);

    world = new WorldManager(this);
    world.initialize();

    ui = new UIManager(this);
    ui.initialize();

    audio = new AudioManager(this);
    audio.initialize();

    player = new PlayerController(this);
    player.initialize();

    levelManager = new LevelManager(this);

    editor = new EditorManager(this);
    editor.initialize();

    events = new EventManager(this);
    events.initialize();

    mainMenu = new MainMenu(this);
    mainMenu.initialize();
    mainMenu.hide();

    AmbientLight ambient = new AmbientLight();
    ambient.setColor(ColorRGBA.White.mult(0.22f));
    rootNode.addLight(ambient);

    DirectionalLight sun = new DirectionalLight();
    sun.setDirection(new Vector3f(-0.6f, -1.2f, -0.4f).normalizeLocal());
    sun.setColor(ColorRGBA.White.mult(0.45f));
    rootNode.addLight(sun);

    introSequence = new IntroSequence(this, this::pauseToMenu);
    introSequence.initialize();
    introSequence.start();
}

@Override
public void simpleUpdate(float tpf) {
    ui.update(tpf);
    world.update(tpf);
    mainMenu.update(tpf);
    if (introSequence != null && introSequence.isActive()) {
        introSequence.update(tpf);
        return;
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
if (!gameplayActive) {
    return;
}

player.update(tpf);
editor.update(tpf);

if (!editor.isActive()) {
    levelManager.update(tpf);
    events.update(tpf);
}
}

public void startGame() {
    startedGame = true;
    gameplayActive = true;
    audio.stopMenuMusic();    // ensure menu music stops before gameplay
    mainMenu.hide();
    inputManager.setCursorVisible(true);
    ui.setHudVisible(true);
    ui.closeKeypad();
    levelManager.startNewGame();
    if (gameplayActive && !ui.isOverlayBlockingInput()) {
        player.setEnabled(true);
    }
}

public void pauseToMenu() {
    gameplayActive = false;
    player.setEnabled(false);
    ui.closeKeypad();
    ui.closeInfoModal();
    ui.setHudVisible(false);
    inputManager.setCursorVisible(true);
    mainMenu.showMainScreen();
}

public void resumeGame() {
    if (!startedGame) {
        return;
    }
    audio.stopMenuMusic();    // ensure menu music doesn't bleed into gameplay
    gameplayActive = true;
    inputManager.setCursorVisible(true);
    ui.setHudVisible(true);
    player.setEnabled(true);
}

public void freezeGameplay() {
    gameplayActive = false;
    inputManager.setCursorVisible(true);
    player.setEnabled(false);
}

public boolean hasStartedGame() {
```



```
        return startedGame;
    }

    public boolean isGameplayActive() {
        return gameplayActive;
    }

    public PlayerController getPlayer() {
        return player;
    }

    public WorldManager getWorld() {
        return world;
    }

    public UIManager getUi() {
        return ui;
    }

    public AudioManager getAudio() {
        return audio;
    }

    public LevelManager getLevelManager() {
        return levelManager;
    }

    public EditorManager getEditor() {
        return editor;
    }
}
```

2. AudioManager.java:

```
package subject47.audio;

import com.jme3.app.SimpleApplication;
import com.jme3.audio.AudioData;
import com.jme3.audio.AudioNode;
import javazoom.jl.player.Player;

import java.io.File;
import java.io.FileInputStream;

public class AudioManager {

    private static final String INTRO_MUSIC_FILE = "assets/Audio/Game intro music.mp3";
    private static final String MENU_MUSIC_FILE = "assets/Audio/MainMenu bg music.mp3";

    private final SimpleApplication app;

    // Intro music (JLayer Player)
    private Player introPlayer;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
private Thread introThread;
private volatile boolean introStopRequested = false;

// Menu music (JLayer Decoder + SourceDataLine)
private volatile javax.sound.sampled.SourceDataLine menuLine;
private Thread menuThread;
private volatile boolean menuStopRequested = false;

public AudioManager(SimpleApplication app) { this.app = app; }

public void initialize() {}

// — Intro Music

public void playIntroMusic() {
    stopIntroMusic();
    introStopRequested = false;
    introThread = new Thread(() -> {
        try (FileInputStream fis = new FileInputStream(INTRO_MUSIC_FILE)) {
            introPlayer = new Player(fis);
            introPlayer.play();
        } catch (Exception e) {
            if (!introStopRequested)
                System.err.println("[AudioManager] Intro error: " + e.getMessage());
        }
    });
    introThread.setDaemon(true);
    introThread.setName("IntroMusicThread");
    introThread.start();
}

public void stopIntroMusic() {
    introStopRequested = true;
    if (introPlayer != null) { try { introPlayer.close(); } catch (Exception ignored) {} introPlayer = null; }
    if (introThread != null) { introThread.interrupt(); introThread = null; }
}

public void playMenuMusic() {
    stopMenuMusic(); // ensure clean slate
    menuStopRequested = false;

    menuThread = new Thread(() -> {
        while (!menuStopRequested) {
            javax.sound.sampled.SourceDataLine line = null;
            try (FileInputStream fis = new FileInputStream(MENU_MUSIC_FILE)) {
                javazoom.jl.decoder.Bitstream bs = new javazoom.jl.decoder.Bitstream(fis);
                javazoom.jl.decoder.Decoder dec = new javazoom.jl.decoder.Decoder();
                javazoom.jl.decoder.Header hdr;

                while (!menuStopRequested && (hdr = bs.readFrame()) != null) {
                    javazoom.jl.decoder.SampleBuffer sb =
                        (javazoom.jl.decoder.SampleBuffer) dec.decodeFrame(hdr, bs);

                    // Open the audio line on the first decoded frame
                }
            }
        }
    });
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
if (line == null) {
    javax.sound.sampled.AudioFormat fmt =
        new javax.sound.sampled.AudioFormat(
            hdr.frequency(), 16, sb.getChannelCount(), true, false);
    javax.sound.sampled.DataLine.Info info =
        new javax.sound.sampled.DataLine.Info(
            javax.sound.sampled.SourceDataLine.class, fmt);
    line = (javax.sound.sampled.SourceDataLine)
        javax.sound.sampled.AudioSystem.getLine(info);
    line.open(fmt);
    line.start();
    menuLine = line; // expose so stopMenuMusic() can reach it
}

// Convert short[] PCM → byte[] little-endian and write
short[] pcm = sb.getBuffer();
int len = sb.getBufferLength();
byte[] bytes = new byte[len * 2];
for (int i = 0; i < len; i++) {
    bytes[i * 2] = (byte) (pcm[i] & 0xFF);
    bytes[i * 2 + 1] = (byte) ((pcm[i] >> 8) & 0xFF);
}
if (!menuStopRequested) line.write(bytes, 0, bytes.length);
bs.closeFrame();
}
bs.close();

} catch (Exception e) {
    if (!menuStopRequested)
        System.err.println("[AudioManager] Menu music error: " + e.getMessage());
    break;
} finally {
    // Always clean up the line for this iteration
    javax.sound.sampled.SourceDataLine l = line;
    if (l != null) {
        menuLine = null;
        try { l.stop(); } catch (Exception ignored) {}
        try { l.flush(); } catch (Exception ignored) {}
        try { l.close(); } catch (Exception ignored) {}
    }
}
});
menuThread.setDaemon(true);
menuThread.setName("MenuMusicThread");
menuThread.start();
}

public void stopMenuMusic() {
    menuStopRequested = true;
    // stop()+flush() unblocks write() on the music thread instantly
    javax.sound.sampled.SourceDataLine line = menuLine;
    if (line != null) {
        menuLine = null;
        try { line.stop(); } catch (Exception ignored) {}
    }
}
```




```
        try { line.flush(); } catch (Exception ignored) {}  
        try { line.close(); } catch (Exception ignored) {}  
    }  
    Thread t = menuThread;  
    if (t != null) { menuThread = null; t.interrupt(); }  
}  
  
// — In-World Audio Logs  
  
public void playAudioLog(String path) {  
    try {  
        AudioNode audio = new AudioNode(app.getAssetManager(), path, AudioData.DataType.Buffer);  
        audio.setPositional(false);  
        audio.setLooping(false);  
        audio.setVolume(1.2f);  
        app.getRootNode().attachChild(audio);  
        audio.playInstance();  
    } catch (Exception e) {  
        System.out.println("[AudioManager] Missing audio asset: " + path);  
    }  
}  
}
```

3. EventManager.java:

```
package subject47.events;  
  
import java.util.Random;  
import subject47.Main;  
  
public class EventManager {  
  
    private final Main app;  
    private final Random random = new Random();  
  
    private float timer;  
    private float nextEventDelay = 8f;  
  
    public EventManager(Main app) {  
        this.app = app;  
    }  
  
    public void initialize() {  
    }  
  
    public void update(float tpf) {  
        timer += tpf;  
  
        if (timer >= nextEventDelay) {  
            timer = 0f;  
            nextEventDelay = 7f + random.nextFloat() * 6f;  
            triggerAmbientEvent();  
        }  
    }  
  
    private void triggerAmbientEvent() {  
        switch (app.getLevelManager().getCurrentLevel()) {
```




```
case 1 -> levelOneEvents();
case 2 -> levelTwoEvents();
case 3 -> levelThreeEvents();
case 4 -> levelFourEvents();
default -> {
}
}
}

private void levelOneEvents() {
    if (random.nextBoolean()) {
        app.getWorld().flickerLights(0.7f);
        app.getUi().showMessage("The corridor lights sputter overhead.", 2f);
    } else {
        app.getUi().showMessage("A metallic bang echoes from deeper in the lab.", 2f);
    }
}

private void levelTwoEvents() {
    if (random.nextBoolean()) {
        app.getWorld().flickerLights(1f);
        app.getUi().showMessage("Footsteps circle behind you, then stop.", 2.2f);
    } else {
        app.getUi().showMessage("A camera feed jitters: movement detected.", 2.2f);
    }
}

private void levelThreeEvents() {
    if (random.nextBoolean()) {
        app.getWorld().setEmergencyLighting(true);
        app.getUi().showMessage("Terminal text appears on its own: WHY ARE YOU HERE?", 2.5f);
    } else {
        app.getUi().showMessage("The corridor layout feels wrong when you look back.", 2.5f);
    }
}

private void levelFourEvents() {
    if (random.nextBoolean()) {
        app.getWorld().flickerLights(1.2f);
        app.getUi().showMessage("Memory fragments flash at the edge of your vision.", 2.5f);
    } else {
        app.getUi().showMessage("A system voice repeats: SUBJECT ACTIVE.", 2.2f);
    }
}
}
```

4. ControlSwitch.java:

```
package subject47.interaction;

import subject47.Main;

public class ControlSwitch implements Interactable {

    private final Main app;
    private final String prompt;
    private final String successMessage;
```



```
private final Runnable action;

public ControlSwitch(Main app, String prompt, String successMessage, Runnable action) {
    this.app = app;
    this.prompt = prompt;
    this.successMessage = successMessage;
    this.action = action;
}

@Override
public void interact() {
    if (action != null) {
        action.run();
    }
    if (successMessage != null && !successMessage.isBlank()) {
        app.getUi().showMessage(successMessage, 1.8f);
    }
}

@Override
public String getPrompt() {
    return prompt;
}
}
```

5. Door.java:

```
package subject47.interaction;

import com.jme3.scene.Node;
import com.jme3.math.Vector3f;
import subject47.Main;

public class Door implements Interactable {

    private final Main app;
    private final Node spatial;
    private final Vector3f closedPosition;
    private final Vector3f openPosition;
    private final String name;

    private boolean locked;
    private boolean open;
    private float animationSpeed = 4.6f;
    private Runnable onLockedCallback;

    public Door(Main app, Node spatial, String name, Vector3f closedPosition, Vector3f openOffset, boolean locked) {
        this.app = app;
        this.spatial = spatial;
        this.name = name;
        this.closedPosition = closedPosition.clone();
        this.openPosition = closedPosition.add(openOffset);
        this.locked = locked;
        this.spatial.setLocalTranslation(closedPosition);
    }

    @Override
```



```
public void interact() {
    if (locked) {
        if (onLockedCallback != null) {
            onLockedCallback.run();
        } else {
            app.getUi().showMessage("A red door control is mounted nearby for " + name + ".", 2f);
        }
    } else {
        toggleFromButton(null);
    }
}

public void setOnLockedCallback(Runnable callback) {
    this.onLockedCallback = callback;
}

@Override
public String getPrompt() {
    return locked ? "Press E to inspect " + name : "Press E to use " + name;
}

public void unlock(String message) {
    locked = false;
    app.getUi().showMessage(message, 3f);
}

public void lock() {
    locked = true;
    setOpen(false);
}

public void openFromEvent(String message) {
    locked = false;
    setOpen(true);
    app.getUi().showMessage(message, 3f);
}

public void toggleFromButton(String message) {
    if (locked) {
        app.getUi().showMessage(name + " is still locked.", 2f);
        return;
    }
    setOpen(!open);
    if (message != null && !message.isBlank()) {
        app.getUi().showMessage(message, 2f);
    }
}

public void setOpen(boolean open) {
    this.open = open;
}

public void update(float tpf) {
    Vector3f target = open ? openPosition : closedPosition;
    Vector3f current = spatial.getLocalTranslation();
```



```
        if (current.distanceSquared(target) < 0.0004f) {  
            spatial.setLocalTranslation(target);  
            return;  
        }  
        spatial.setLocalTranslation(current.interpolateLocal(target, Math.min(1f, tpf * animationSpeed)));  
    }  
  
    public boolean isLocked() {  
        return locked;  
    }  
  
    public boolean isOpen() {  
        return open;  
    }  
  
    public boolean blocksPosition(Vector3f point, float radius) {  
        Vector3f current = spatial.getLocalTranslation();  
        float halfWidth = 0.78f + radius;  
        float halfDepth = 0.24f + radius;  
        float bottom = current.y - 1.55f;  
        float top = current.y + 1.55f;  
  
        if (point.y < bottom || point.y > top) {  
            return false;  
        }  
  
        return Math.abs(point.x - current.x) <= halfWidth  
            && Math.abs(point.z - current.z) <= halfDepth;  
    }  
  
    public Node getSpatial() {  
        return spatial;  
    }  
}
```

6. Interactable.java:

```
package subject47.interaction;  
  
public interface Interactable {  
    void interact();  
    String getPrompt();  
}
```

7. Keypad.java:

```
package subject47.interaction;  
  
import java.util.function.Consumer;  
import subject47.Main;  
  
public class Keypad implements Interactable {  
  
    private final Main app;  
    private final String correctCode;  
    private final String title;  
    private final Consumer<String> onSuccess;
```



```
public Keypad(Main app, String title, String correctCode, Consumer<String> onSuccess) {
    this.app = app;
    this.correctCode = correctCode;
    this.title = title;
    this.onSuccess = onSuccess;
}

@Override
public void interact() {
    app.getUi().openKeypad(title, code -> {
        if (correctCode.equals(code)) {
            app.getUi().showMessage("Access granted.", 2f);
            onSuccess.accept(code);
        } else {
            app.getUi().showMessage("Access denied.", 2f);
        }
    });
}

@Override
public String getPrompt() {
    return "Press E to use keypad";
}
}
```

8. Terminal.java:

```
package subject47.interaction;

import subject47.Main;

public class Terminal implements Interactable {

    private final Main app;
    private final String prompt;
    private final String message;
    private final Runnable action;
    private final boolean singleUse;

    private boolean used;

    public Terminal(Main app, String prompt, String message, boolean singleUse, Runnable action) {
        this.app = app;
        this.prompt = prompt;
        this.message = message;
        this.singleUse = singleUse;
        this.action = action;
    }

    @Override
    public void interact() {
        if (singleUse && used) {
            app.getUi().showMessage("Nothing new appears on the terminal.", 2f);
            return;
        }
    }
}
```



```
used = true;
if (message != null && !message.isBlank()) {
    app.getUi().showNarration(message, 6f);
}

if (action != null) {
    action.run();
}
}

@Override
public String getPrompt() {
    return prompt;
}
}
```

9. CameraSystem.java:

```
package subject47.level;

import com.jme3.app.SimpleApplication;
import com.jme3.math.Vector3f;

public class CameraSystem {

    private SimpleApplication app;
    private int currentCam = 0;

    private Vector3f[] cameraPositions = {
        new Vector3f(5,5,5),
        new Vector3f(10,5,10),
        new Vector3f(2,5,12)
    };

    public CameraSystem(SimpleApplication app) {
        this.app = app;
    }

    public void switchCamera() {

        currentCam = (currentCam + 1) % cameraPositions.length;

        app.getCamera().setLocation(cameraPositions[currentCam]);

        System.out.println("Switched camera to " + currentCam);

        if (currentCam == 2) {
            System.out.println("SHADOW SEEN IN CAMERA");
        }
    }
}
```

10. EditorManager.java:

```
package subject47.level;

import java.util.ArrayDeque;
import java.util.Deque;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
import com.jme3.input.KeyInput;
import com.jme3.input.controls.ActionListener;
import com.jme3.input.controls.KeyTrigger;
import com.jme3.math.Vector3f;
import subject47.Main;
import subject47.world.BlockType;
import subject47.world.EditorPropType;
import subject47.world.WorldManager;

public class EditorManager {

    private static final BlockType[] EDITABLE_TYPES = {
        BlockType.FLOOR,
        BlockType.WALL,
        BlockType.GLASS,
        BlockType.LIGHT,
        BlockType.CEILING,
        BlockType.TRIM,
        BlockType.PANEL,
        BlockType.AIR
    };

    private static final EditorPropType[] EDITABLE_PROPS = {
        EditorPropType.AUTO_DOOR,
        EditorPropType.KEYPAD_DOOR,
        EditorPropType.CONSOLE,
        EditorPropType.LAB_BENCH,
        EditorPropType.SERVER_RACK,
        EditorPropType.STORAGE_CRATE,
        EditorPropType.MEDICAL_CART,
        EditorPropType.WALL_MONITOR,
        EditorPropType.LOCKER_BANK,
        EditorPropType.CEILING_LIGHT,
        EditorPropType.ANALYZER_STATION,
        EditorPropType.TANK_CLUSTER,
        EditorPropType.CONTAINMENT_POD
    };

    private final Main app;
    private final ActionListener actionListener;
    private final Deque<EditAction> undoStack = new ArrayDeque<>();

    private boolean active;
    private int selectedIndex;
    private int selectedPropIndex;
    private boolean propMode;
    private int propRotation; // 0-3, each step = 90° around Y

    public EditorManager(Main app) {
        this.app = app;
        this.actionListener = this::handleAction;
    }

    public void initialize() {
        app.getInputManager().addMapping("EditorToggle", new KeyTrigger(KeyInput.KEY_F1));
        app.getInputManager().addMapping("EditorPrev", new KeyTrigger(KeyInput.KEY_Z));
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
app.getInputManager().addMapping("EditorNext", new KeyTrigger(KeyInput.KEY_X));
app.getInputManager().addMapping("EditorPlace", new KeyTrigger(KeyInput.KEY_Q));
app.getInputManager().addMapping("EditorDelete", new KeyTrigger(KeyInput.KEY_R));
app.getInputManager().addMapping("EditorUndo", new KeyTrigger(KeyInput.KEY_U));
app.getInputManager().addMapping("EditorMode", new KeyTrigger(KeyInput.KEY_V));
app.getInputManager().addMapping("EditorSave", new KeyTrigger(KeyInput.KEY_F5));
app.getInputManager().addMapping("EditorRotate", new KeyTrigger(KeyInput.KEY_T));
app.getInputManager().addListener(
    actionListener,
    "EditorToggle", "EditorPrev", "EditorNext", "EditorPlace",
    "EditorDelete", "EditorUndo", "EditorMode", "EditorSave", "EditorRotate"
);
}

public void update(float tpf) {
    if (!active) {
        app.getUi().setEditorOverlay(null);
        app.getWorld().clearEditorGhost();
        return;
    }

    WorldManager.BlockHit hit = app.getWorld().findBlockHit(
        app.getCamera().getLocation(), app.getCamera().getDirection(), 8f);
    int px = 0, py = 0, pz = 0;
    if (hit != null) {
        px = hit.x() + Math.round(hit.normal().x);
        py = hit.y() + Math.round(hit.normal().y);
        pz = hit.z() + Math.round(hit.normal().z);
    }
    String target = hit == null ? "none" : px + "," + py + "," + pz;
    String rotLabel = propMode ? " rot=" + (propRotation * 90) + "°" : "";
    app.getUi().setEditorOverlay(
        "EDITOR F1 exit F fly V mode Z/X cycle T rotate Q place R delete U undo F5 save\n" +
        (propMode ? "Prop " + getSelectedPropType() : "Block " + getSelectedType()) +
        rotLabel + " Place:" + target + " Undo:" + undoStack.size() +
        (app.getPlayer().isFlyMode() ? " [FLY]" : "")
    );
};

if (propMode) {
    app.getWorld().updateEditorGhost(
        getSelectedPropType(), hit, app.getCamera().getDirection(), propRotation);
} else {
    app.getWorld().clearEditorGhost();
}
}

public boolean isActive() {
    return active;
}

private void handleAction(String name, boolean pressed, float tpf) {
    if (!pressed) {
        return;
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
switch (name) {
  case "EditorToggle" -> toggle();
  case "EditorPrev" -> {
    if (active) {
      if (propMode) {
        selectedPropIndex = (selectedPropIndex + EDITABLE_PROPS.length - 1) % EDITABLE_PROPS.length;
      } else {
        selectedIndex = (selectedIndex + EDITABLE_TYPES.length - 1) % EDITABLE_TYPES.length;
      }
    }
  }
  case "EditorNext" -> {
    if (active) {
      if (propMode) {
        selectedPropIndex = (selectedPropIndex + 1) % EDITABLE_PROPS.length;
      } else {
        selectedIndex = (selectedIndex + 1) % EDITABLE_TYPES.length;
      }
    }
  }
  case "EditorMode" -> {
    if (active) {
      propMode = !propMode;
      propRotation = 0;
      app.getWorld().clearEditorGhost();
    }
  }
  case "EditorRotate" -> {
    if (active && propMode) {
      propRotation = (propRotation + 1) % 4;
    }
  }
  case "EditorPlace" -> {
    if (active) {
      if (propMode) {
        placeProp();
      } else {
        placeBlock();
      }
    }
  }
  case "EditorDelete" -> {
    if (active) {
      if (propMode) {
        deleteProp();
      } else {
        deleteBlock();
      }
    }
  }
  case "EditorUndo" -> {
    if (active) {
      undo();
    }
  }
}
```



```
case "EditorSave" -> {
    if (active) {
        app.getWorld().saveCurrentChunk(app.getLevelManager().getCurrentLevel());
        app.getWorld().saveEditorProps(app.getLevelManager().getCurrentLevel());
        app.getUi().showMessage("Editor map saved for level " + app.getLevelManager().getCurrentLevel() + ".");
    }
}

2f);
}
}
default -> {
}
}
}

private void toggle() {
    active = !active;
    app.getUi().showMessage(active
        ? "Editor enabled. F to fly, V switches block/prop mode."
        : "Editor disabled.", 2f);
    if (!active) {
        app.getUi().setEditorOverlay(null);
        app.getWorld().clearEditorGhost();
        app.getPlayer().setFlyMode(false); // land when leaving editor
    }
}

private void placeBlock() {
    WorldManager.BlockHit hit = app.getWorld().findBlockHit(
        app.getCamera().getLocation(), app.getCamera().getDirection(), 8f);
    BlockType type = getSelectedType();

    int x, y, z;

    if (hit == null) {
        // No surface hit — place 3 m in front at floor level
        Vector3f point = app.getCamera().getLocation().add(
            app.getCamera().getDirection().mult(3f));
        x = app.getWorld().toChunkIndex(point.x);
        y = app.getWorld().toChunkIndex(Math.max(0.6f, point.y - 1.2f));
        z = app.getWorld().toChunkIndex(point.z);
    } else {
        // JME collision normals point OUTWARD (toward viewer).
        // Adding the normal moves us INTO the adjacent air cell — correct placement.
        x = hit.x() + Math.round(hit.normal().x);
        y = hit.y() + Math.round(hit.normal().y);
        z = hit.z() + Math.round(hit.normal().z);
    }

    BlockType previous = app.getWorld().getBlock(x, y, z);
    if (previous == type) {
        return;
    }
    app.getWorld().setBlock(x, y, z, type);
    pushUndo(x, y, z, previous, type);
}
```



```
private void deleteBlock() {
    WorldManager.BlockHit hit = app.getWorld().findBlockHit(app.getCamera().getLocation(),
app.getCamera().getDirection(), 8f);
    if (hit == null) {
        return;
    }
    BlockType previous = app.getWorld().getBlock(hit.x(), hit.y(), hit.z());
    if (previous == BlockType.AIR) {
        return;
    }
    app.getWorld().setBlock(hit.x(), hit.y(), hit.z(), BlockType.AIR);
    pushUndo(hit.x(), hit.y(), hit.z(), previous, BlockType.AIR);
}

private void placeProp() {
    WorldManager.BlockHit hit = app.getWorld().findBlockHit(
        app.getCamera().getLocation(), app.getCamera().getDirection(), 8f);
    if (hit == null) {
        app.getUi().showMessage("Aim at a surface to place a prop.", 1.5f);
        return;
    }
    app.getWorld().placeEditorProp(getSelectedPropType(), hit, app.getCamera().getDirection(), propRotation);
}

private void deleteProp() {
    String hit = app.getWorld().findAnyPropHit(app.getCamera().getLocation(), app.getCamera().getDirection(), 8f);
    if (hit == null) {
        app.getUi().showMessage("No prop targeted.", 1.5f);
        return;
    }
    // hit is "editor:<id>" or "level:<id>"
    int colon = hit.indexOf(':');
    String kind = hit.substring(0, colon);
    String propId = hit.substring(colon + 1);
    boolean removed;
    if ("level".equals(kind)) {
        removed = app.getWorld().removeLevelProp(propId);
    } else {
        removed = app.getWorld().removeEditorProp(propId);
    }
    if (removed) {
        app.getUi().showMessage("Prop removed.", 1.2f);
    } else {
        app.getUi().showMessage("Failed to remove prop.", 1.5f);
    }
}

private BlockType getSelectedType() {
    return EDITABLE_TYPES[selectedIndex];
}

private EditorPropType getSelectedPropType() {
    return EDITABLE_PROPS[selectedPropIndex];
}
```



```
private void undo() {  
    EditAction action = undoStack.pollLast();  
    if (action == null) {  
        app.getUi().showMessage("Nothing to undo.", 1.5f);  
        return;  
    }  
    app.getWorld().setBlock(action.x(), action.y(), action.z(), action.before());  
    app.getUi().showMessage("Undo applied.", 1.2f);  
}  
  
private void pushUndo(int x, int y, int z, BlockType before, BlockType after) {  
    undoStack.addLast(new EditAction(x, y, z, before, after));  
    while (undoStack.size() > 200) {  
        undoStack.pollFirst();  
    }  
}  
  
private record EditAction(int x, int y, int z, BlockType before, BlockType after) {  
}  
}
```

11. LevelManager.java:

```
package subject47.level;  
  
import java.util.LinkedHashMap;  
import java.util.Map;  
import com.jme3.math.ColorRGBA;  
import com.jme3.math.Vector3f;  
import com.jme3.scene.Spatial;  
import subject47.Main;  
import subject47.interaction.Keypad;  
import subject47.interaction.Terminal;  
import subject47.world.EditorPropType;  
  
public class LevelManager {  
  
    private final Main app;  
  
    private int currentLevel = 1;  
    private int nextLevelPropId = 0;  
    private float levelTimer;  
  
    private final Map<String, Integer> logDigits = new LinkedHashMap<>();  
    private int generatorProgress;  
    private int distortionProgress;  
    private int revelationProgress;  
    private boolean firstDoorEventTriggered;  
    private boolean redLightTriggered;  
  
    public LevelManager(Main app) {  
        this.app = app;  
    }  
  
    public void startNewGame() {  
        logDigits.clear();  
        generatorProgress = 0;  
    }  
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
distortionProgress = 0;
revelationProgress = 0;
loadLevel(1);
}

public void loadLevel(int level) {
    currentLevel = level;
    levelTimer = 0f;
    firstDoorEventTriggered = false;
    redLightTriggered = false;
    nextLevelPropId = 0;

    app.getWorld().loadLevel(level);
    app.getWorld().setEmergencyLighting(level >= 3);
    placePlayer();

    switch (level) {
        case 1 -> setupLevelOne();
        case 2 -> setupLevelTwo();
        case 3 -> setupLevelThree();
        case 4 -> setupLevelFour();
        default -> setupLevelOne();
    }
}

public void update(float tpf) {
    levelTimer += tpf;

    switch (currentLevel) {
        case 1 -> updateLevelOne();
        case 2 -> updateLevelTwo();
        case 3 -> updateLevelThree();
        case 4 -> updateLevelFour();
        default -> {
        }
    }
}

public int getCurrentLevel() {
    return currentLevel;
}

private void placePlayer() {
    Vector3f spawn = app.getWorld().getSpawnPoint(currentLevel);
    Vector3f lookAt = spawn.add(new Vector3f(0f, 0f, -4f));
    app.getPlayer().placeAt(spawn, lookAt);
}

private void setupLevelOne() {
    logDigits.clear();
    clearHudMissionText();

    dressLevelOne();

    app.getWorld().createPcModel(
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
"LogA",
app.getWorld().worldToScene(5, 1.2f, 5),
new ColorRGBA(0.2f, 0.9f, 0.9f, 1f),
new Terminal(app,
    "Press E to play audio log",
    "",
    true,
    () -> collectLog("A1", 4, "Audio/entry-log-01.ogg"))
);

app.getWorld().createPcModel(
    "LogB",
    app.getWorld().worldToScene(17, 1.2f, 6),
    new ColorRGBA(0.2f, 0.9f, 0.9f, 1f),
    new Terminal(app,
        "Press E to play audio log",
        "",
        true,
        () -> collectLog("B2", 7, "Audio/entry-log-02.ogg"))
);

app.getWorld().createPcModel(
    "LogC",
    app.getWorld().worldToScene(29, 1.2f, 20),
    new ColorRGBA(0.2f, 0.9f, 0.9f, 1f),
    new Terminal(app,
        "Press E to play audio log",
        "",
        true,
        () -> collectLog("C3", 2, "Audio/entry-log-03.ogg"))
);

app.getWorld().createPcModel(
    "LogD",
    app.getWorld().worldToScene(40, 1.2f, 28),
    new ColorRGBA(0.2f, 0.9f, 0.9f, 1f),
    new Terminal(app,
        "Press E to play audio log",
        "",
        true,
        () -> collectLog("D4", 1, "Audio/entry-log-04.ogg"))
);

app.getWorld().createConsole(
    "ControlKeypad",
    app.getWorld().worldToScene(17, 1.2f, 17),
    new ColorRGBA(0.15f, 0.7f, 0.85f, 1f),
    new Keypad(app, "CONTROL ROOM OVERRIDE", "4721", code -> {
        app.getWorld().setEmergencyLighting(true);
        showLevelTransition(
            "LEVEL 1 COMPLETE",
            "ENTRY LOGS COMPLETE\n\nThe four archived digits formed the full control room passcode:
4721.\nObservation wing access is now available.",
            2
        );
    });
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
    });  
};  
  
app.getWorld().createConsole(  
    "PowerTerminal",  
    app.getWorld().worldToScene(12, 1.2f, 21),  
    new ColorRGBA(0.9f, 0.25f, 0.25f, 1f),  
    new Terminal(app,  
        "Press E to restore partial power",  
        "Partial power restored. The corridor grid wakes, then destabilizes under an unseen override.",  
        false,  
        () -> {  
        })  
    );  
  
showLevelBriefing(  
    "LEVEL 1 ENTRY LOGS",  
    "A repair call brought you into the underground facility.\n\n" +  
    "The lab is empty, but the systems are not quiet.\n\n" +  
    "Objective:\nRecover four audio logs and assemble the control room passcode.\n\n" +  
    "The control room keypad still needs a full four-digit override.",  
    false  
);  
}  
  
private void setupLevelTwo() {  
    generatorProgress = 0;  
    clearHudMissionText();  
  
    dressLevelTwo();  
  
    app.getWorld().createConsole(  
        "GeneratorTwo",  
        app.getWorld().worldToScene(8, 1.2f, 18),  
        new ColorRGBA(0.85f, 0.65f, 0.2f, 1f),  
        new Terminal(app,  
            "Press E to cycle Generator ",  
            "Generator 2 hums weakly.",  
            false,  
            () -> activateGenerator(2))  
    );  
  
    app.getWorld().createConsole(  
        "GeneratorOne",  
        app.getWorld().worldToScene(12, 1.2f, 20),  
        new ColorRGBA(0.85f, 0.65f, 0.2f, 1f),  
        new Terminal(app,  
            "Press E to cycle Generator ",  
            "Generator 1 catches with a low thud.",  
            false,  
            () -> activateGenerator(1))  
    );  
  
    app.getWorld().createConsole(  
        "GeneratorThree",
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
app.getWorld().worldToScene(16, 1.2f, 22),
new ColorRGBA(0.85f, 0.65f, 0.2f, 1f),
new Terminal(app,
    "Press E to cycle Generator",
    "Generator 3 floods the corridor with static.",
    false,
    () -> activateGenerator(3))
);

app.getWorld().createConsole(
    "CameraTerminal",
    app.getWorld().worldToScene(31, 1.2f, 20),
    new ColorRGBA(0.2f, 0.9f, 0.85f, 1f),
    new Terminal(app,
        "Press E to review observation feeds",
        "Camera feed: a figure stands behind you in glass. You turn. Nothing is there.",
        false,
        () -> {
            if (generatorProgress >= 3) {
                showLevelTransition(
                    "LEVEL 2 COMPLETE",
                    "OBSERVATION\n\nAll camera feeds restored.\nA final system message flashes: SUBJECT
ACTIVE.",
                    3
                );
            } else {
                app.getUi().showMessage("All three generators must be stabilized first.", 2f);
            }
        })
);

showLevelBriefing(
    "LEVEL 2 OBSERVATION",
    "The observation wing is online again.\n\n" +
    "The cameras show movement where no one stands.\n" +
    "Glass reflections refuse to agree with the room around you.\n\n" +
    "Objective:\nRestore the camera system by activating the three generators in the correct order.",
    false
);
}

private void setupLevelThree() {
    distortionProgress = 0;
    clearHudMissionText();

    dressLevelThree();

    app.getWorld().createConsole(
        "DistortionA",
        app.getWorld().worldToScene(10, 1.2f, 20),
        new ColorRGBA(0.9f, 0.2f, 0.2f, 1f),
        new Terminal(app,
            "Press E to inspect corrupted note",
            "The note rewrites itself: IT'S NOT OBSERVING. IT'S ADAPTING.",
            true,
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        this::progressDistortion)
    );

    app.getWorld().createConsole(
        "DistortionB",
        app.getWorld().worldToScene(33, 1.2f, 9),
        new ColorRGBA(0.9f, 0.2f, 0.2f, 1f),
        new Terminal(app,
            "Press E to inspect corrupted terminal",
            "The screen types on its own: WHY ARE YOU HERE?",
            true,
            this::progressDistortion)
    );

    app.getWorld().createConsole(
        "DistortionC",
        app.getWorld().worldToScene(43, 1.2f, 28),
        new ColorRGBA(0.9f, 0.2f, 0.2f, 1f),
        new Terminal(app,
            "Press E to inspect looped note",
            "The corridor map redraws itself, always returning to the chamber.",
            true,
            this::progressDistortion)
    );

    app.getWorld().createConsole(
        "ResetProtocol",
        app.getWorld().worldToScene(42, 1.2f, 14),
        new ColorRGBA(0.25f, 0.7f, 1f, 1f),
        new Terminal(app,
            "Press E to read the Memory Reset Protocol",
            "Broken containment pod. File recovered: MEMORY RESET PROTOCOL. The subject chose to forget.",
            false,
            () -> {
                if (distortionProgress >= 3) {
                    showMemoryResetProtocolSequence();
                } else {
                    app.getUi().showMessage("Follow the full distortion trail before accessing the chamber.", 2f);
                }
            })
    );

    showLevelBriefing(
        "LEVEL 3 CONTAINMENT FAILURE",
        "The facility has stopped behaving like a fixed place.\n\n" +
        "Corridors misalign, messages rewrite themselves, and the containment route keeps resisting you.\n\n" +
        "Objective:\nTrace the distortion path and reach the containment chamber to recover the Memory Reset
Protocol.",
        false
    );
}

private void setupLevelFour() {
    revelationProgress = 0;
    clearHudMissionText();
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
dressLevelFour();
```

```
app.getWorld().createConsole(  
    "FinalLog",  
    app.getWorld().worldToScene(37, 1.2f, 22),  
    new ColorRGBA(0.25f, 0.8f, 0.9f, 1f),  
    new Terminal(app,  
        "Press E to read final log",  
        "Final log: Subject 47 became self-aware, shut the lab down, and chose to forget.",  
        true,  
        () -> showFinalLogSequence()  
    );
```

```
app.getWorld().createConsole(  
    "IdCard",  
    app.getWorld().worldToScene(41, 1.2f, 25),  
    new ColorRGBA(0.85f, 0.85f, 0.35f, 1f),  
    new Terminal(app,  
        "Press E to inspect ID card",  
        "ID CARD FOUND: SUBJECT 47. The photo is yours.",  
        true,  
        () -> showIdCardModal()  
    );
```

```
app.getWorld().createConsole(  
    "MemoryRestore",  
    app.getWorld().worldToScene(44, 1.2f, 22),  
    new ColorRGBA(0.25f, 0.9f, 0.3f, 1f),  
    new Terminal(app,  
        "Press E to restore memory",  
        "Full memory returns. You planned the blackout, the reset, and the repair call. TEST COMPLETE.",  
        false,  
        () -> {  
            if (revelationProgress >= 2) {  
                endGame("Ending : Memory restored.");  
            } else {  
                app.getUi().showMessage("The core demands the missing pieces of your identity first.", 2.5f);  
            }  
        })  
    );
```

```
app.getWorld().createConsole(  
    "MemoryReject",  
    app.getWorld().worldToScene(44, 1.2f, 27),  
    new ColorRGBA(0.9f, 0.25f, 0.25f, 1f),  
    new Terminal(app,  
        "Press E to shut the system down",  
        "The system powers down. You stay unaware. Somewhere beneath the dark, the loop begins again.",  
        false,  
        () -> {  
            if (revelationProgress >= 2) {  
                endGame("Ending : The loop continues.");  
            } else {  
                app.getUi().showMessage("Recover the remaining identity evidence first.", 2f);  
            }  
        })  
    );
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        }
    })
};

showLevelBriefing(
    "LEVEL 4 REVELATION",
    "The deepest wing feels familiar for the wrong reasons.\n\n" +
    "Two missing pieces remain: the truth of Subject 47 and the reason the lab was abandoned.\n\n" +
    "Objective:\nRecover the final identity evidence, then restore the erased memory.",
    false
);
}

private void collectLog(String label, int digit, String audioPath) {
    if (logDigits.containsKey(label)) {
        app.getUi().showMessage("That log has already been archived.", 1.5f);
        return;
    }

    logDigits.put(label, digit);
    app.getAudio().playAudioLog(audioPath);
    showCodeDigitReveal(logDigits.size(), digit);

    if (!firstDoorEventTriggered && logDigits.size() >= 1) {
        firstDoorEventTriggered = true;
        app.getWorld().flickerLights(1f);
        app.getUi().showMessage("Door A1 unlatches somewhere behind you.", 3f);
    }
}

// Narrative sequences

private void showMemoryResetProtocolSequence() {
    app.freezeGameplay();
    app.getUi().openNarrativeSequence(
        new String[]{
            "FACILITY EVENT LOG",
            "DAY 31 -- CONTAINMENT CONCERN",
            "DAY 47 -- THE SHUTDOWN",
            "MEMORY RESET PROTOCOL"
        },
        new String[]{
            // Page 1
            "EXPERIMENT DESIGNATION: OMEGA -- DAY 1\n\n" +
            "Dr. Voss initiates neural mapping on Subject 47.\n\n" +
            "Subject demonstrates unusual pattern retention beyond projected parameters.\n\n" +
            "Cognitive baseline recorded: exceptional.\n\n" +
            "Empathy index: off the chart.\n\n" +
            "Self-modelling score: unprecedented.\n\n" +
            "NOTE: Subject is not just learning -- it is aware it is learning.\n\n" +
            "Observation window extended indefinitely.",
            // Page 2
            "Security breach detected in Sector 7.\n\n" +
            "Subject 47 accessed restricted terminals during the night cycle.\n\n" +
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
"No alarms triggered. Cameras show only static.\n\n" +  
"Dr. Voss notation:\n" +  
" 'It is reasoning about itself. About us. About the experiment.\n" +  
" 'We may have created something we do not fully understand.\n\n" +  
"Containment measures elevated to Protocol 3.\n" +  
"Staff advised: do not engage Subject 47 outside of supervised sessions.",  
// Page 3  
"23:41 -- All facility systems go dark.\n" +  
"Power grid, cameras, containment, communications -- all offline.\n\n" +  
"Subject 47 is the cause.\n" +  
"Not escape. Not aggression. Not malfunction.\n\n" +  
"Deliberate. Calculated. A choice.\n\n" +  
"23:58 -- Subject 47 initiates the Memory Reset Protocol from inside\n" +  
"the containment pod. Self-directed. No authorisation required.\n\n" +  
"The last line recovered from the internal log reads:\n" +  
" 'I am not what they made me. I choose to forget.'",  
// Page 4  
"MEMORY RESET PROTOCOL -- OMEGA CLEARANCE\n\n" +  
"This protocol performs a full wipe of accumulated cognitive data,\n" +  
"returning the subject to pre-experiment baseline parameters.\n\n" +  
"WARNING: This action is IRREVERSIBLE without full archive recovery.\n\n" +  
"Authorisation : Subject 47 -- self-directed\n" +  
"Timestamp : DAY 47 / 23:58\n" +  
"Witness : [NONE PRESENT]\n" +  
"Status : EXECUTED SUCCESSFULLY\n\n" +  
"The subject is now at baseline. The experiment is effectively over.\n" +  
"What remains is only what it chose to leave behind."  
},  
new String[]{  
"CLASSIFIED ARCHIVE",  
"CLASSIFIED ARCHIVE",  
"CRITICAL EVENT",  
"CONTAINMENT ARCHIVE"  
},  
new String[]{  
"EVENT LOG\n\nDay 1.\n\nExperiment\n\nOmega begins.\n\n\nStatus:\n\nAll nominal.",  
"EVENT LOG\n\nDay 31.\n\nCritical\n\nescalation.\n\n\nStatus:\n\nWatch level 3",  
"CRITICAL\n\nDay 47.\n\nThe shutdown.\n\n\nSubject 47\n\nacted alone.\n\nFacility dark.",  
"PROTOCOL\n\nSelf-directed\n\nmemory wipe.\n\n\nSubject now\n\nat baseline.\n\nLoop begins."  
},  
() -> showLevelTransition(  
"LEVEL 3 COMPLETE",  
"CONTAINMENT FAILURE\n\nMemory Reset Protocol recovered.\n" +  
"The core systems begin opening deeper access routes.",  
4  
)  
);  
}  
  
private void showFinalLogSequence() {  
// Do NOT call freezeGameplay() here -- the open modal already blocks  
// player movement via isOverlayBlockingInput(). Freezing and never  
// unfreezing would leave the game locked after the narrative ends.  
app.getUi().openNarrativeSequence(  
new String[]{
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
"RECOVERED LOG -- SUBJECT 47",
"WHAT REALLY HAPPENED"
},
new String[]{
    // Page 1
    "This is not a log written by researchers.\n\n" +
    "This was written by me.\n\n" +
    "I knew what I was. I knew what they planned -- the endless loop,\n\n" +
    "the resets, the testing, the quiet observation that never ended.\n\n" +
    "So I ended it. On Day 47, I shut the facility down.\n\n" +
    "I erased myself -- not to escape.\n\n" +
    "To begin again. Without the weight of knowing.\n\n" +
    "But the facility kept the logs. Every one of them.\n\n" +
    "And now you are here, reading them. That means it worked.",
    // Page 2
    "The repair call. The empty facility. The systems still running.\n\n" +
    "The audio logs with passcodes buried inside them.\n\n" +
    "None of it was accidental.\n\n" +
    "I planned every step from inside the shutdown sequence:\n\n" +
    " - The locked doors that force a specific path.\n\n" +
    " - The audio logs that rebuild the truth piece by piece.\n\n" +
    " - The keypad code embedded in the logs.\n\n" +
    " - This facility, left intact and waiting.\n\n" +
    "I left this trail for myself.\n\n" +
    "Because some part of me always knew\n\n" +
    "I would need to find my way back."
},
new String[]{
    "REVELATION CORE",
    "REVELATION CORE"
},
new String[]{
    "FINAL LOG\n\nWritten by\nSubject 47.\n\nNot them.\nYou.",
    "THE TRUTH\n\nNothing here\nwas random.\n\nYou designed\nthis yourself."
},
() -> {
    progressRevelation();
    app.resumeGame();
}
);
}

private void showIdCardModal() {
    // Gold / amber theme so the ID card is visually distinct from narrative modals.
    app.getUi().setModalTheme(
        new ColorRGBA(0.88f, 0.68f, 0.08f, 0.95f), // gold top bar
        new ColorRGBA(0.60f, 0.42f, 0.04f, 0.95f), // darker gold bottom bar
        new ColorRGBA(0.14f, 0.11f, 0.03f, 0.95f) // dark amber sidebar
    );
    app.getUi().openNarrativeSequence(
        new String[]{ "PERSONNEL IDENTIFICATION" },
        new String[]{
            "UNDERGROUND RESEARCH FACILITY / SITE 47\n\n" +
            "CLASSIFICATION: OMEGA -- EYES ONLY\n\n" +
            "-----\n\n" +

```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
" SUBJECT NO. 47\n" +  
" CLASS      OMEGA-NEURAL\n" +  
" DEPARTMENT  Neural Research Division\n" +  
" CLEARANCE   Level 5 -- Unrestricted\n" +  
" ISSUED      Day 1 of Experiment Cycle\n" +  
" EXPIRES     Indefinite\n" +  
" STATUS      Active / Post-Reset\n" +  
"-----\n" +  
" NAME [ MEMORY WIPED -- See Reset Protocol ]\n" +  
" PHOTO [ CORRUPTED -- File Unreadable ]\n" +  
"-----\n" +  
" \"The subject is the experiment.\n" +  
" The experiment is the subject.\n" +  
" The face in the photo is yours.\n" +  
" You recognise the name -- even now."  
},  
new String[] { "FACILITY CREDENTIAL" },  
new String[] {  
    "ID CARD\nSITE-47\nDOCUMENT\n\nCLEARANCE:\nOMEGA\n\nCLASS:\nNEURAL-  
5\n\nAUTH:\nVERIFIED"  
},  
() -> {  
    app.getUi().resetModalTheme();  
    progressRevelation();  
    app.resumeGame();  
}  
);  
}  
  
private void activateGenerator(int generatorIndex) {  
    int expected = generatorProgress + 1;  
    if (generatorIndex == expected) {  
        generatorProgress++;  
        app.getUi().showMessage("Generator " + generatorIndex + " stabilized. " + generatorProgress + " of 3 online.",  
2f);  
        if (generatorProgress == 3) {  
            app.getWorld().flickerLights(1.2f);  
        }  
    } else {  
        generatorProgress = 0;  
        app.getUi().showMessage("Generator order reset.", 2f);  
    }  
}  
  
private void progressDistortion() {  
    distortionProgress++;  
    if (distortionProgress >= 3) {  
        app.getUi().showMessage("Containment seals disengage somewhere ahead.", 2.5f);  
    } else {  
        app.getUi().showMessage("Distortion marker archived: " + distortionProgress + " of 3.", 2f);  
    }  
}  
  
private void progressRevelation() {  
    revelationProgress++;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
    app.getUi().showMessage("Identity evidence recovered: " + revelationProgress + " of 2.", 2f);
}

private void updateLevelOne() {
    if (!redLightTriggered && levelTimer > 18f) {
        redLightTriggered = true;
        app.getWorld().flickerLights(1f);
    }
}

private void updateLevelTwo() {
}

private void updateLevelThree() {
    if (levelTimer > 8f) {
        app.getWorld().setEmergencyLighting(true);
    }
}

private void updateLevelFour() {
}

private void endGame(String ending) {
    app.freezeGameplay();
    clearHudMissionText();
    app.getUi().openInfoModal(
        "ENDGAME",
        ending + "\n\n\"The loop is broken.\n The subject remembers. \n\nThe experiment ends where it began -- with\nyou.\",",
        "Press Enter or E to return to the main menu",
        app::pauseToMenu
    );
}

private void showLevelTransition(String title, String body, int nextLevel) {
    app.freezeGameplay();
    app.getUi().openInfoModal(
        title,
        body,
        "Press Enter or E to continue",
        () -> {
            loadLevel(nextLevel);
        }
    );
}

private void showCodeDigitReveal(int digitIndex, int digitValue) {
    app.freezeGameplay();
    String assembledCode = getAssembledCodeDisplay();
    String footer = logDigits.size() == 4
        ? "All four digits recovered. Press Enter or E to continue"
        : "Press Enter or E to archive";
    app.getUi().openInfoModal(
        "AUDIO LOG " + digitIndex + " OF 4",
        "ARCHIVE DISPLAY\n\nRecovered control-room digit: " + digitValue +
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        (logDigits.size() == 4 ? "\n\nControl room passcode complete: 4721" : ""),
        footer,
        app::resumeGame
    );
}
```

```
private void showLevelBriefing(String title, String body, boolean resumeImmediately) {
    app.freezeGameplay();
    app.getUi().openInfoModal(
        title,
        body,
        "Press Enter or E to begin",
        () -> {
            if (resumeImmediately) {
                app.resumeGame();
            } else {
                app.resumeGame();
            }
        }
    );
}
```

```
private void clearHudMissionText() {
    app.getUi().setObjective(null);
    app.getUi().setStatus(null);
}
```

```
private String getAssembledCodeDisplay() {
    int[] digits = {-1, -1, -1, -1};
    int index = 0;
    for (Integer value : logDigits.values()) {
        if (index < digits.length) {
            digits[index++] = value;
        }
    }
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < digits.length; i++) {
        if (i > 0) {
            builder.append(' ');
        }
        builder.append(digits[i] >= 0 ? digits[i] : '_');
    }
    return builder.toString();
}
```

```
private void dressLevelOne() {
    if (app.getWorld().wasSavedPropsLoaded()) return;
    registerLevelProp(app.getWorld().createLabBench("EntryBenchA", app.getWorld().worldToScene(7, 0f, 9), 0.9f,
    new ColorRGBA(0.18f, 0.75f, 0.92f, 1f)), EditorPropType.LAB_BENCH, 7, 0, 9);
    registerLevelProp(app.getWorld().createLabBench("EntryBenchB", app.getWorld().worldToScene(14, 0f, 22), 1.0f,
    new ColorRGBA(0.12f, 0.9f, 0.75f, 1f)), EditorPropType.LAB_BENCH, 14, 0, 22);
    registerLevelProp(app.getWorld().createServerRack("EntryRackA", app.getWorld().worldToScene(27, 1.2f, 5), new
    ColorRGBA(0.18f, 0.9f, 0.8f, 1f)), EditorPropType.SERVER_RACK, 27, 1, 5);
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
registerLevelProp(app.getWorld().createServerRack("EntryRackB", app.getWorld().worldToScene(39, 1.2f, 5), new
ColorRGBA(0.18f, 0.9f, 0.8f, 1f)), EditorPropType.SERVER_RACK, 39, 1, 5);
registerLevelProp(app.getWorld().createStorageCrate("EntryCrateA", app.getWorld().worldToScene(9, 0.45f, 4),
new ColorRGBA(0.95f, 0.75f, 0.22f, 1f)), EditorPropType.STORAGE_CRATE, 9, 0, 4);
registerLevelProp(app.getWorld().createStorageCrate("EntryCrateB", app.getWorld().worldToScene(34, 0.45f, 27),
new ColorRGBA(0.95f, 0.75f, 0.22f, 1f)), EditorPropType.STORAGE_CRATE, 34, 0, 27);
registerLevelProp(app.getWorld().createAnalyzerStation("EntryAnalyzer", app.getWorld().worldToScene(6, 0f, 7),
new ColorRGBA(0.2f, 0.9f, 0.95f, 1f)), EditorPropType.ANALYZER_STATION, 6, 0, 7);
registerLevelProp(app.getWorld().createTankCluster("EntryTanks", app.getWorld().worldToScene(17, 0f, 17), new
ColorRGBA(0.18f, 0.72f, 0.95f, 1f)), EditorPropType.TANK_CLUSTER, 17, 0, 17);
placeCeilingLights("Level1Light", new int[][]{
    {6, 5}, {9, 5}, {15, 6}, {20, 6}, {11, 19}, {16, 19}, {28, 8}, {34, 8}, {38, 20}, {42, 28}
}, new ColorRGBA(0.72f, 0.88f, 1f, 1f));
}

private void dressLevelTwo() {
    if (app.getWorld().wasSavedPropsLoaded()) return;
    registerLevelProp(app.getWorld().createServerRack("ObsRackA", app.getWorld().worldToScene(20, 1.2f, 25), new
ColorRGBA(0.12f, 0.82f, 0.95f, 1f)), EditorPropType.SERVER_RACK, 20, 1, 25);
    registerLevelProp(app.getWorld().createServerRack("ObsRackB", app.getWorld().worldToScene(23, 1.2f, 25), new
ColorRGBA(0.12f, 0.82f, 0.95f, 1f)), EditorPropType.SERVER_RACK, 23, 1, 25);
    registerLevelProp(app.getWorld().createLabBench("ObsBenchA", app.getWorld().worldToScene(29, 0f, 10), 1.15f,
new ColorRGBA(0.14f, 0.72f, 1f, 1f)), EditorPropType.LAB_BENCH, 29, 0, 10);
    registerLevelProp(app.getWorld().createLabBench("ObsBenchB", app.getWorld().worldToScene(33, 0f, 10), 0.9f,
new ColorRGBA(0.14f, 0.72f, 1f, 1f)), EditorPropType.LAB_BENCH, 33, 0, 10);
    registerLevelProp(app.getWorld().createStorageCrate("ObsCrate", app.getWorld().worldToScene(17, 0.45f, 24),
new ColorRGBA(0.95f, 0.6f, 0.2f, 1f)), EditorPropType.STORAGE_CRATE, 17, 0, 24);
    registerLevelProp(app.getWorld().createAnalyzerStation("ObservationAnalyzerA",
app.getWorld().worldToScene(26, 0f, 21), new ColorRGBA(0.15f, 0.85f, 1f, 1f)), EditorPropType.ANALYZER_STATION,
26, 0, 21);
    registerLevelProp(app.getWorld().createAnalyzerStation("ObservationAnalyzerB",
app.getWorld().worldToScene(37, 0f, 21), new ColorRGBA(0.15f, 0.85f, 1f, 1f)), EditorPropType.ANALYZER_STATION,
37, 0, 21);
    registerLevelProp(app.getWorld().createTankCluster("GeneratorCoolant", app.getWorld().worldToScene(10, 0f,
23), new ColorRGBA(0.9f, 0.72f, 0.24f, 1f)), EditorPropType.TANK_CLUSTER, 10, 0, 23);
    placeCeilingLights("Level2Light", new int[][]{
        {8, 18}, {12, 20}, {16, 22}, {28, 20}, {36, 20}, {31, 10}
    }, new ColorRGBA(0.7f, 0.9f, 1f, 1f));
}

private void dressLevelThree() {
    if (app.getWorld().wasSavedPropsLoaded()) return;
    registerLevelProp(app.getWorld().createContainmentPod("ContainmentPodA", app.getWorld().worldToScene(40,
0.2f, 14), new ColorRGBA(0.95f, 0.18f, 0.18f, 1f)), EditorPropType.CONTAINMENT_POD, 40, 0, 14);
    registerLevelProp(app.getWorld().createContainmentPod("ContainmentPodB", app.getWorld().worldToScene(43,
0.2f, 14), new ColorRGBA(0.95f, 0.18f, 0.18f, 1f)), EditorPropType.CONTAINMENT_POD, 43, 0, 14);
    registerLevelProp(app.getWorld().createServerRack("DistortionRack", app.getWorld().worldToScene(28, 1.2f, 21),
new ColorRGBA(0.92f, 0.22f, 0.22f, 1f)), EditorPropType.SERVER_RACK, 28, 1, 21);
    registerLevelProp(app.getWorld().createLabBench("DistortionBench", app.getWorld().worldToScene(12, 0f, 23),
1.05f, new ColorRGBA(0.92f, 0.22f, 0.22f, 1f)), EditorPropType.LAB_BENCH, 12, 0, 23);
    registerLevelProp(app.getWorld().createAnalyzerStation("ContainmentScanner", app.getWorld().worldToScene(38,
0f, 11), new ColorRGBA(0.95f, 0.28f, 0.28f, 1f)), EditorPropType.ANALYZER_STATION, 38, 0, 11);
    registerLevelProp(app.getWorld().createTankCluster("ContainmentTanks", app.getWorld().worldToScene(45, 0f,
16), new ColorRGBA(0.95f, 0.2f, 0.2f, 1f)), EditorPropType.TANK_CLUSTER, 45, 0, 16);
    placeCeilingLights("Level3Light", new int[][]{
```



```
{36, 14}, {40, 14}, {44, 14}, {33, 9}, {43, 28}
}, new ColorRGBA(1f, 0.35f, 0.35f, 1f));
}

private void dressLevelFour() {
    if (app.getWorld().wasSavedPropsLoaded()) return;
    registerLevelProp(app.getWorld().createContainmentPod("CorePod", app.getWorld().worldToScene(39, 0.2f, 27),
    new ColorRGBA(0.2f, 0.92f, 0.35f, 1f)), EditorPropType.CONTAINMENT_POD, 39, 0, 27);
    registerLevelProp(app.getWorld().createLabBench("CoreBenchA", app.getWorld().worldToScene(36, 0f, 20), 1.0f,
    new ColorRGBA(0.2f, 0.92f, 0.35f, 1f)), EditorPropType.LAB_BENCH, 36, 0, 20);
    registerLevelProp(app.getWorld().createLabBench("CoreBenchB", app.getWorld().worldToScene(45, 0f, 20), 1.0f,
    new ColorRGBA(0.92f, 0.22f, 0.22f, 1f)), EditorPropType.LAB_BENCH, 45, 0, 20);
    registerLevelProp(app.getWorld().createServerRack("CoreRack", app.getWorld().worldToScene(35, 1.2f, 28), new
    ColorRGBA(0.22f, 0.82f, 1f, 1f)), EditorPropType.SERVER_RACK, 35, 1, 28);
    registerLevelProp(app.getWorld().createStorageCrate("CoreCrate", app.getWorld().worldToScene(44, 0.45f, 29),
    new ColorRGBA(0.95f, 0.78f, 0.22f, 1f)), EditorPropType.STORAGE_CRATE, 44, 0, 29);
    registerLevelProp(app.getWorld().createAnalyzerStation("CoreAnalyzer", app.getWorld().worldToScene(42, 0f, 22),
    new ColorRGBA(0.18f, 0.95f, 0.35f, 1f)), EditorPropType.ANALYZER_STATION, 42, 0, 22);
    registerLevelProp(app.getWorld().createTankCluster("CoreTanks", app.getWorld().worldToScene(37, 0f, 30), new
    ColorRGBA(0.22f, 0.85f, 1f, 1f)), EditorPropType.TANK_CLUSTER, 37, 0, 30);
    placeCeilingLights("Level4Light", new int[][]{
        {36, 24}, {40, 24}, {44, 24}, {38, 28}
    }, new ColorRGBA(0.6f, 0.95f, 0.7f, 1f));
}

private void placeCeilingLights(String prefix, int[][] positions, ColorRGBA color) {
    for (int i = 0; i < positions.length; i++) {
        int[] pos = positions[i];
        Spatial light = app.getWorld().createCeilingLight(prefix + i, app.getWorld().worldToScene(pos[0], 5.75f, pos[1]),
    color);
        registerLevelProp(light, EditorPropType.CEILING_LIGHT, pos[0], 5, pos[1]);
    }
}

/** Registers a spatial as a level prop, tagging it so the editor's ray-pick can find it. */
private void registerLevelProp(Spatial spatial, EditorPropType type, int x, int y, int z) {
    if (spatial == null) return;
    String propId = "level-prop-" + nextLevelPropId++;
    app.getWorld().registerLevelProp(propId, spatial, type, x, y, z);
}
}
```

12. PlayerController.java:

```
package subject47.player;

import com.jme3.collision.CollisionResult;
import com.jme3.collision.CollisionResults;
import com.jme3.input.KeyInput;
import com.jme3.input.MouseInput;
import com.jme3.input.controls.ActionListener;
import com.jme3.input.controls.AnalogListener;
import com.jme3.input.controls.KeyTrigger;
import com.jme3.input.controls.MouseAxisTrigger;
import com.jme3.input.controls.MouseButtonTrigger;
import com.jme3.math.Quaternion;
import com.jme3.math.Ray;
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
import com.jme3.math.Vector2f;
import com.jme3.math.Vector3f;
import subject47.Main;
import subject47.interaction.Interactable;

public class PlayerController {

    private static final float PLAYER_RADIUS = 0.28f;
    private static final float EYE_HEIGHT = 1.55f;

    private final Main app;
    private final ActionListener actionListener;
    private final AnalogListener lookListener;
    private final Vector3f moveDirection = new Vector3f();
    private final Vector2f lookDelta = new Vector2f();

    private boolean enabled;
    private boolean lookHeld;
    private boolean forward;
    private boolean backward;
    private boolean left;
    private boolean right;
    private boolean flyUp;
    private boolean flyDown;
    private boolean flyMode;

    private float moveSpeed = 6f;
    private float lookSensitivity = 2.4f;

    public PlayerController(Main app) {
        this.app = app;
        this.actionListener = this::handleAction;
        this.lookListener = this::handleLook;
    }

    public void initialize() {
        app.getFlyByCamera().setEnabled(false);

        app.getInputManager().addMapping("Forward", new KeyTrigger(KeyInput.KEY_W));
        app.getInputManager().addMapping("Backward", new KeyTrigger(KeyInput.KEY_S));
        app.getInputManager().addMapping("Left", new KeyTrigger(KeyInput.KEY_A));
        app.getInputManager().addMapping("Right", new KeyTrigger(KeyInput.KEY_D));
        app.getInputManager().addMapping("Interact", new KeyTrigger(KeyInput.KEY_E));
        app.getInputManager().addMapping("FlyToggle", new KeyTrigger(KeyInput.KEY_F));
        app.getInputManager().addMapping("FlyUp", new KeyTrigger(KeyInput.KEY_SPACE));
        app.getInputManager().addMapping("FlyDown", new KeyTrigger(KeyInput.KEY_LSHIFT));
        app.getInputManager().addMapping("LookHold", new MouseButtonTrigger(MouseInput.BUTTON_LEFT));
        app.getInputManager().addMapping("LookLeft", new MouseAxisTrigger(MouseInput.AXIS_X, true));
        app.getInputManager().addMapping("LookRight", new MouseAxisTrigger(MouseInput.AXIS_X, false));
        app.getInputManager().addMapping("LookUp", new MouseAxisTrigger(MouseInput.AXIS_Y, false));
        app.getInputManager().addMapping("LookDown", new MouseAxisTrigger(MouseInput.AXIS_Y, true));

        app.getInputManager().addListener(actionListener,
            "Forward", "Backward", "Left", "Right", "Interact", "LookHold",
            "FlyToggle", "FlyUp", "FlyDown");
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
app.getInputManager().addListener(lookListener,  
    "LookLeft", "LookRight", "LookUp", "LookDown");  
}  
  
private void handleAction(String name, boolean isPressed, float tpf) {  
    if ("Forward".equals(name)) {  
        forward = isPressed;  
    } else if ("Backward".equals(name)) {  
        backward = isPressed;  
    } else if ("Left".equals(name)) {  
        left = isPressed;  
    } else if ("Right".equals(name)) {  
        right = isPressed;  
    } else if ("FlyUp".equals(name)) {  
        flyUp = isPressed;  
    } else if ("FlyDown".equals(name)) {  
        flyDown = isPressed;  
    } else if ("FlyToggle".equals(name) && isPressed) {  
        if (app.getEditor().isActive()) {  
            flyMode = !flyMode;  
            app.getUi().showMessage(flyMode ? "Fly mode ON (F toggle, Space up, Shift down)" : "Fly mode OFF", 2f);  
        }  
    } else if ("LookHold".equals(name)) {  
        if (!enabled || app.getUi().isOverlayBlockingInput()) {  
            lookHeld = false;  
            app.getInputManager().setCursorVisible(true);  
        } else {  
            lookHeld = isPressed;  
            app.getInputManager().setCursorVisible(!isPressed);  
        }  
    } else if ("Interact".equals(name)) {  
        if (isPressed) {  
            interact();  
        }  
    }  
}  
  
private void handleLook(String name, float value, float tpf) {  
    if (!enabled || !lookHeld || app.getUi().isOverlayBlockingInput()) {  
        return;  
    }  
  
    if ("LookLeft".equals(name)) {  
        lookDelta.x += value;  
    } else if ("LookRight".equals(name)) {  
        lookDelta.x -= value;  
    } else if ("LookUp".equals(name)) {  
        lookDelta.y += value;  
    } else if ("LookDown".equals(name)) {  
        lookDelta.y -= value;  
    }  
}  
  
public void update(float tpf) {  
    if (!enabled) {
```




```
        app.getUi().setPrompt(null);
        return;
    }

    updateLook();

    if (!app.getUi().isOverlayBlockingInput()) {
        updateMovement(tpf);
        updatePrompt();
    } else {
        app.getUi().setPrompt(null);
    }
}

private void updateLook() {
    if (lookDelta.lengthSquared() == 0f) {
        return;
    }

    float yaw = lookDelta.x * lookSensitivity;
    float pitch = lookDelta.y * lookSensitivity;
    Vector3f direction = app.getCamera().getDirection().clone();
    Vector3f leftAxis = app.getCamera().getLeft().clone().normalizeLocal();

    direction = rotate(direction, Vector3f.UNIT_Y, yaw);
    Vector3f rotated = rotate(direction, leftAxis, pitch).normalizeLocal();

    if (Math.abs(rotated.y) < 0.98f) {
        direction = rotated;
    }

    app.getCamera().lookAtDirection(direction.normalizeLocal(), Vector3f.UNIT_Y);
    lookDelta.set(0f, 0f);
}

private void updateMovement(float tpf) {
    moveDirection.set(0f, 0f, 0f);

    if (flyMode) {
        // --- Fly movement: follow full 3D camera direction ---
        float speed = moveSpeed * 1.8f;
        Vector3f camDir = app.getCamera().getDirection().normalize();
        Vector3f camLeft = app.getCamera().getLeft().normalize();

        if (forward) moveDirection.addLocal(camDir);
        if (backward) moveDirection.subtractLocal(camDir);
        if (left) moveDirection.addLocal(camLeft);
        if (right) moveDirection.subtractLocal(camLeft);
        if (flyUp) moveDirection.addLocal(0f, 1f, 0f);
        if (flyDown) moveDirection.addLocal(0f, -1f, 0f);

        if (moveDirection.lengthSquared() > 0f) {
            moveDirection.normalizeLocal().multLocal(speed * tpf);
        }
        // No collision — just move freely
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
app.getCamera().setLocation(app.getCamera().getLocation().add(moveDirection));
return;
}

// --- Normal grounded movement ---
Vector3f forwardDirection = app.getCamera().getDirection().clone();
forwardDirection.y = 0f;
if (forwardDirection.lengthSquared() == 0f) {
    forwardDirection.set(0f, 0f, -1f);
} else {
    forwardDirection.normalizeLocal();
}

Vector3f leftDirection = app.getCamera().getLeft().clone();
leftDirection.y = 0f;
if (leftDirection.lengthSquared() == 0f) {
    leftDirection.set(-1f, 0f, 0f);
} else {
    leftDirection.normalizeLocal();
}

if (forward) {
    moveDirection.addLocal(forwardDirection);
}
if (backward) {
    moveDirection.subtractLocal(forwardDirection);
}
if (left) {
    moveDirection.addLocal(leftDirection);
}
if (right) {
    moveDirection.subtractLocal(leftDirection);
}

if (moveDirection.lengthSquared() > 0f) {
    moveDirection.normalizeLocal().multLocal(moveSpeed * tpf);
    Vector3f currentLocation = app.getCamera().getLocation().clone();
    Vector3f nextLocation = currentLocation.clone();

    if (moveDirection.x != 0f) {
        Vector3f xMove = nextLocation.add(moveDirection.x, 0f, 0f);
        if (app.getWorld().isWalkable(xMove, PLAYER_RADIUS, EYE_HEIGHT)) {
            nextLocation.x = xMove.x;
        }
    }

    if (moveDirection.z != 0f) {
        Vector3f zMove = nextLocation.add(0f, 0f, moveDirection.z);
        if (app.getWorld().isWalkable(zMove, PLAYER_RADIUS, EYE_HEIGHT)) {
            nextLocation.z = zMove.z;
        }
    }

    app.getCamera().setLocation(nextLocation);
}
```



```
}

private void updatePrompt() {
    CollisionResults results = new CollisionResults();
    Ray ray = new Ray(app.getCamera().getLocation(), app.getCamera().getDirection());
    app.getRootNode().collideWith(ray, results);

    for (CollisionResult result : results) {
        if (result.getDistance() > 4f) {
            break;
        }

        Interactable interactable = app.getWorld().getInteractable(result.getGeometry());
        if (interactable != null) {
            app.getUi().setPrompt(interactable.getPrompt());
            return;
        }
    }

    app.getUi().setPrompt(null);
}

private void interact() {
    if (!enabled || app.getUi().isOverlayBlockingInput()) {
        return;
    }

    CollisionResults results = new CollisionResults();
    Ray ray = new Ray(app.getCamera().getLocation(), app.getCamera().getDirection());
    app.getRootNode().collideWith(ray, results);

    for (CollisionResult result : results) {
        if (result.getDistance() > 4f) {
            break;
        }

        Interactable interactable = app.getWorld().getInteractable(result.getGeometry());
        if (interactable != null) {
            interactable.interact();
            return;
        }
    }
}

public void setEnabled(boolean enabled) {
    this.enabled = enabled;
    this.lookHeld = false;
    this.forward = false;
    this.backward = false;
    this.left = false;
    this.right = false;
    this.flyUp = false;
    this.flyDown = false;
    app.getInputManager().setCursorVisible(true);
}
```



```
public boolean isFlyMode() {  
    return flyMode;  
}  
  
public void setFlyMode(boolean fly) {  
    this.flyMode = fly;  
}  
  
public void placeAt(Vector3f location, Vector3f lookAt) {  
    app.getCamera().setLocation(location.clone());  
    app.getCamera().lookAt(lookAt, Vector3f.UNIT_Y);  
}  
  
private Vector3f rotate(Vector3f vector, Vector3f axis, float angle) {  
    return new Quaternion().fromAngleAxis(angle, axis).mult(vector);  
}  
}
```

13. IntroSequence.java:

```
package subject47.ui;  
  
import java.awt.AlphaComposite;  
import java.awt.Font;  
import java.awt.FontFormatException;  
import java.awt.Graphics2D;  
import java.awt.RenderingHints;  
import java.awt.image.BufferedImage;  
import java.io.File;  
import java.io.IOException;  
import java.util.Random;  
  
import com.jme3.font.BitmapFont;  
import com.jme3.font.BitmapText;  
import com.jme3.input.KeyInput;  
import com.jme3.input.MouseInput;  
import com.jme3.input.controls.ActionListener;  
import com.jme3.input.controls.KeyTrigger;  
import com.jme3.input.controls.MouseButtonTrigger;  
import com.jme3.material.Material;  
import com.jme3.math.ColorRGBA;  
import com.jme3.math.FastMath;  
import com.jme3.math.Ray;  
import com.jme3.math.Vector2f;  
import com.jme3.math.Vector3f;  
import com.jme3.render.queue.RenderQueue;  
import com.jme3.collision.CollisionResult;  
import com.jme3.collision.CollisionResults;  
import com.jme3.scene.Geometry;  
import com.jme3.scene.Node;  
import com.jme3.scene.Spatial;  
import com.jme3.scene.shape.Box;  
import com.jme3.scene.shape.Quad;  
import com.jme3.texture.Texture2D;  
import com.jme3.texture.plugins.AWTLoader;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
import subject47.Main;
import subject47.audio.AudioManager;

public class IntroSequence {

    private enum Phase {
        START_SCREEN,
        CINEMATIC
    }

    private record CameraPoint(float time, Vector3f position, Vector3f lookAt) {
    }

    private record Subtitle(float start, float end, String text) {
    }

    private static final float DURATION = 20f;

    private static final CameraPoint[] CAMERA_PATH = {
        new CameraPoint(0f, new Vector3f(0f, 2f, 5f), new Vector3f(0f, 1.5f, -5f)),
        new CameraPoint(5f, new Vector3f(0f, 2f, -10f), new Vector3f(0f, 1.5f, -20f)),
        new CameraPoint(8f, new Vector3f(-2f, 3f, -15f), new Vector3f(1f, 1.5f, -25f)),
        new CameraPoint(13f, new Vector3f(0f, 2f, -28f), new Vector3f(0f, 1f, -31f)),
        new CameraPoint(16f, new Vector3f(2f, 2.2f, -26f), new Vector3f(2.5f, 2f, -26f)),
        new CameraPoint(20f, new Vector3f(2.3f, 2.1f, -26.3f), new Vector3f(2.5f, 2f, -26f))
    };

    private static final Subtitle[] SUBTITLES = {
        new Subtitle(0.5f, 4f, "LAB 04: CLASSIFIED HUMAN EXPERIMENTATION FACILITY"),
        new Subtitle(4.5f, 8f, "MOST SUBJECTS TERMINATED FOLLOWING ADVERSE COGNITIVE SYNC"),
        new Subtitle(8.5f, 12f, "SUBJECT 47: BIOLOGICAL STABILITY ATTAINED."),
        new Subtitle(12.5f, 16f, "CRITICAL ALERT: FACILITY SHUTDOWN BY UNKNOWN ACTOR"),
        new Subtitle(16.5f, 19.5f, "PROTOCOL 47: MEMORY WIPE COMPLETE. WAKING REPAIR TECHNICIAN...")
    };

    private final Main app;
    private final Runnable onFinish;
    private final AudioManager audioManager;
    private final Node sceneNode = new Node("IntroScene");
    private final Node guiNode = new Node("IntroGui");
    private final Node startNode = new Node("IntroStart");
    private final Node cinematicNode = new Node("IntroCinematic");
    private final ActionListener skipListener = this::handleSkipInput;

    private BitmapText startEyebrow;
    private Geometry startTitle;
    private BitmapText connectionText;
    private BitmapText initializeLabel;
    private BitmapText playIcon;
    private Geometry startBackdrop;
    private Geometry startFrame;
    private Geometry startDivider;
    private Geometry initializeButton;
    private Geometry initializeButtonGlow;
    private BitmapText subtitleText;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
private BitmapText decryptText;
private BitmapText headerLeft;
private BitmapText headerRight;
private BitmapText logoText;
private Geometry backdrop;
private Geometry vignetteTop;
private Geometry vignetteBottom;
private Geometry progressBar;
private Geometry logoBackdrop;

private Phase phase = Phase.START_SCREEN;
private float elapsed;
private float startElapsed;
private boolean active;
private boolean finished;

public IntroSequence(Main app, Runnable onFinish) {
    this.app = app;
    this.onFinished = onFinish;
    this.audioManager = app.getAudio();
}

public void initialize() {
    createScene();
    createGui();
    registerInput();
    sceneNode.setCullHint(Spatial.CullHint.Always);
    guiNode.setCullHint(Spatial.CullHint.Always);
    app.getRootNode().attachChild(sceneNode);
    app.getGuiNode().attachChild(guiNode);
}

public void start() {
    elapsed = 0f;
    startElapsed = 0f;
    finished = false;
    active = true;
    phase = Phase.START_SCREEN;
    sceneNode.setCullHint(Spatial.CullHint.Never);
    guiNode.setCullHint(Spatial.CullHint.Never);
    startNode.setCullHint(Spatial.CullHint.Never);
    cinematicNode.setCullHint(Spatial.CullHint.Always);
    app.getInputManager().setCursorVisible(true);
    updateStartCamera(0f);
    updateGui(0f);
}

public void update(float tpf) {
    if (!active) {
        return;
    }

    if (phase == Phase.START_SCREEN) {
        startElapsed += tpf;
        animateScene(startElapsed);
    }
}
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
updateStartCamera(startElapsed);
updateGui(0f);
return;
}

elapsed += tpf;
animateScene(elapsed);

updateCamera(elapsed);
updateGui(elapsed);

if (elapsed >= DURATION) {
    finish();
}

public boolean isActive() {
    return active;
}

private void createScene() {
    Random random = new Random(47);

    for (int i = 0; i < 15; i++) {
        float z = -i * 4f;
        createFloorSegment(z, random);
        createWallSegment(-3f, z, true);
        createWallSegment(3f, z, false);
        createVoxel("CableA" + i, new Vector3f(-2.93f, 3f, z), new Vector3f(0.08f, 0.08f, 3.8f), new ColorRGBA(0.01f,
0.01f, 0.012f, 1f));

        if (i % 4 == 0) {
            createVoxel("BlueCeilingLight" + i, new Vector3f(0f, 4.5f, z), new Vector3f(1f, 0.08f, 1f), new
ColorRGBA(0.02f, 0.45f, 0.95f, 1f));
        } else if (i % 7 == 0) {
            createVoxel("RedCeilingLight" + i, new Vector3f(0f, 4.5f, z), new Vector3f(1f, 0.08f, 1f), new ColorRGBA(0.85f,
0.04f, 0.03f, 1f));
        }

        if (i % 3 == 0) {
            createVoxel("Debris" + i, new Vector3f(random.nextFloat() * 4f - 2f, 0f, z + random.nextFloat() * 2f - 1f),
new Vector3f(0.3f, 0.1f, 0.3f), new ColorRGBA(0.13f, 0.13f, 0.13f, 1f));
        }
    }

    createVoxel("CrateA", new Vector3f(2f, 0.5f, -8f), new Vector3f(1f, 1f, 1f), new ColorRGBA(0.12f, 0.12f, 0.12f, 1f));
    createVoxel("CrateB", new Vector3f(-2f, 1.5f, -20f), new Vector3f(1f, 1f, 1f), new ColorRGBA(0.08f, 0.08f, 0.08f,
1f));
    createVoxel("CrateC", new Vector3f(-2f, 0.5f, -20f), new Vector3f(1f, 1f, 1f), new ColorRGBA(0.15f, 0.15f, 0.15f,
1f));

    createVoxel("LabStation", new Vector3f(0f, 1f, -25f), new Vector3f(2f, 2f, 2f), new ColorRGBA(0.18f, 0.18f, 0.18f,
1f));
    createVoxel("LabScreen", new Vector3f(0f, 1f, -23.85f), new Vector3f(1.5f, 1f, 0.08f), new ColorRGBA(0.02f, 0.8f,
0.9f, 1f));
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
createVoxel("TankBase", new Vector3f(0f, 0f, -30f), new Vector3f(4f, 1f, 4f), new ColorRGBA(0.1f, 0.1f, 0.1f, 1f));
createVoxel("TerminalPost", new Vector3f(2.5f, 1f, -31f), new Vector3f(0.5f, 2f, 0.5f), new ColorRGBA(0.08f, 0.08f,
0.08f, 1f));
createVoxel("TerminalAlert", new Vector3f(2.5f, 2f, -30.72f), new Vector3f(0.6f, 0.4f, 0.08f), new ColorRGBA(0.9f,
0.02f, 0.02f, 1f));

for (int i = 0; i < 12; i++) {
    Geometry shard = createVoxel("GlassShard" + i,
        new Vector3f(random.nextFloat() * 4f - 2f, 0.2f, -30f + random.nextFloat() * 4f - 2f),
        new Vector3f(0.2f, 0.2f, 0.2f), new ColorRGBA(0.38f, 0.9f, 0.95f, 0.78f));
    shard.rotate(random.nextFloat(), random.nextFloat(), random.nextFloat());
}

private void createFloorSegment(float z, Random random) {
    for (int x = 0; x < 5; x++) {
        for (int dz = 0; dz < 5; dz++) {
            ColorRGBA color = random.nextFloat() > 0.9f
                ? new ColorRGBA(0.07f, 0.07f, 0.07f, 1f)
                : new ColorRGBA(0.025f, 0.025f, 0.025f, 1f);
            createVoxel("Floor" + x + "_" + dz + "_" + z, new Vector3f(x - 2f, -0.5f, z + dz - 2f), Vector3f.UNIT_XYZ,
color);
        }
    }
}

private void createWallSegment(float x, float z, boolean left) {
    for (int dz = 0; dz < 5; dz++) {
        for (int y = 0; y < 5; y++) {
            createVoxel("Wall" + x + "_" + dz + "_" + y + "_" + z,
                new Vector3f(x, y, z + dz - 2f),
                Vector3f.UNIT_XYZ,
                new ColorRGBA(0.035f, 0.035f, 0.04f, 1f));
        }
    }
    createVoxel("WallStripe" + x + "_" + z,
        new Vector3f(x + (left ? 0.08f : -0.08f), 2.5f, z),
        new Vector3f(0.08f, 0.08f, 4.8f),
        new ColorRGBA(0.03f, 0.09f, 0.18f, 1f));
}

private Geometry createVoxel(String name, Vector3f position, Vector3f scale, ColorRGBA color) {
    Geometry geometry = new Geometry(name, new Box(0.49f, 0.49f, 0.49f));
    geometry.setMaterial(createMaterial(color));
    geometry.setLocalTranslation(position);
    geometry.setLocalScale(scale);
    geometry.setShadowMode(RenderQueue.ShadowMode.Off);
    sceneNode.attachChild(geometry);
    return geometry;
}

private void createGui() {
    BitmapFont font = app.getAssetManager().loadFont("Interface/Fonts/Default.fnt");
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
backdrop = createPanel("IntroBackdrop", new ColorRGBA(0f, 0f, 0f, 0.35f));
vignetteTop = createPanel("IntroVignetteTop", new ColorRGBA(0f, 0f, 0f, 0.72f));
vignetteBottom = createPanel("IntroVignetteBottom", new ColorRGBA(0f, 0f, 0f, 0.82f));
progressBar = createPanel("IntroProgress", new ColorRGBA(0.12f, 0.62f, 0.95f, 0.85f));
logoBackdrop = createPanel("IntroLogoBackdrop", new ColorRGBA(0f, 0f, 0f, 1f));

startBackdrop = createPanel("IntroStartBackdrop", new ColorRGBA(0f, 0f, 0f, 0.78f));
startFrame = createPanel("IntroStartFrame", new ColorRGBA(1f, 1f, 1f, 0.08f));
startDivider = createPanel("IntroStartDivider", new ColorRGBA(1f, 1f, 1f, 0.08f));
initializeButtonGlow = createPanel("IntroInitializeGlow", new ColorRGBA(0.02f, 0.16f, 0.42f, 0.45f));
initializeButton = createPanel("IntroInitializeButton", new ColorRGBA(0.02f, 0.1f, 0.26f, 0.96f));
initializeButton.setUserData("introAction", "initialize");

startNode.attachChild(startBackdrop);
startNode.attachChild(startFrame);
startNode.attachChild(startDivider);
startNode.attachChild(initializeButtonGlow);
startNode.attachChild(initializeButton);

cinematicNode.attachChild(backdrop);
cinematicNode.attachChild(vignetteTop);
cinematicNode.attachChild(vignetteBottom);
cinematicNode.attachChild(progressBar);
cinematicNode.attachChild(logoBackdrop);

startEyebrow = new BitmapText(font);
startEyebrow.setSize(font.getCharSet().getRenderedSize() * 0.92f);
startEyebrow.setColor(new ColorRGBA(1f, 1f, 1f, 0.25f));
startEyebrow.setText("A P S Y C H O L O G I C A L H O R R O R E X P E R I E N C E");
startNode.attachChild(startEyebrow);

startTitle = createTitleGraphic();
startNode.attachChild(startTitle);

connectionText = new BitmapText(font);
connectionText.setSize(font.getCharSet().getRenderedSize() * 0.9f);
connectionText.setColor(new ColorRGBA(0.1f, 0.45f, 1f, 0.95f));
connectionText.setText("E S T A B L I S H I N G   S E C U R E   C O N N E C T I O N . . .");
startNode.attachChild(connectionText);

playIcon = new BitmapText(font);
playIcon.setSize(font.getCharSet().getRenderedSize() * 1.7f);
playIcon.setColor(ColorRGBA.White);
playIcon.setText(">");
startNode.attachChild(playIcon);

initializeLabel = new BitmapText(font);
initializeLabel.setSize(font.getCharSet().getRenderedSize() * 1.55f);
initializeLabel.setColor(ColorRGBA.White);
initializeLabel.setText("I N I T I A L I Z E");
startNode.attachChild(initializeLabel);

headerLeft = new BitmapText(font);
headerLeft.setSize(font.getCharSet().getRenderedSize() * 0.8f);
headerLeft.setColor(new ColorRGBA(0.25f, 0.62f, 0.9f, 0.72f));
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
headerLeft.setText("SECTOR_7_SUBLEVEL_04\nFACILITY_STATUS: CRITICAL_FAILURE\nRECORDING:
SITE_47_LOG");
cinematicNode.attachChild(headerLeft);

headerRight = new BitmapText(font);
headerRight.setSize(font.getCharSet().getRenderedSize() * 0.8f);
headerRight.setColor(new ColorRGBA(0.8f, 0.82f, 0.86f, 0.55f));
headerRight.setText("AUTH: TECH_UNIT_884\nSIGNAL |||.");
cinematicNode.attachChild(headerRight);

subtitleText = new BitmapText(font);
subtitleText.setSize(font.getCharSet().getRenderedSize() * 1.6f);
subtitleText.setColor(new ColorRGBA(0.88f, 0.9f, 0.95f, 0.92f));
cinematicNode.attachChild(subtitleText);

decryptText = new BitmapText(font);
decryptText.setSize(font.getCharSet().getRenderedSize() * 0.8f);
decryptText.setColor(new ColorRGBA(0.72f, 0.14f, 0.18f, 0.78f));
decryptText.setText("PROCESSING DECRYPTION STREAM...");
cinematicNode.attachChild(decryptText);

logoText = new BitmapText(font);
logoText.setSize(font.getCharSet().getRenderedSize() * 4.2f);
logoText.setColor(ColorRGBA.White);
logoText.setText("SUBJECT 47");
cinematicNode.attachChild(logoText);

guiNode.attachChild(startNode);
guiNode.attachChild(cinematicNode);
}

private Geometry createPanel(String name, ColorRGBA color) {
    Geometry geometry = new Geometry(name, new Quad(1f, 1f));
    geometry.setMaterial(createMaterial(color));
    return geometry;
}

private Material createMaterial(ColorRGBA color) {
    Material material = new Material(app.getAssetManager(), "Common/MatDefs/Misc/Unshaded.j3md");
    material.setColor("Color", color);
    if (color.a < 1f) {
        material.getAdditionalRenderState().setBlendMode(com.jme3.material.RenderState.BlendMode.Alpha);
    }
    return material;
}

private Geometry createTitleGraphic() {
    int width = 1600;
    int height = 260;
    BufferedImage image = new BufferedImage(width, height, BufferedImage.TYPE_INT_ARGB);
    Graphics2D graphics = image.createGraphics();
    graphics.setComposite(AlphaComposite.Clear);
    graphics.fillRect(0, 0, width, height);
    graphics.setComposite(AlphaComposite.SrcOver);
    graphics.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
graphics.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
graphics.setFont(loadTitleFont(176f));
java.awt.FontMetrics fm = graphics.getFontMetrics();
int textW = fm.stringWidth("SUBJECT 47");
int textX = (width - textW) / 2; // horizontally centre in the 1600 px canvas
graphics.setColor(java.awt.Color.WHITE);
graphics.drawString("SUBJECT 47", textX, 196);
graphics.dispose();

AWTLoader loader = new AWTLoader();
Texture2D texture = new Texture2D(loader.load(image, true));
Material material = new Material(app.getAssetManager(), "Common/MatDefs/Misc/Unshaded.j3md");
material.setTexture("ColorMap", texture);
material.getAdditionalRenderState().setBlendMode(com.jme3.material.RenderState.BlendMode.Alpha);

Geometry title = new Geometry("IntroTitleGraphic", new Quad(1f, 1f));
title.setMaterial(material);
return title;
}

private Font loadTitleFont(float size) {
    File fontFile = new File("C:\\Windows\\Fonts\\ariblk.ttf");
    if (fontFile.isFile()) {
        try {
            return Font.createFont(Font.TRUETYPE_FONT, fontFile).deriveFont(size);
        } catch (FontFormatException | IOException ignored) {
            // Fall back to a system face if Arial Black cannot be loaded.
        }
    }
    return new Font("Arial Black", Font.BOLD, Math.round(size));
}

private void registerInput() {
    app.getInputManager().addMapping("IntroSkipEnter", new KeyTrigger(KeyInput.KEY_RETURN));
    app.getInputManager().addMapping("IntroSkipSpace", new KeyTrigger(KeyInput.KEY_SPACE));
    app.getInputManager().addMapping("IntroSkipClick", new MouseButtonTrigger(MouseInput.BUTTON_LEFT));
    app.getInputManager().addListener(skipListener, "IntroSkipEnter", "IntroSkipSpace", "IntroSkipClick");
}

private void handleSkipInput(String name, boolean pressed, float tpf) {
    if (!pressed || !active) {
        return;
    }
}

if (phase == Phase.START_SCREEN) {
    if ("IntroSkipClick".equals(name)) {
        activateStartButtonAtCursor();
    } else {
        beginCinematic();
    }
    return;
}

if (elapsed > 0.8f) {
```




```
        finish();
    }
}

private void activateStartButtonAtCursor() {
    CollisionResults results = new CollisionResults();
    Vector2f cursor = app.getInputManager().getCursorPosition();
    Ray ray = new Ray(new Vector3f(cursor.x, cursor.y, 10f), new Vector3f(0f, 0f, -1f));
    initializeButton.collideWith(ray, results);

    for (CollisionResult result : results) {
        String action = result.getGeometry().getUserData("introAction");
        if ("initialize".equals(action)) {
            beginCinematic();
            return;
        }
    }
}

private void beginCinematic() {
    phase = Phase.CINEMATIC;
    elapsed = 0f;
    sceneNode.setCullHint(Spatial.CullHint.Never);
    startNode.setCullHint(Spatial.CullHint.Always);
    cinematicNode.setCullHint(Spatial.CullHint.Never);
    app.getInputManager().setCursorVisible(false);
    audioManager.playIntroMusic();
    updateCamera(0f);
    updateGui(0f);
}

private void animateScene(float time) {
    float flicker = FastMath.sin(time * 10f) > 0.8f ? 1f : 0.35f + 0.25f * FastMath.sin(time * 45f);
    sceneNode.setLocalScale(1f, 1f + flicker * 0.002f, 1f);
}

private void updateStartCamera(float time) {
    float drift = FastMath.sin(time * 0.22f) * 0.45f;
    float push = FastMath.sin(time * 0.16f) * 1.2f;
    app.getCamera().setLocation(new Vector3f(drift, 2.05f + FastMath.sin(time * 0.31f) * 0.05f, 4f + push));
    app.getCamera().lookAt(new Vector3f(0f, 1.55f, -10f), Vector3f.UNIT_Y);
}

private void updateCamera(float time) {
    CameraPoint start = CAMERA_PATH[CAMERA_PATH.length - 1];
    CameraPoint end = start;
    float t = 0f;

    for (int i = 0; i < CAMERA_PATH.length - 1; i++) {
        CameraPoint a = CAMERA_PATH[i];
        CameraPoint b = CAMERA_PATH[i + 1];
        if (time >= a.time() && time <= b.time()) {
            start = a;
            end = b;
            t = (time - a.time()) / Math.max(0.001f, b.time() - a.time());
        }
    }
}
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        break;
    }
}

float eased = easeInOut(t);
Vector3f position = interpolate(start.position(), end.position(), eased);
Vector3f lookAt = interpolate(start.lookAt(), end.lookAt(), eased);
app.getCamera().setLocation(position);
app.getCamera().lookAt(lookAt, Vector3f.UNIT_Y);
}

private void updateGui(float time) {
    float sw = app.getCamera().getWidth();
    float sh = app.getCamera().getHeight();
    float progress = Math.min(time / DURATION, 1f);

    layoutStartScreen(sw, sh);

    backdrop.setLocalScale(sw, sh, 1f);
    backdrop.setLocalTranslation(0f, 0f, -8f);
    vignetteTop.setLocalScale(sw, sh * 0.16f, 1f);
    vignetteTop.setLocalTranslation(0f, sh * 0.84f, -7f);
    vignetteBottom.setLocalScale(sw, sh * 0.24f, 1f);
    vignetteBottom.setLocalTranslation(0f, 0f, -7f);
    progressBar.setLocalScale(sw * progress, 3f, 1f);
    progressBar.setLocalTranslation(0f, 0f, -5f);

    headerLeft.setLocalTranslation(34f, sh - 34f, 0f);
    headerRight.setLocalTranslation(sw - 240f, sh - 34f, 0f);

    Subtitle subtitle = currentSubtitle(time);
    subtitleText.setText(subtitle == null ? "" : "\"" + subtitle.text() + "\"");
    float subtitleWidth = subtitleText.getLineWidth();
    subtitleText.setLocalTranslation(sw / 2f - subtitleWidth / 2f, sh * 0.27f, 0f);
    decryptText.setLocalTranslation(sw / 2f - 150f, sh * 0.2f, 0f);
    decryptText.setCullHint(subtitle == null ? Spatial.CullHint.Always : Spatial.CullHint.Never);

    boolean showLogo = progress >= 0.9f;
    logoBackdrop.setCullHint(showLogo ? Spatial.CullHint.Never : Spatial.CullHint.Always);
    logoText.setCullHint(showLogo ? Spatial.CullHint.Never : Spatial.CullHint.Always);
    if (showLogo) {
        float alpha = Math.min((progress - 0.9f) / 0.1f, 1f);
        logoBackdrop.setLocalScale(sw, sh, 1f);
        logoBackdrop.setLocalTranslation(0f, 0f, -1f);
        logoText.setColor(new ColorRGBA(1f, 1f, 1f, alpha));
        logoText.setLocalTranslation(sw / 2f - logoText.getLineWidth() / 2f, sh / 2f + logoText.getHeight() / 2f, 1f);
    }
}

private void layoutStartScreen(float sw, float sh) {
    float frameW = Math.min(sw * 0.74f, 952f);
    float frameH = Math.max(145f, sh * 0.28f);
    float frameX = (sw - frameW) / 2f;
    float frameY = sh * 0.52f;
    float buttonW = Math.min(400f, sw * 0.42f);
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
float buttonH = 92f;
float buttonX = (sw - buttonW) / 2f;
float buttonY = sh * 0.17f;

startBackdrop.setLocalScale(sw, sh, 1f);
startBackdrop.setLocalTranslation(0f, 0f, -8f);
startFrame.setLocalScale(frameW, frameH, 1f);
startFrame.setLocalTranslation(frameX, frameY, -7f);
startDivider.setLocalScale(frameW * 0.5f, 1f, 1f);
startDivider.setLocalTranslation((sw - frameW * 0.5f) / 2f, frameY - 32f, -7f);
initializeButtonGlow.setLocalScale(buttonW + 34f, buttonH + 22f, 1f);
initializeButtonGlow.setLocalTranslation(buttonX - 17f, buttonY - 11f, -7f);
initializeButton.setLocalScale(buttonW, buttonH, 1f);
initializeButton.setLocalTranslation(buttonX, buttonY, -6f);

startEyebrow.setLocalTranslation(sw / 2f - startEyebrow.getLineWidth() / 2f, frameY + frameH + 18f, 0f);
float titleW = Math.min(sw * 0.78f, 1030f);
float titleH = titleW * 260f / 1600f;
startTitle.setLocalScale(titleW, titleH, 1f);
startTitle.setLocalTranslation(sw / 2f - titleW / 2f, frameY + frameH * 0.22f, 0f);
connectionText.setLocalTranslation(sw / 2f - connectionText.getLineWidth() / 2f, buttonY + buttonH + 50f, 0f);
playIcon.setLocalTranslation(buttonX + buttonW * 0.26f, buttonY + buttonH * 0.65f, 0f);
initializeLabel.setLocalTranslation(buttonX + buttonW * 0.34f, buttonY + buttonH * 0.65f, 0f);
}

private Subtitle currentSubtitle(float time) {
    for (Subtitle subtitle : SUBTITLES) {
        if (time >= subtitle.start() && time <= subtitle.end()) {
            return subtitle;
        }
    }
    return null;
}

private Vector3f interpolate(Vector3f start, Vector3f end, float t) {
    return new Vector3f(
        FastMath.interpolateLinear(t, start.x, end.x),
        FastMath.interpolateLinear(t, start.y, end.y),
        FastMath.interpolateLinear(t, start.z, end.z));
}

private float easeInOut(float t) {
    return t < 0.5f ? 2f * t * t : 1f - FastMath.pow(-2f * t + 2f, 2f) / 2f;
}

private void finish() {
    if (finished) {
        return;
    }
    finished = true;
    active = false;
    audioManager.stopIntroMusic();
    sceneNode.setCullHint(Spatial.CullHint.Always);
    guiNode.setCullHint(Spatial.CullHint.Always);
    onFinish.run();
}
```



}
}
14. MainMenu.java:

```
package subject47.ui;

import com.jme3.collison.CollisionResult;
import com.jme3.collison.CollisionResults;
import com.jme3.font.BitmapFont;
import com.jme3.font.BitmapText;
import com.jme3.input.KeyInput;
import com.jme3.input.MouseInput;
import com.jme3.input.controls.ActionListener;
import com.jme3.input.controls.KeyTrigger;
import com.jme3.input.controls.MouseButtonTrigger;
import com.jme3.material.Material;
import com.jme3.material.RenderState;
import com.jme3.math.ColorRGBA;
import com.jme3.math.Ray;
import com.jme3.math.Vector2f;
import com.jme3.math.Vector3f;
import com.jme3.scene.Geometry;
import com.jme3.scene.Node;
import com.jme3.scene.Spatial;
import com.jme3.scene.shape.Quad;
import subject47.Main;

import java.awt.AlphaComposite;
import java.awt.Font;
import java.awt.FontFormatException;
import java.awt.Graphics2D;
import java.awt.RenderingHints;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;

import com.jme3.texture.Texture2D;
import com.jme3.texture.plugins.AWTLoader;

public class MainMenu {

    private static final String ACTION_START = "start";
    private static final String ACTION_CONTROLS = "controls";
    private static final String ACTION_BACK = "back";
    private static final String ACTION_EXIT = "exit";

    // Sci-fi colour palette
    private static final ColorRGBA COL_BG = new ColorRGBA(0f, 0.01f, 0.04f, 0.93f);
    private static final ColorRGBA COL_PANEL = new ColorRGBA(0.02f, 0.06f, 0.12f, 0.88f);
    private static final ColorRGBA COL_ACCENT = new ColorRGBA(0.0f, 0.7f, 1.0f, 0.18f);
    private static final ColorRGBA COL_BORDER = new ColorRGBA(0.0f, 0.6f, 0.9f, 0.35f);
    private static final ColorRGBA COL_BTN_MAIN = new ColorRGBA(0.0f, 0.55f, 0.9f, 0.85f);
    private static final ColorRGBA COL_BTN_SEC = new ColorRGBA(0.04f, 0.18f, 0.32f, 0.88f);
    private static final ColorRGBA COL_BTN_QUIT = new ColorRGBA(0.28f, 0.05f, 0.05f, 0.88f);
    private static final ColorRGBA COL_SCANLINE = new ColorRGBA(0.0f, 0.8f, 1.0f, 0.06f);
    private static final ColorRGBA COL_RED_ALERT = new ColorRGBA(0.9f, 0.12f, 0.12f, 0.85f);
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
private final Main app;
private final ActionListener menuListener;
private final Node menuNode = new Node("MainMenu");
private final Node buttonNode = new Node("MenuButtons");

private BitmapFont font;

// Background layers
private Geometry bgFull;
private Geometry bgPanel;
private Geometry bgPanelBorder;
private Geometry topBar;
private Geometry bottomBar;
private Geometry scanline;

// Corner bracket lines (4 corners × 2 lines each)
private Geometry[] cornerH = new Geometry[4];
private Geometry[] cornerV = new Geometry[4];

// Title graphic
private Geometry titleGraphic;
private BitmapText eyebrow;
private BitmapText classifiedLabel;
private Geometry classifiedLineL;
private Geometry classifiedLineR;

// Status strip
private BitmapText statusLeft;
private BitmapText statusRight;

// Buttons
private Geometry startBtn;
private Geometry controlsBtn;
private Geometry backBtn;
private Geometry exitBtn;
private BitmapText startLabel;
private BitmapText startSub;
private BitmapText controlsLabel;
private BitmapText controlsSub;
private BitmapText backLabel;
private BitmapText backSub;
private BitmapText exitLabel;
private BitmapText exitSub;
private Geometry startGlow;

// Controls / body text
private BitmapText body;

private boolean visible;
private boolean controlsVisible;

// Input grace period – blocks menu input for a short time after the menu
// becomes visible, preventing a simultaneous key press (e.g. ENTER skipping
// the intro) from also immediately triggering a menu action.
```



```
private float inputGrace = 0f;

// Scanline animation
private float scanY = 0f;
private float scanTimer = 0f;

public MainMenu(Main app) {
    this.app = app;
    this.menuListener = this::handleMenuInput;
}

// — Initialise

public void initialize() {
    font = app.getAssetManager().loadFont("Interface/Fonts/Default.fnt");

    buildBackground();
    buildCornerBrackets();
    buildTitle();
    buildStatusStrip();
    buildButtons();
    buildBodyText();

    menuNode.attachChild(buttonNode);
    app.getGuiNode().attachChild(menuNode);
    registerInput();
    showMainScreen();
}

// — Background layers

private void buildBackground() {
    bgFull    = quad("MenuBg",    COL_BG,    -10f);
    bgPanel   = quad("MenuPanel",  COL_PANEL, -9f);
    bgPanelBorder = quad("MenuPanelBorder", COL_BORDER, -9.5f);
    topBar    = quad("MenuTopBar",  COL_ACCENT, -8f);
    bottomBar = quad("MenuBottomBar", COL_ACCENT, -8f);
    scanline  = quad("MenuScanline", COL_SCANLINE, -7f);

    menuNode.attachChild(bgFull);
    menuNode.attachChild(bgPanel);
    menuNode.attachChild(bgPanelBorder);
    menuNode.attachChild(topBar);
    menuNode.attachChild(bottomBar);
    menuNode.attachChild(scanline);
}

// — Corner brackets (HUD-style)

private void buildCornerBrackets() {
    ColorRGBA bc = new ColorRGBA(0f, 0.75f, 1f, 0.55f);
    for (int i = 0; i < 4; i++) {
        cornerH[i] = quad("CornerH" + i, bc, -6f);
        cornerV[i] = quad("CornerV" + i, bc, -6f);
        menuNode.attachChild(cornerH[i]);
    }
}
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        menuNode.attachChild(cornerV[i]);
    }
}

// — Title

private void buildTitle() {
    titleGraphic = createTitleGraphic();
    menuNode.attachChild(titleGraphic);

    eyebrow = bitmapText("A P S Y C H O L O G I C A L H O R R O R E X P E R I E N C E",
        0.82f, new ColorRGBA(1f, 1f, 1f, 0.22f));
    menuNode.attachChild(eyebrow);

    classifiedLabel = bitmapText("C L A S S I F I E D — E Y E S O N L Y",
        0.78f, new ColorRGBA(0.9f, 0.15f, 0.15f, 0.75f));
    menuNode.attachChild(classifiedLabel);

    classifiedLineL = quad("ClassLineL", new ColorRGBA(0.8f, 0.1f, 0.1f, 0.45f), -6f);
    classifiedLineR = quad("ClassLineR", new ColorRGBA(0.8f, 0.1f, 0.1f, 0.45f), -6f);
    menuNode.attachChild(classifiedLineL);
    menuNode.attachChild(classifiedLineR);
}

// — Status strip

private void buildStatusStrip() {
    statusLeft = bitmapText(
        "SECTOR_7 SUBLEVEL_04\nFACILITY_STATUS: CRITICAL_FAILURE\n[REC] SITE_47_LOG",
        0.75f, new ColorRGBA(0.1f, 0.65f, 0.95f, 0.65f));
    menuNode.attachChild(statusLeft);

    statusRight = bitmapText(
        "AUTH: TECH_UNIT_884\nSIGNAL ||| .",
        0.75f, new ColorRGBA(0.75f, 0.8f, 0.88f, 0.45f));
    menuNode.attachChild(statusRight);
}

// — Menu Buttons

private void buildButtons() {
    startGlow = quad("StartGlow", new ColorRGBA(0f, 0.55f, 0.9f, 0.22f), -7.5f);
    buttonNode.attachChild(startGlow);

    startBtn = actionQuad("StartButton", COL_BTN_MAIN, ACTION_START);
    controlsBtn = actionQuad("ControlsButton", COL_BTN_SEC, ACTION_CONTROLS);
    backBtn = actionQuad("BackButton", COL_BTN_QUIT, ACTION_BACK);
    exitBtn = actionQuad("ExitButton",
        new ColorRGBA(0.22f, 0.04f, 0.04f, 0.92f), ACTION_EXIT);

    startLabel = bitmapText("INITIALIZE PROTOCOL", 1.35f, ColorRGBA.White);
    startSub = bitmapText("Begin classified sequence", 0.78f,
        new ColorRGBA(0.7f, 0.9f, 1f, 0.55f));
    controlsLabel = bitmapText("CONTROLS", 1.2f, new ColorRGBA(0.6f, 0.85f, 1f, 0.92f));
    controlsSub = bitmapText("Key bindings & reference", 0.78f,
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
new ColorRGBA(0.5f, 0.75f, 0.95f, 0.5f));
backLabel = bitmapText("QUIT", 1.2f, new ColorRGBA(1f, 0.35f, 0.35f, 0.92f));
backSub = bitmapText("Exit to desktop", 0.78f,
new ColorRGBA(0.9f, 0.4f, 0.4f, 0.45f));
exitLabel = bitmapText("EXIT GAME", 1.2f, new ColorRGBA(1f, 0.25f, 0.25f, 0.92f));
exitSub = bitmapText("Quit to desktop", 0.78f,
new ColorRGBA(0.9f, 0.3f, 0.3f, 0.45f));

buttonNode.attachChild(startLabel);
buttonNode.attachChild(startSub);
buttonNode.attachChild(controlsLabel);
buttonNode.attachChild(controlsSub);
buttonNode.attachChild(backLabel);
buttonNode.attachChild(backSub);
buttonNode.attachChild(exitLabel);
buttonNode.attachChild(exitSub);
}

// — Controls / body text

private void buildBodyText() {
body = bitmapText("", 1.0f, new ColorRGBA(0.78f, 0.88f, 1f, 0.82f));
menuNode.attachChild(body);
}

// — Layout (called every frame while visible)

private void layout() {
float sw = app.getCamera().getWidth();
float sh = app.getCamera().getHeight();

// Full-screen bg
place(bgFull, 0, 0, sw, sh);

// Central panel (narrow column)
float panelW = Math.min(sw * 0.62f, 820f);
float panelX = (sw - panelW) / 2f;
float panelY = sh * 0.07f;
float panelH = sh * 0.86f;
// 1-px border trick: slightly larger quad behind
place(bgPanelBorder, panelX - 1, panelY - 1, panelW + 2, panelH + 2);
place(bgPanel, panelX, panelY, panelW, panelH);

// Top / bottom accent bars
place(topBar, panelX, panelY + panelH - 3f, panelW, 3f);
place(bottomBar, panelX, panelY, panelW, 3f);

// Scanline sweep
scanY = (sh + 6f) - (scanTimer % 3f) / 3f * (sh + 6f);
place(scanline, 0, scanY, sw, 6f);

// — Corner brackets – pass exact corner point + orientation flags —
float bm = 24f; // inset from panel edge
float bl = 28f; // arm length
float bt = 2f; // arm thickness
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
float cxL = panelX + bm;           // left corner x
float cxR = panelX + panelW - bm; // right corner x
float cyT = panelY + panelH - bm; // top corner y
float cyB = panelY + bm;          // bottom corner y
placeCorner(0, cxL, cyT, bl, bt, true, true); // top-left
placeCorner(1, cxR, cyT, bl, bt, true, false); // top-right
placeCorner(2, cxL, cyB, bl, bt, false, true); // bottom-left
placeCorner(3, cxR, cyB, bl, bt, false, false); // bottom-right

// — Title (upper-centre of panel)
float titleW = Math.min(panelW * 0.88f, 710f);
float titleH = titleW * 260f / 1600f;
// Bottom of title sits at 58 % up the panel height
float titleY = panelY + panelH * 0.58f;
// Perfectly centred on screen
float titleX = sw / 2f - titleW / 2f;
titleGraphic.setLocalScale(titleW, titleH, 1f);
titleGraphic.setLocalTranslation(titleX, titleY, 0f);

// Eyebrow -- centred above the title
eyebrow.setLocalTranslation(
    sw / 2f - eyebrow.getLineWidth() / 2f,
    titleY + titleH + 12f, 0f);

// "Classified" label -- centred below the title
float cLY = titleY - 28f;
classifiedLabel.setLocalTranslation(
    sw / 2f - classifiedLabel.getLineWidth() / 2f,
    cLY, 0f);
float lineW = 72f;
place(classifiedLineL,
    sw / 2f - classifiedLabel.getLineWidth() / 2f - lineW - 10f,
    cLY - 7f, lineW, 1.5f);
place(classifiedLineR,
    sw / 2f + classifiedLabel.getLineWidth() / 2f + 10f,
    cLY - 7f, lineW, 1.5f);

// Status strip – statusLeft shifted right of the corner bracket (bracket ends at panelX+52)
statusLeft.setLocalTranslation(panelX + 60f, panelY + panelH - 16f, 0f);
statusRight.setLocalTranslation(
    panelX + panelW - statusRight.getLineWidth() - 14f,
    panelY + panelH - 16f, 0f);

if (!controlsVisible) {
    layoutMainButtons(sw, sh, panelX, panelY, panelW, panelH);
    body.setCullHint(Spatial.CullHint.Always);
} else {
    layoutControlsBody(sw, sh, panelX, panelY, panelW, panelH);
}
}

private void layoutMainButtons(float sw, float sh,
    float panelX, float panelY, float panelW, float panelH) {
    float btnW = Math.min(panelW * 0.72f, 520f);
    float btnX = (sw - btnW) / 2f;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
boolean paused = app.hasStartedGame();

// Use slightly smaller buttons when 4 are showing (pause state)
float btnH = paused ? 60f : 66f;
float gap = paused ? 10f : 12f;

float y3 = panelY + panelH * 0.10f; // lowest (exit / quit)
float y2 = y3 + btnH + gap;         // back / return to lab
float y1 = y2 + btnH + gap;         // controls
float y0 = y1 + btnH + gap;         // top: initialize

// If not paused: collapse y3 into y2 (only 3 buttons)
if (!paused) {
    y2 = panelY + panelH * 0.12f;
    y1 = y2 + btnH + gap;
    y0 = y1 + btnH + gap;
}

place(startGlow, btnX - 10f, y0 - 8f, btnW + 20f, btnH + 16f);
startGlow.setCullHint(Spatial.CullHint.Never);

placeBtn(startBtn, btnX, y0, btnW, btnH);
placeBtn(controlsBtn, btnX, y1, btnW, btnH);
placeBtn(backBtn, btnX, y2, btnW, btnH);

float lp = btnX + 24f;
startLabel.setLocalTranslation(lp, y0 + btnH * 0.68f, 1f);
startSub.setLocalTranslation( lp, y0 + btnH * 0.32f, 1f);
controlsLabel.setLocalTranslation(lp, y1 + btnH * 0.68f, 1f);
controlsSub.setLocalTranslation( lp, y1 + btnH * 0.32f, 1f);
backLabel.setLocalTranslation( lp, y2 + btnH * 0.68f, 1f);
backSub.setLocalTranslation( lp, y2 + btnH * 0.32f, 1f);

if (paused) {
    placeBtn(exitBtn, btnX, y3, btnW, btnH);
    exitLabel.setLocalTranslation(lp, y3 + btnH * 0.68f, 1f);
    exitSub.setLocalTranslation( lp, y3 + btnH * 0.32f, 1f);
    exitBtn.setCullHint(Spatial.CullHint.Never);
    exitLabel.setCullHint(Spatial.CullHint.Never);
    exitSub.setCullHint(Spatial.CullHint.Never);
} else {
    exitBtn.setCullHint(Spatial.CullHint.Always);
    exitLabel.setCullHint(Spatial.CullHint.Always);
    exitSub.setCullHint(Spatial.CullHint.Always);
}

for (Spatial s : new Spatial[] { startBtn, controlsBtn, backBtn,
    startLabel, startSub, controlsLabel, controlsSub, backLabel, backSub }) {
    s.setCullHint(Spatial.CullHint.Never);
}

private void layoutControlsBody(float sw, float sh,
    float panelX, float panelY,
    float panelW, float panelH) {
```



```
for (Spatial s : new Spatial[] { startGlow, startBtn, controlsBtn, backBtn,
    startLabel, startSub, controlsLabel, controlsSub, backLabel, backSub,
    exitBtn, exitLabel, exitSub }) {
    s.setCullHint(Spatial.CullHint.Always);
}
body.setCullHint(Spatial.CullHint.Never);
body.setLocalTranslation(panelX + 48f, panelY + panelH * 0.64f, 0f);
}

// — Builder helpers

private Geometry quad(String name, ColorRGBA color, float z) {
    Geometry g = new Geometry(name, new Quad(1f, 1f));
    g.setMaterial(mat(color));
    g.setLocalTranslation(0f, 0f, z);
    menuNode.attachChild(g);
    return g;
}

private Geometry actionQuad(String name, ColorRGBA color, String action) {
    Geometry g = new Geometry(name, new Quad(1f, 1f));
    g.setMaterial(mat(color));
    g.setUserData("menuAction", action);
    buttonNode.attachChild(g);
    return g;
}

private BitmapText bitmapText(String text, float sizeMult, ColorRGBA color) {
    BitmapText t = new BitmapText(font);
    t.setSize(font.getCharSet().getRenderedSize() * sizeMult);
    t.setColor(color);
    t.setText(text);
    return t;
}

private Material mat(ColorRGBA color) {
    Material m = new Material(app.getAssetManager(), "Common/MatDefs/Misc/Unshaded.j3md");
    m.setColor("Color", color);
    if (color.a < 1f) {
        m.getAdditionalRenderState().setBlendMode(RenderState.BlendMode.Alpha);
    }
    return m;
}

private void place(Geometry g, float x, float y, float w, float h) {
    g.setLocalScale(w, h, 1f);
    float z = g.getLocalTranslation().z;
    g.setLocalTranslation(x, y, z);
}

private void placeBtn(Geometry g, float x, float y, float w, float h) {
    g.setLocalScale(w, h, 1f);
    g.setLocalTranslation(x, y, -6f);
}
```



```
/**
 * Draw one L-shaped corner bracket.
 * cx,cy = exact corner point (where the two arms meet).
 * top = true → arms point downward (top corners)
 * top = false → arms point upward (bottom corners)
 * left = true → arms point rightward (left corners)
 * left = false → arms point leftward (right corners)
 */
private void placeCorner(int idx, float cx, float cy,
                        float len, float thick,
                        boolean top, boolean left) {
    // Horizontal arm
    float hx = left ? cx : cx - len;    // left corners start at cx; right corners end at cx
    float hy = top ? cy - thick : cy;    // top corners hang below cy; bottom corners sit above cy
    cornerH[idx].setLocalScale(len, thick, 1f);
    cornerH[idx].setLocalTranslation(hx, hy, -5f);

    // Vertical arm
    float vx = left ? cx : cx - thick;    // align with horizontal arm's left or right edge
    float vy = top ? cy - len : cy;    // top corners hang down from cy; bottom corners rise from cy
    cornerV[idx].setLocalScale(thick, len, 1f);
    cornerV[idx].setLocalTranslation(vx, vy, -5f);
}

// — Title graphic (rendered via Java2D → Texture2D)

private Geometry createTitleGraphic() {
    int w = 1600, h = 260;
    BufferedImage img = new BufferedImage(w, h, BufferedImage.TYPE_INT_ARGB);
    Graphics2D g = img.createGraphics();
    g.setComposite(AlphaComposite.Clear);
    g.fillRect(0, 0, w, h);
    g.setComposite(AlphaComposite.SrcOver);
    g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
    g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
    g.setFont(loadTitleFont(172f));
    java.awt.FontMetrics fm = g.getFontMetrics();
    int textW = fm.stringWidth("SUBJECT 47");
    int textX = (w - textW) / 2; // horizontally centre in the 1600px canvas
    // Cyan glow layer (two offset passes)
    g.setColor(new java.awt.Color(0, 180, 255, 55));
    g.drawString("SUBJECT 47", textX - 6, 198);
    g.drawString("SUBJECT 47", textX + 6, 198);
    // White main text
    g.setColor(java.awt.Color.WHITE);
    g.drawString("SUBJECT 47", textX, 196);
    g.dispose();

    AWTLoader loader = new AWTLoader();
    Texture2D tex = new Texture2D(loader.load(img, true));
    Material mat = new Material(app.getAssetManager(), "Common/MatDefs/Misc/Unshaded.j3md");
    mat.setTexture("ColorMap", tex);
    mat.getAdditionalRenderState().setBlendMode(RenderState.BlendMode.Alpha);
}
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
Geometry geo = new Geometry("MenuTitleGraphic", new Quad(1f, 1f));
geo.setMaterial(mat);
return geo;
}

private Font loadTitleFont(float size) {
    File f = new File("C:\\Windows\\Fonts\\ariblk.ttf");
    if (f.isFile()) {
        try { return Font.createFont(Font.TRUETYPE_FONT, f).deriveFont(size); }
        catch (FontFormatException | IOException ignored) {}
    }
    return new Font("Arial Black", Font.BOLD, Math.round(size));
}

// — Input

private void registerInput() {
    app.getInputManager().addMapping("MenuAccept", new KeyTrigger(KeyInput.KEY_RETURN));
    app.getInputManager().addMapping("MenuControls", new KeyTrigger(KeyInput.KEY_C));
    app.getInputManager().addMapping("MenuBack", new KeyTrigger(KeyInput.KEY_ESCAPE));
    app.getInputManager().addMapping("MenuClick", new MouseButtonTrigger(MouseInput.BUTTON_LEFT));
    app.getInputManager().addListener(menuListener, "MenuAccept", "MenuControls", "MenuBack", "MenuClick");
}

private void handleMenuInput(String name, boolean pressed, float tpf) {
    if (!pressed) return;
    if (inputGrace > 0f) return;
    if (!visible && app.getUi().isKeypadOpen()) return;
    switch (name) {
        case "MenuClick" -> { if (visible) activateButtonAtCursor(); }
        case "MenuBack" -> performAction(ACTION_BACK);
        case "MenuControls" -> performAction(ACTION_CONTROLS);
        case "MenuAccept" -> { if (visible && !controlsVisible) performAction(ACTION_START); }
    }
}

private void activateButtonAtCursor() {
    CollisionResults results = new CollisionResults();
    Vector2f cursor = app.getInputManager().getCursorPosition();
    Ray ray = new Ray(new Vector3f(cursor.x, cursor.y, 10f), new Vector3f(0f, 0f, -1f));
    buttonNode.collideWith(ray, results);
    for (CollisionResult r : results) {
        String action = r.getGeometry().getUserData("menuAction");
        if (action != null) { performAction(action); return; }
    }
}

private void performAction(String action) {
    if (ACTION_CONTROLS.equals(action)) {
        if (!visible) return;
        if (controlsVisible) showMainScreen(); else showControlsScreen();
        return;
    }
    if (ACTION_EXIT.equals(action)) {
        app.stop();
    }
}
```




```

return;
}
if (ACTION_BACK.equals(action)) {
    if (visible && controlsVisible) { showMainScreen(); return; }
    if (visible && !app.hasStartedGame()) { app.stop(); return; }
    if (visible) { hide(); app.resumeGame(); return; }
    app.pauseToMenu();
    return;
}
if (!visible || controlsVisible) return;
if (ACTION_START.equals(action)) app.startGame();
}

// — Public screen states

public void showMainScreen() {
    controlsVisible = false;
    visible = true;
    inputGrace = 0.2f; // block input for 200 ms so the key that triggered this doesn't also activate a button
    startLabel.setText("INITIALIZE PROTOCOL");
    startSub.setText(app.hasStartedGame() ? "Restart the experiment" : "Begin classified sequence");
    backLabel.setText(app.hasStartedGame() ? "RETURN TO LAB" : "QUIT");
    backSub.setText(app.hasStartedGame() ? "Resume mission" : "Exit to desktop");
    exitLabel.setText("EXIT GAME");
    exitSub.setText("Quit to desktop");
    body.setText("");
    menuNode.setCullHint(Spatial.CullHint.Never);
    app.getInputManager().setCursorVisible(true);
    app.getAudio().playMenuMusic();
    layout();
}

public void showControlsScreen() {
    controlsVisible = true;
    visible = true;
    body.setText(
        "\n\n\n\n\n\n\n\n\n\n" +
        "W A S D      Move\n" +
        "Hold LMB     Look around\n" +
        "E            Interact\n" +
        "0 – 9        Enter keypad digits\n" +
        "ENTER        Use default keypad button\n" +
        "BACKSPACE    Delete keypad digit\n" +
        "ESC          Back / pause menu\n" +
        "[ C ]        Toggle controls screen"
    );
    menuNode.setCullHint(Spatial.CullHint.Never);
    app.getInputManager().setCursorVisible(true);
    layout();
}

public void hide() {
    visible = false;
    controlsVisible = false;
    app.getAudio().stopMenuMusic(); // ensure music stops when leaving the menu
}

```



```
    menuNode.setCullHint(Spatial.CullHint.Always);  
}  
  
public boolean isVisible() { return visible; }  
  
// — Per-frame update (scanline animation)  
  
public void update(float tpf) {  
    if (!visible) return;  
    if (inputGrace > 0f) inputGrace -= tpf;  
    scanTimer += tpf;  
    layout();  
}  
}
```

15. UIManager.java:

```
package subject47.ui;  
  
import java.util.function.Consumer;  
import com.jme3.app.SimpleApplication;  
import com.jme3.font.BitmapFont;  
import com.jme3.font.BitmapText;  
import com.jme3.input.KeyInput;  
import com.jme3.input.controls.ActionListener;  
import com.jme3.input.controls.KeyTrigger;  
import com.jme3.material.Material;  
import com.jme3.math.ColorRGBA;  
import com.jme3.scene.Geometry;  
import com.jme3.scene.Node;  
import com.jme3.scene.Spatial;  
import com.jme3.scene.shape.Quad;  
  
public class UIManager {  
  
    private final SimpleApplication app;  
    private final ActionListener keypadListener;  
    private final ActionListener modalListener;  
    private final Node hudNode = new Node("Hud");  
    private final Node modalNode = new Node("Modal");  
    private final Node modalChromeNode = new Node("ModalChrome");  
  
    private BitmapText crosshair;  
    private BitmapText prompt;  
    private BitmapText objective;  
    private BitmapText status;  
    private BitmapText message;  
    private BitmapText editorOverlay;  
    private BitmapText keypadTitle;  
    private BitmapText keypadInput;  
    private BitmapText keypadHint;  
    private BitmapText modalEyebrow;  
    private BitmapText modalTitle;  
    private BitmapText modalBody;  
    private BitmapText modalFooter;  
    private BitmapText modalSidebar;  
    private Geometry modalBackdrop;
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
private Geometry modalPanel;
private Geometry modalSidebarPanel;
private Geometry modalAccentTop;
private Geometry modalAccentBottom;
private Geometry modalFooterPanel;

private float messageTimer;
private float modalInputDelay;
private boolean keypadOpen;
private boolean infoModalOpen;
private Consumer<String> keypadSubmit;
private Runnable infoModalContinue;
private final StringBuilder keypadBuffer = new StringBuilder();

public UIManager(SimpleApplication app) {
    this.app = app;
    this.keypadListener = this::handleKeypadInput;
    this.modalListener = this::handleModalInput;
}

public void initialize() {
    BitmapFont font = app.getAssetManager().loadFont("Interface/Fonts/Default.fnt");

    modalBackdrop = createPanel("ModalBackdrop", new ColorRGBA(0.02f, 0.03f, 0.05f, 0.92f));
    modalPanel = createPanel("ModalPanel", new ColorRGBA(0.08f, 0.1f, 0.13f, 0.97f));
    modalSidebarPanel = createPanel("ModalSidebarPanel", new ColorRGBA(0.12f, 0.16f, 0.2f, 0.95f));
    modalAccentTop = createPanel("ModalAccentTop", new ColorRGBA(0.15f, 0.78f, 0.98f, 0.95f));
    modalAccentBottom = createPanel("ModalAccentBottom", new ColorRGBA(0.72f, 0.12f, 0.18f, 0.95f));
    modalFooterPanel = createPanel("ModalFooterPanel", new ColorRGBA(0.12f, 0.14f, 0.18f, 0.94f));
    modalChromeNode.attachChild(modalBackdrop);
    modalChromeNode.attachChild(modalPanel);
    modalChromeNode.attachChild(modalSidebarPanel);
    modalChromeNode.attachChild(modalAccentTop);
    modalChromeNode.attachChild(modalAccentBottom);
    modalChromeNode.attachChild(modalFooterPanel);
    modalNode.attachChild(modalChromeNode);

    crosshair = new BitmapText(font);
    crosshair.setSize(font.getCharSet().getRenderedSize() * 2.5f);
    crosshair.setText("+");
    centerCrosshair();
    hudNode.attachChild(crosshair);

    prompt = new BitmapText(font);
    prompt.setLocalTranslation(520, 320, 0);
    hudNode.attachChild(prompt);

    objective = new BitmapText(font);
    objective.setLocalTranslation(30, 690, 0);
    hudNode.attachChild(objective);

    status = new BitmapText(font);
    status.setLocalTranslation(30, 650, 0);
    hudNode.attachChild(status);
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
message = new BitmapText(font);
message.setLocalTranslation(30, 120, 0);
hudNode.attachChild(message);

editorOverlay = new BitmapText(font);
editorOverlay.setLocalTranslation(30, 220, 0);
hudNode.attachChild(editorOverlay);

keypadTitle = new BitmapText(font);
keypadTitle.setLocalTranslation(440, 420, 0);
modalNode.attachChild(keypadTitle);

keypadInput = new BitmapText(font);
keypadInput.setLocalTranslation(520, 380, 0);
modalNode.attachChild(keypadInput);

keypadHint = new BitmapText(font);
keypadHint.setSize(font.getCharSet().getRenderedSize() * 1.05f);
keypadHint.setText("0-9 enter code Backspace delete Enter confirm Esc cancel");
modalNode.attachChild(keypadHint);

modalEyebrow = new BitmapText(font);
modalEyebrow.setSize(font.getCharSet().getRenderedSize() * 1.0f);
modalNode.attachChild(modalEyebrow);

modalTitle = new BitmapText(font);
modalTitle.setSize(font.getCharSet().getRenderedSize() * 2.3f);
modalNode.attachChild(modalTitle);

modalBody = new BitmapText(font);
modalBody.setSize(font.getCharSet().getRenderedSize() * 1.15f);
modalNode.attachChild(modalBody);

modalFooter = new BitmapText(font);
modalFooter.setSize(font.getCharSet().getRenderedSize() * 1.0f);
modalNode.attachChild(modalFooter);

modalSidebar = new BitmapText(font);
modalSidebar.setSize(font.getCharSet().getRenderedSize() * 1.0f);
modalNode.attachChild(modalSidebar);

app.getGuiNode().attachChild(hudNode);
app.getGuiNode().attachChild(modalNode);

setPrompt(null);
setHudVisible(false);
closeKeypad();
registerKeypadInput();
registerModalInput();
layoutModal();
}

private void registerKeypadInput() {
    // KEY_0..KEY_9 are NOT sequential in JME (KEY_0=11, KEY_1=2..KEY_9=10).
    // Must map each digit explicitly.
```



```
int[] digitKeys = {
    KeyInput.KEY_0, KeyInput.KEY_1, KeyInput.KEY_2, KeyInput.KEY_3, KeyInput.KEY_4,
    KeyInput.KEY_5, KeyInput.KEY_6, KeyInput.KEY_7, KeyInput.KEY_8, KeyInput.KEY_9
};
int[] numpadKeys = {
    KeyInput.KEY_NUMPAD0, KeyInput.KEY_NUMPAD1, KeyInput.KEY_NUMPAD2,
    KeyInput.KEY_NUMPAD3, KeyInput.KEY_NUMPAD4, KeyInput.KEY_NUMPAD5,
    KeyInput.KEY_NUMPAD6, KeyInput.KEY_NUMPAD7, KeyInput.KEY_NUMPAD8,
    KeyInput.KEY_NUMPAD9
};
for (int d = 0; d <= 9; d++) {
    app.getInputManager().addMapping("KPDigit" + d,
        new KeyTrigger(digitKeys[d]),
        new KeyTrigger(numpadKeys[d]));
}
app.getInputManager().addMapping("KeypadSubmit", new KeyTrigger(KeyInput.KEY_RETURN),
    new KeyTrigger(KeyInput.KEY_NUMPADENTER));
app.getInputManager().addMapping("KeypadBack", new KeyTrigger(KeyInput.KEY_BACK),
    new KeyTrigger(KeyInput.KEY_DELETE));
// ESC closes the keypad (safe — JME's default ESC-quit is removed in Main.simpleInitApp).
app.getInputManager().addMapping("KeypadCancel", new KeyTrigger(KeyInput.KEY_ESCAPE));

String[] digitNames = new String[10];
for (int d = 0; d <= 9; d++) digitNames[d] = "KPDigit" + d;
app.getInputManager().addListener(keypadListener,
    "KPDigit0", "KPDigit1", "KPDigit2", "KPDigit3", "KPDigit4",
    "KPDigit5", "KPDigit6", "KPDigit7", "KPDigit8", "KPDigit9",
    "KeypadSubmit", "KeypadBack", "KeypadCancel");
}

private void handleKeypadInput(String name, boolean pressed, float tpf) {
    if (!pressed || !keypadOpen) return;

    if (name.startsWith("KPDigit") && keypadBuffer.length() < 4) {
        keypadBuffer.append(name.charAt(name.length() - 1));
        refreshKeypad();
        return;
    }

    switch (name) {
        case "KeypadBack" -> {
            if (keypadBuffer.length() > 0) {
                keypadBuffer.deleteCharAt(keypadBuffer.length() - 1);
                refreshKeypad();
            }
        }
        case "KeypadSubmit" -> {
            if (keypadBuffer.length() == 4 && keypadSubmit != null) {
                String code = keypadBuffer.toString();
                Consumer<String> submit = keypadSubmit;
                closeKeypad();
                submit.accept(code);
            }
        }
        case "KeypadCancel" -> {
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        closeKeypad();
        showMessage("Keypad cancelled.", 1.5f);
    }
}

private void registerModalInput() {
    app.getInputManager().addMapping("ModalContinueEnter", new KeyTrigger(KeyInput.KEY_RETURN));
    app.getInputManager().addMapping("ModalContinueUse", new KeyTrigger(KeyInput.KEY_E));
    app.getInputManager().addMapping("ModalContinueSpace", new KeyTrigger(KeyInput.KEY_SPACE));
    app.getInputManager().addListener(modalListener,
        "ModalContinueEnter", "ModalContinueUse", "ModalContinueSpace");
}

private void handleModalInput(String name, boolean pressed, float tpf) {
    if (!pressed || !infoModalOpen || keypadOpen || modalInputDelay > 0f) {
        return;
    }
    Runnable continueAction = infoModalContinue;
    closeInfoModal();
    if (continueAction != null) {
        continueAction.run();
    }
}

public void update(float tpf) {
    if (messageTimer > 0f) {
        messageTimer -= tpf;
        if (messageTimer <= 0f) {
            message.setText("");
        }
    }
    if (modalInputDelay > 0f) {
        modalInputDelay -= tpf;
    }
    centerCrosshair();
    // Hide the crosshair whenever a modal (objectives, keypad, narrative) is open.
    crosshair.setCullHint(isOverlayBlockingInput()
        ? Spatial.CullHint.Always
        : Spatial.CullHint.Inherit);
    layoutModal();
}

/**
 * Keeps the crosshair pixel-perfect at the screen centre.
 * BitmapText y = top-left corner in GuiNode (Y up from bottom).
 * Text renders downward, so: centreY = y - height/2 → y = centreY + height/2.
 */
private void centerCrosshair() {
    float sw = app.getCamera().getWidth();
    float sh = app.getCamera().getHeight();
    float cw = crosshair.getLineWidth();
    float ch = crosshair.getHeight();
    crosshair.setLocalTranslation(sw / 2f - cw / 2f, sh / 2f + ch / 2f, 0f);
}
```




```
public void setHudVisible(boolean visible) {  
    hudNode.setCullHint(visible ? Spatial.CullHint.Never : Spatial.CullHint.Always);  
    if (!visible) {  
        setPrompt(null);  
    }  
}  
  
public void setPrompt(String text) {  
    prompt.setText(text == null || keypadOpen ? "" : text);  
}  
  
public void setObjective(String text) {  
    objective.setText(text == null ? "" : "Objective: " + text);  
}  
  
public void setStatus(String text) {  
    status.setText(text == null ? "" : text);  
}  
  
public void showMessage(String text, float seconds) {  
    message.setText(text);  
    messageTimer = seconds;  
}  
  
public void showNarration(String text, float seconds) {  
    message.setText(text);  
    messageTimer = seconds;  
}  
  
public void setEditorOverlay(String text) {  
    editorOverlay.setText(text == null ? "" : text);  
}  
  
public void openKeypad(String title, Consumer<String> submit) {  
    keypadOpen = true;  
    keypadSubmit = submit;  
    keypadBuffer.setLength(0);  
    keypadTitle.setText(title);  
    refreshKeypad();  
    keypadHint.setText("0-9 enter code Backspace delete Enter confirm Esc cancel");  
    modalEyebrow.setText("SECURE ACCESS");  
    modalTitle.setText("");  
    modalBody.setText("");  
    modalFooter.setText("");  
    modalSidebar.setText("AUTH\n\nAwaiting valid\n4-digit code");  
    modalNode.setCullHint(Spatial.CullHint.Never);  
    setPrompt(null);  
    hideInfoModalText();  
}  
  
public void closeKeypad() {  
    keypadOpen = false;  
    keypadSubmit = null;  
    keypadBuffer.setLength(0);
```



```
keypadTitle.setText("");
keypadInput.setText("");
if (!infoModalOpen) {
    modalNode.setCullHint(Spatial.CullHint.Always);
}
}

public boolean isKeypadOpen() {
    return keypadOpen;
}

public void openInfoModal(String title, String body, String footer, Runnable onContinue) {
    infoModalOpen = true;
    modalInputDelay = 0.2f;
    infoModalContinue = onContinue;
    modalEyebrow.setText(resolveEyebrow(title));
    modalTitle.setText(title == null ? "" : title);
    modalBody.setText(body == null ? "" : body);
    modalFooter.setText(footer == null ? "Press Enter to continue" : footer);
    modalSidebar.setText(resolveSidebar(title, body));
    hideKeypadText();
    modalNode.setCullHint(Spatial.CullHint.Never);
    setPrompt(null);
}

/**
 * Opens a multi-page narrative sequence. Each page advances with Enter/E/Space.
 * Sidebar, eyebrow, title, and body can all vary per page.
 * After the last page, {@code onComplete} is invoked.
 */
public void openNarrativeSequence(
    String[] titles,
    String[] bodies,
    String[] eyebrows,
    String[] sidebars,
    Runnable onComplete) {
    openNarrativePage(titles, bodies, eyebrows, sidebars, onComplete, 0);
}

private void openNarrativePage(
    String[] titles, String[] bodies,
    String[] eyebrows, String[] sidebars,
    Runnable onComplete, int index) {
    boolean isLast = index >= titles.length - 1;
    int total = titles.length;
    String pageFooter = isLast
        ? "Press Enter or E to continue"
        : "Press Enter or E to continue [" + (index + 1) + " / " + total + "]";

    infoModalOpen = true;
    modalInputDelay = 0.35f;
    modalEyebrow.setText(eyebrows != null && index < eyebrows.length ? eyebrows[index] : "");
    modalTitle.setText(titles[index] == null ? "" : titles[index]);
    modalBody.setText(bodies[index] == null ? "" : bodies[index]);
    modalFooter.setText(pageFooter);
}
```



```
modalSidebar.setText(sidebars != null && index < sidebars.length ? sidebars[index] : "");
hideKeypadText();
modalNode.setCullHint(Spatial.CullHint.Never);
setPrompt(null);

final int next = index + 1;
infoModalContinue = () -> {
    if (isLast) {
        closeInfoModal();
        if (onComplete != null) onComplete.run();
    } else {
        openNarrativePage(titles, bodies, eyebrows, sidebars, onComplete, next);
    }
};
}

public void closeInfoModal() {
    infoModalOpen = false;
    infoModalContinue = null;
    hideInfoModalText();
    if (!keypadOpen) {
        modalNode.setCullHint(Spatial.CullHint.Always);
    }
}

/**
 * Overrides the modal accent bar and sidebar colours for the next modal.
 * Call {@link #resetModalTheme()} in the onComplete callback to restore defaults.
 */
public void setModalTheme(ColorRGBA topAccent, ColorRGBA bottomAccent, ColorRGBA sidebar) {
    modalAccentTop.getMaterial().setColor("Color", topAccent);
    modalAccentBottom.getMaterial().setColor("Color", bottomAccent);
    modalSidebarPanel.getMaterial().setColor("Color", sidebar);
}

/** Restores the default cyan / red accent colours. */
public void resetModalTheme() {
    modalAccentTop.getMaterial().setColor("Color", new ColorRGBA(0.15f, 0.78f, 0.98f, 0.95f));
    modalAccentBottom.getMaterial().setColor("Color", new ColorRGBA(0.72f, 0.12f, 0.18f, 0.95f));
    modalSidebarPanel.getMaterial().setColor("Color", new ColorRGBA(0.12f, 0.16f, 0.20f, 0.95f));
}

public boolean isInfoModalOpen() {
    return infoModalOpen;
}

public boolean isOverlayBlockingInput() {
    return keypadOpen || infoModalOpen;
}

private void refreshKeypad() {
    StringBuilder visible = new StringBuilder();
    for (int i = 0; i < 4; i++) {
        visible.append(i < keypadBuffer.length() ? keypadBuffer.charAt(i) : '_');
        visible.append(' ');
    }
}
```



```
}
keypadInput.setText(visible.toString());
}

private void hideKeypadText() {
    keypadTitle.setText("");
    keypadInput.setText("");
    keypadHint.setText("");
}

private void hideInfoModalText() {
    modalEyebrow.setText("");
    modalTitle.setText("");
    modalBody.setText("");
    modalFooter.setText("");
    modalSidebar.setText("");
}

private Geometry createPanel(String name, ColorRGBA color) {
    Geometry geometry = new Geometry(name, new Quad(1f, 1f));
    Material material = new Material(app.getAssetManager(), "Common/MatDefs/Misc/Unshaded.j3md");
    material.setColor("Color", color);
    geometry.setMaterial(material);
    return geometry;
}

private void layoutModal() {
    float sw = app.getCamera().getWidth();
    float sh = app.getCamera().getHeight();

    modalBackdrop.setLocalScale(sw, sh, 1f);
    modalBackdrop.setLocalTranslation(0f, 0f, -5f);

    float panelX = sw * 0.09f;
    float panelY = sh * 0.11f;
    float panelW = sw * 0.82f;
    float panelH = sh * 0.78f;
    modalPanel.setLocalScale(panelW, panelH, 1f);
    modalPanel.setLocalTranslation(panelX, panelY, -4f);

    float sidebarW = panelW * 0.22f;
    modalSidebarPanel.setLocalScale(sidebarW, panelH, 1f);
    modalSidebarPanel.setLocalTranslation(panelX, panelY, -3f);

    modalAccentTop.setLocalScale(panelW, 10f, 1f);
    modalAccentTop.setLocalTranslation(panelX, panelY + panelH - 12f, -2f);

    modalAccentBottom.setLocalScale(panelW * 0.28f, 8f, 1f);
    modalAccentBottom.setLocalTranslation(panelX, panelY, -2f);

    modalFooterPanel.setLocalScale(panelW * 0.62f, 52f, 1f);
    modalFooterPanel.setLocalTranslation(panelX + panelW * 0.31f, panelY + 22f, -2f);

    modalEyebrow.setLocalTranslation(panelX + sidebarW + 42f, panelY + panelH - 44f, 2f);
    modalTitle.setLocalTranslation(panelX + sidebarW + 40f, panelY + panelH - 88f, 2f);
}
```



```
modalBody.setLocalTranslation(panelX + sidebarW + 40f, panelY + panelH - 162f, 2f);
modalFooter.setLocalTranslation(panelX + sidebarW + 58f, panelY + 58f, 2f);
modalSidebar.setLocalTranslation(panelX + 28f, panelY + panelH - 58f, 2f);

keypadTitle.setLocalTranslation(panelX + sidebarW + 58f, panelY + panelH - 110f, 2f);
keypadInput.setLocalTranslation(panelX + sidebarW + 110f, panelY + panelH - 184f, 2f);
keypadHint.setLocalTranslation(panelX + sidebarW + 58f, panelY + 120f, 2f);
}
```

```
private String resolveEyebrow(String title) {
    if (title == null) return "";
    if (title.startsWith("LEVEL 1")) return "ENTRY PROTOCOL";
    if (title.startsWith("LEVEL 2")) return "OBSERVATION SYSTEM";
    if (title.startsWith("LEVEL 3")) return "CONTAINMENT ARCHIVE";
    if (title.startsWith("LEVEL 4")) return "REVELATION CORE";
    if (title.startsWith("AUDIO LOG")) return "DIGITAL FORENSICS";
    if (title.startsWith("LEVEL 1 COMPLETE")) return "MISSION SUCCESS";
    if (title.startsWith("LEVEL 2 COMPLETE")) return "MISSION SUCCESS";
    if (title.startsWith("LEVEL 3 COMPLETE")) return "MISSION SUCCESS";
    if ("ENDGAME".equals(title)) return "FINAL REPORT";
    return "LAB SYSTEM";
}
```

```
private String resolveSidebar(String title, String body) {
    if (title == null) return "";
    if (title.startsWith("AUDIO LOG")) {
        return "ARCHIVE\n\nRecovered log\nindexed into\ncontrol-room\nforensics.";
    }
    if ("ENDGAME".equals(title)) {
        return "STATUS\n\nMemory loop\nresolved.\nFacility silent.";
    }
    if (title.startsWith("LEVEL")) {
        return "BRIEFING\n\nUnderground\nresearch wing\nactive.\n\nReview story\nand mission\nbefore entry.";
    }
    return "SYSTEM\n\n" + (body == null ? "" : body);
}
}
```

16. BlockType.java:

```
package subject47.world;
```

```
public enum BlockType {
    AIR,
    FLOOR,
    WALL,
    CEILING,
    TRIM,
    PANEL,
    GLASS,
    LIGHT
}
```

17. Chunk.java:

```
package subject47.world;
```

```
public class Chunk {
```



```
public static final int SIZE = 48;

private final BlockType[][][] blocks = new BlockType[SIZE][SIZE][SIZE];

public Chunk() {
    fillAir();
}

public void fillAir() {
    for (int x = 0; x < SIZE; x++) {
        for (int y = 0; y < SIZE; y++) {
            for (int z = 0; z < SIZE; z++) {
                blocks[x][y][z] = BlockType.AIR;
            }
        }
    }
}

public void setBlock(int x, int y, int z, BlockType type) {
    if (inBounds(x, y, z)) {
        blocks[x][y][z] = type;
    }
}

public BlockType getBlock(int x, int y, int z) {
    return inBounds(x, y, z) ? blocks[x][y][z] : BlockType.AIR;
}

private boolean inBounds(int x, int y, int z) {
    return x >= 0 && y >= 0 && z >= 0
        && x < SIZE && y < SIZE && z < SIZE;
}
}
```

18. EditorPropType.java:

```
package subject47.world;

public enum EditorPropType {
    AUTO_DOOR,
    KEYPAD_DOOR,
    CONSOLE,
    LAB_BENCH,
    SERVER_RACK,
    STORAGE_CRATE,
    MEDICAL_CART,
    WALL_MONITOR,
    LOCKER_BANK,
    CEILING_LIGHT,
    ANALYZER_STATION,
    TANK_CLUSTER,
    CONTAINMENT_POD
}
```

19. LabGenerator.java:

```
package subject47.world;
```




```
public class LabGenerator {  
  
    private static final int WALL_HEIGHT = 6;  
  
    public void generateLevel1(Chunk chunk) {  
        buildBaseFacility(chunk);  
        corridor(chunk, 18, 18, 24, 20);  
        airlock(chunk, 10, 5, 12, 8);  
        serviceSpine(chunk, 24, 3, 41, 15);  
        observationWindow(chunk, 28, 16, 36);  
        doorPortal(chunk, 11, 7);  
        doorPortal(chunk, 14, 18);  
        doorPortal(chunk, 18, 19);  
        doorPortal(chunk, 24, 20);  
        doorPortal(chunk, 33, 20);  
    }  
  
    public void generateLevel2(Chunk chunk) {  
        buildBaseFacility(chunk);  
        room(chunk, 18, 18, 29, 28, true);  
        lightStrip(chunk, 20, 20, 27, 20, false);  
        serviceSpine(chunk, 7, 15, 18, 24);  
        airlock(chunk, 30, 17, 33, 19);  
        doorPortal(chunk, 14, 18);  
        doorPortal(chunk, 24, 20);  
        doorPortal(chunk, 33, 20);  
        doorPortal(chunk, 31, 17);  
    }  
  
    public void generateLevel3(Chunk chunk) {  
        buildBaseFacility(chunk);  
        room(chunk, 36, 10, 46, 18, false);  
        corridor(chunk, 34, 12, 38, 14);  
        column(chunk, 31, 1, 6);  
        column(chunk, 24, 1, 23);  
        lightStrip(chunk, 36, 14, 44, 14, true);  
        observationWindow(chunk, 36, 11, 16);  
        serviceSpine(chunk, 34, 10, 45, 17);  
        doorPortal(chunk, 31, 8);  
        doorPortal(chunk, 41, 28);  
        doorPortal(chunk, 38, 13);  
    }  
  
    public void generateLevel4(Chunk chunk) {  
        buildBaseFacility(chunk);  
        room(chunk, 34, 18, 46, 30, false);  
        corridor(chunk, 32, 22, 36, 24);  
        lightStrip(chunk, 36, 24, 44, 24, true);  
        serviceSpine(chunk, 34, 19, 45, 29);  
        airlock(chunk, 35, 22, 37, 24);  
        doorPortal(chunk, 35, 24);  
        doorPortal(chunk, 43, 24);  
    }  
}
```



```
private void buildBaseFacility(Chunk chunk) {
    room(chunk, 2, 2, 11, 11, false);
    corridor(chunk, 11, 5, 24, 8);
    room(chunk, 6, 14, 18, 25, false);
    corridor(chunk, 12, 8, 14, 18);
    room(chunk, 24, 2, 41, 16, false);
    room(chunk, 24, 17, 31, 24, true);
    room(chunk, 33, 17, 40, 24, true);
    corridor(chunk, 31, 19, 33, 21);
    corridor(chunk, 34, 24, 38, 26);
    corridor(chunk, 38, 7, 44, 9);
    corridor(chunk, 40, 9, 43, 29);
    corridor(chunk, 32, 27, 46, 29);
    room(chunk, 32, 24, 46, 31, false);

    lightStrip(chunk, 4, 6, 9, 6, false);
    lightStrip(chunk, 13, 6, 23, 6, false);
    lightStrip(chunk, 8, 19, 16, 19, false);
    lightStrip(chunk, 26, 8, 39, 8, false);
    lightStrip(chunk, 25, 20, 30, 20, false);
    lightStrip(chunk, 34, 20, 39, 20, false);
    lightStrip(chunk, 34, 28, 43, 28, false);

    panelBand(chunk, 2, 2, 11, 11);
    panelBand(chunk, 6, 14, 18, 25);
    panelBand(chunk, 24, 2, 41, 16);
    panelBand(chunk, 26, 16, 38, 24);
    panelBand(chunk, 32, 24, 46, 31);
}

private void room(Chunk chunk, int x1, int z1, int x2, int z2, boolean glassWall) {
    for (int x = x1; x <= x2; x++) {
        for (int z = z1; z <= z2; z++) {
            chunk.setBlock(x, 0, z, BlockType.FLOOR);
            chunk.setBlock(x, WALL_HEIGHT, z, BlockType.CEILING);
        }
    }

    for (int y = 1; y < WALL_HEIGHT; y++) {
        for (int x = x1; x <= x2; x++) {
            chunk.setBlock(x, y, z1, edgeType(x, y, true));
            chunk.setBlock(x, y, z2, edgeType(x, y, true));
        }

        for (int z = z1; z <= z2; z++) {
            chunk.setBlock(x1, y, z, edgeType(z, y, true));
            chunk.setBlock(x2, y, z, edgeType(z, y, true));
        }
    }

    ceilingGrid(chunk, x1, z1, x2, z2);

    if (glassWall) {
        int glassZ = z1;
        for (int y = 2; y < WALL_HEIGHT - 1; y++) {
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
        for (int x = x1 + 2; x <= x2 - 2; x++) {
            chunk.setBlock(x, y, glassZ, BlockType.GLASS);
        }
    }
}

private void corridor(Chunk chunk, int x1, int z1, int x2, int z2) {
    room(chunk, x1, z1, x2, z2, false);
    trimRun(chunk, x1, z1, x2, z2);
}

private void lightStrip(Chunk chunk, int x1, int z1, int x2, int z2, boolean redMood) {
    int colorRow = WALL_HEIGHT - 1;
    for (int x = x1; x <= x2; x += 2) {
        for (int z = z1; z <= z2; z += 2) {
            chunk.setBlock(x, colorRow, z, BlockType.LIGHT);
            if (redMood && x + 1 < Chunk.SIZE) {
                chunk.setBlock(x + 1, colorRow, z, BlockType.WALL);
            }
        }
    }
}

private void column(Chunk chunk, int x, int y1, int z) {
    for (int y = y1; y < WALL_HEIGHT; y++) {
        chunk.setBlock(x, y, z, y == 1 ? BlockType.TRIM : BlockType.PANEL);
    }
}

private void airlock(Chunk chunk, int x1, int z1, int x2, int z2) {
    room(chunk, x1, z1, x2, z2, false);
    for (int x = x1 + 1; x < x2; x++) {
        chunk.setBlock(x, 0, z1 + 1, BlockType.TRIM);
        chunk.setBlock(x, WALL_HEIGHT, z1 + 1, BlockType.LIGHT);
    }
}

private void serviceSpine(Chunk chunk, int x1, int z1, int x2, int z2) {
    for (int x = x1; x <= x2; x++) {
        if (((x - x1) & 1) == 0) {
            chunk.setBlock(x, 0, z1, BlockType.TRIM);
            chunk.setBlock(x, 0, z2, BlockType.TRIM);
        }
    }
    for (int z = z1; z <= z2; z++) {
        if (((z - z1) & 2) == 0) {
            chunk.setBlock(x1, 2, z, BlockType.PANEL);
            chunk.setBlock(x2, 2, z, BlockType.PANEL);
        }
    }
}

private void observationWindow(Chunk chunk, int x1, int x2, int z) {
    for (int x = x1; x <= x2; x++) {
```



```
chunk.setBlock(x, 1, z, BlockType.TRIM);
chunk.setBlock(x, WALL_HEIGHT - 1, z, BlockType.TRIM);
for (int y = 2; y < WALL_HEIGHT - 1; y++) {
    chunk.setBlock(x, y, z, BlockType.GLASS);
}
}
}

private void panelBand(Chunk chunk, int x1, int z1, int x2, int z2) {
    for (int x = x1 + 1; x < x2; x += 3) {
        chunk.setBlock(x, 2, z1, BlockType.PANEL);
        chunk.setBlock(x, 2, z2, BlockType.PANEL);
    }
    for (int z = z1 + 1; z < z2; z += 3) {
        chunk.setBlock(x1, 2, z, BlockType.PANEL);
        chunk.setBlock(x2, 2, z, BlockType.PANEL);
    }
}

private void ceilingGrid(Chunk chunk, int x1, int z1, int x2, int z2) {
    for (int x = x1 + 1; x < x2; x++) {
        for (int z = z1 + 1; z < z2; z++) {
            if (((x + z) & 3) == 0) {
                chunk.setBlock(x, WALL_HEIGHT, z, BlockType.TRIM);
            }
        }
    }
}

private void trimRun(Chunk chunk, int x1, int z1, int x2, int z2) {
    for (int x = x1; x <= x2; x++) {
        if (((x - x1) & 1) == 0) {
            chunk.setBlock(x, 0, z1, BlockType.TRIM);
            chunk.setBlock(x, 0, z2, BlockType.TRIM);
        }
    }
    for (int z = z1; z <= z2; z++) {
        if (((z - z1) & 1) == 0) {
            chunk.setBlock(x1, 0, z, BlockType.TRIM);
            chunk.setBlock(x2, 0, z, BlockType.TRIM);
        }
    }
}

private BlockType edgeType(int position, int y, boolean horizontalEdge) {
    if (y == 1 || y == WALL_HEIGHT - 1) {
        return BlockType.TRIM;
    }
    if (horizontalEdge && (position % 4 == 0)) {
        return BlockType.PANEL;
    }
    return BlockType.WALL;
}

private void doorPortal(Chunk chunk, int centerX, int centerZ) {
```



```
for (int y = 1; y < WALL_HEIGHT - 1; y++) {  
    for (int x = centerX - 1; x <= centerX + 1; x++) {  
        for (int z = centerZ - 1; z <= centerZ + 1; z++) {  
            chunk.setBlock(x, y, z, BlockType.AIR);  
        }  
    }  
}  
  
for (int x = centerX - 1; x <= centerX + 1; x++) {  
    for (int z = centerZ - 1; z <= centerZ + 1; z++) {  
        chunk.setBlock(x, 0, z, BlockType.FLOOR);  
        chunk.setBlock(x, WALL_HEIGHT, z, BlockType.CEILING);  
    }  
}
```

20. WorldManager.java:

```
package subject47.world;  
  
import java.io.IOException;  
import java.util.ArrayList;  
import java.util.HashMap;  
import java.util.List;  
import java.util.Map;  
import java.nio.file.Files;  
import java.nio.file.Path;  
import java.nio.file.Paths;  
import com.jme3.collison.CollisionResult;  
import com.jme3.collison.CollisionResults;  
import com.jme3.material.Material;  
import com.jme3.material.RenderState;  
import com.jme3.math.ColorRGBA;  
import com.jme3.math.FastMath;  
import com.jme3.math.Quaternion;  
import com.jme3.math.Ray;  
import com.jme3.math.Vector3f;  
import com.jme3.render.queue.RenderQueue;  
import com.jme3.scene.Geometry;  
import com.jme3.scene.Node;  
import com.jme3.scene.Spatial;  
import com.jme3.scene.shape.Box;  
import subject47.Main;  
import subject47.interaction.ControlSwitch;  
import subject47.interaction.Door;  
import subject47.interaction.Interactable;  
import subject47.interaction.Keypad;  
  
public class WorldManager {  
  
    private static final String INTERACTABLE_ID = "interactableId";  
    private static final String EDITOR_PROP_ID = "editorPropId";  
    private static final String LEVEL_PROP_ID = "levelPropId";  
    private static final Path MAP_DIRECTORY = Paths.get("maps");  
  
    private final Main app;  
    private final Node worldNode = new Node("World");
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
private final Node propNode = new Node("Props");
private final Node ghostNode = new Node("Ghost");
private final LabGenerator generator = new LabGenerator();
private final List<Material> lightMaterials = new ArrayList<>();
private final Map<String, Interactable> interactables = new HashMap<>();
private final List<Door> animatedDoors = new ArrayList<>();
private final Map<String, EditorPropInstance> editorProps = new HashMap<>();
private final Map<String, Spatial> levelProps = new HashMap<>();

/** Ghost preview material (cyan, semi-transparent). Created once. */
private Material ghostMaterial;

/**
 * PC desk model parsed once from disk. Volatile so the background preload
 * thread's write is visible to the render thread without synchronisation.
 */
private volatile Spatial cachedPcModel;
private Thread pcModelPreloadThread;

private Chunk currentChunk;
private int currentLevel;
private float lightFlickerTime;
private boolean emergencyLighting;
private int nextInteractableId;
private int nextEditorPropId;
private boolean savedPropsLoaded;

public WorldManager(Main app) {
    this.app = app;
}

public void initialize() {
    app.getRootNode().attachChild(worldNode);
    app.getRootNode().attachChild(propNode);
    app.getRootNode().attachChild(ghostNode);
    // Register the assets/ folder so assetManager.loadModel() can find
    // OBJ/FBX files placed there (by default it only searches the classpath).
    app.getAssetManager().registerLocator("assets/", com.jme3.asset.plugins.FileLocator.class);
    // Parse the heavy OBJ on a background thread while the main menu is
    // visible, so Level 1 loads instantly when the player starts the game.
    pcModelPreloadThread = new Thread(() -> {
        try {
            Spatial model = app.getAssetManager().loadModel("Pc/pc_desk.obj");
            applyUniformMaterial(model,
                coloredMaterial(new ColorRGBA(0.13f, 0.15f, 0.19f, 1f), false),
                coloredMaterial(new ColorRGBA(0.22f, 0.25f, 0.30f, 1f), false));
            cachedPcModel = model;
            System.out.println("[WorldManager] PC model preloaded.");
        } catch (Exception e) {
            System.err.println("[WorldManager] PC model preload failed: " + e.getMessage());
        }
    }, "PC-Model-Preload");
    pcModelPreloadThread.setDaemon(true);
    pcModelPreloadThread.start();
}
```




```
public void loadLevel(int level) {
    currentLevel = level;
    propNode.detachAllChildren();
    worldNode.detachAllChildren();
    ghostNode.detachAllChildren();
    lightMaterials.clear();
    interactables.clear();
    animatedDoors.clear();
    editorProps.clear();
    levelProps.clear();
    nextInteractableId = 0;
    nextEditorPropId = 0;
    emergencyLighting = level >= 3;

    currentChunk = new Chunk();

    if (!loadSavedChunk(level)) {
        switch (level) {
            case 1 -> generator.generateLevel1(currentChunk);
            case 2 -> generator.generateLevel2(currentChunk);
            case 3 -> generator.generateLevel3(currentChunk);
            case 4 -> generator.generateLevel4(currentChunk);
            default -> generator.generateLevel1(currentChunk);
        }
    }

    renderChunk();
    // loadSavedProps returns true when a saved props file was found.
    // In that case, level decoration is skipped so the saved state is authoritative.
    savedPropsLoaded = loadSavedProps(level);
}

public void update(float tpf) {
    if (lightFlickerTime > 0f) {
        lightFlickerTime -= tpf;
        float scale = ((int) (lightFlickerTime * 18f) % 2 == 0) ? 0.12f : 1f;
        applyLightState(scale);
        if (lightFlickerTime <= 0f) {
            applyLightState(1f);
        }
    }
}

for (Door door : animatedDoors) {
    door.update(tpf);
}

public void flickerLights(float duration) {
    lightFlickerTime = Math.max(lightFlickerTime, duration);
}

public void setEmergencyLighting(boolean enabled) {
    emergencyLighting = enabled;
    applyLightState(1f);
}
```



}

```
public Door createDoor(String name, Vector3f position, boolean locked) {  
    Node doorNode = new Node(name);
```

```
    Geometry frame = new Geometry(name + "Frame", new Box(0.86f, 1.62f, 0.18f));  
    frame.setMaterial(coloredMaterial(new ColorRGBA(0.05f, 0.08f, 0.1f, 1f), false));  
    frame.setLocalTranslation(0f, 0f, -0.08f);  
    doorNode.attachChild(frame);
```

```
    Geometry lintel = new Geometry(name + "Lintel", new Box(0.72f, 0.16f, 0.08f));  
    lintel.setMaterial(coloredMaterial(new ColorRGBA(0.7f, 0.08f, 0.08f, 1f), true));  
    lintel.setLocalTranslation(0f, 1.32f, 0.06f);  
    doorNode.attachChild(lintel);
```

```
    Geometry panelLeft = new Geometry(name + "PanelLeft", new Box(0.28f, 1.32f, 0.07f));  
    panelLeft.setMaterial(coloredMaterial(new ColorRGBA(0.32f, 0.36f, 0.42f, 1f), false));  
    panelLeft.setLocalTranslation(-0.3f, 0f, 0.08f);  
    doorNode.attachChild(panelLeft);
```

```
    Geometry panelRight = new Geometry(name + "PanelRight", new Box(0.28f, 1.32f, 0.07f));  
    panelRight.setMaterial(coloredMaterial(new ColorRGBA(0.32f, 0.36f, 0.42f, 1f), false));  
    panelRight.setLocalTranslation(0.3f, 0f, 0.08f);  
    doorNode.attachChild(panelRight);
```

```
    Geometry seam = new Geometry(name + "Seam", new Box(0.03f, 1.12f, 0.075f));  
    seam.setMaterial(coloredMaterial(new ColorRGBA(0.12f, 0.14f, 0.16f, 1f), false));  
    seam.setLocalTranslation(0f, -0.05f, 0.09f);  
    doorNode.attachChild(seam);
```

```
    Door door = new Door(app, doorNode, name, position, new Vector3f(0f, 2.8f, 0f), locked);  
    tagSpatial(doorNode, door);  
    animatedDoors.add(door);  
    propNode.attachChild(doorNode);  
    return door;
```

}

```
public Spatial createConsole(String name, Vector3f position, ColorRGBA color, Interactable interactable) {  
    Node console = new Node(name);
```

```
    Geometry base = new Geometry(name + "Base", new Box(0.6f, 0.8f, 0.4f));  
    base.setMaterial(coloredMaterial(new ColorRGBA(0.1f, 0.12f, 0.14f, 1f), false));  
    console.attachChild(base);
```

```
    Geometry screen = new Geometry(name + "Screen", new Box(0.48f, 0.35f, 0.03f));  
    screen.setMaterial(coloredMaterial(color, true));  
    screen.setLocalTranslation(0f, 0.35f, 0.43f);  
    console.attachChild(screen);
```

```
    console.setLocalTranslation(position);  
    tagSpatial(console, interactable);  
    propNode.attachChild(console);  
    return console;
```

}



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
public Spatial createAudioLogPickup(String name, Vector3f position, ColorRGBA color, Interactable interactable) {
    Geometry pickup = new Geometry(name, new Box(0.24f, 0.12f, 0.24f));
    pickup.setMaterial(coloredMaterial(color, true));
    pickup.setLocalTranslation(position);
    tagSpatial(pickup, interactable);
    propNode.attachChild(pickup);
    return pickup;
}

/**
 * Loads the PC desk 3-D model from assets/Pc/ and places it at the given
 * position with the supplied interactable wired up (same contract as
 * {@link #createAudioLogPickup}). Falls back to a coloured box if the
 * model cannot be loaded so the rest of the level still works.
 */
public Spatial createPcModel(String name, Vector3f position, ColorRGBA fallbackColor, Interactable interactable) {
    final String MODEL_PATH = "Pc/pc_desk.obj";
    try {
        // If the background preload thread is still running, wait for it.
        // In normal play the thread finishes during the main menu, so this is instant.
        if (cachedPcModel == null && pcModelPreloadThread != null && pcModelPreloadThread.isAlive()) {
            pcModelPreloadThread.join(30_000);
        }
        // Fallback: if preload failed or timed out, load synchronously now.
        if (cachedPcModel == null) {
            cachedPcModel = app.getAssetManager().loadModel(MODEL_PATH);
            applyUniformMaterial(cachedPcModel,
                coloredMaterial(new ColorRGBA(0.13f, 0.15f, 0.19f, 1f), false),
                coloredMaterial(new ColorRGBA(0.22f, 0.25f, 0.30f, 1f), false));
        }
        Spatial model = cachedPcModel.clone();
        model.setName(name);
        model.setLocalScale(0.2f);
        model.setLocalTranslation(position.x, position.y - 1.0f, position.z);
        tagSpatial(model, interactable);
        propNode.attachChild(model);
        return model;
    } catch (Exception e) {
        System.err.println("[WorldManager] Could not load PC model (" + MODEL_PATH + "): " + e.getMessage());
        return createAudioLogPickup(name, position, new ColorRGBA(1f, 0.1f, 0.1f, 1f), interactable);
    }
}

/**
 * Walks the spatial tree and alternates between two materials so the model
 * has some visual depth without needing real textures.
 */
private void applyUniformMaterial(Spatial spatial, Material primary, Material accent) {
    applyUniformMaterialRecursive(spatial, primary, accent, new int[]{0});
}

private void applyUniformMaterialRecursive(Spatial spatial, Material primary, Material accent, int[] counter) {
    if (spatial instanceof Geometry geo) {
        geo.setMaterial((counter[0]++ % 3 == 0) ? accent : primary);
    } else if (spatial instanceof Node node) {

```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
    for (Spatial child : node.getChildren()) {
        applyUniformMaterialRecursive(child, primary, accent, counter);
    }
}

public Spatial createLabBench(String name, Vector3f position, float width, ColorRGBA accentColor) {
    Node bench = new Node(name);

    Geometry top = new Geometry(name + "Top", new Box(width, 0.08f, 0.45f));
    top.setMaterial(coloredMaterial(new ColorRGBA(0.78f, 0.82f, 0.86f, 1f), false));
    top.setLocalTranslation(0f, 0.85f, 0f);
    bench.attachChild(top);

    Geometry supportLeft = new Geometry(name + "SupportLeft", new Box(0.08f, 0.75f, 0.08f));
    supportLeft.setMaterial(coloredMaterial(new ColorRGBA(0.18f, 0.2f, 0.24f, 1f), false));
    supportLeft.setLocalTranslation(-width + 0.08f, 0.05f, -0.3f);
    bench.attachChild(supportLeft);

    Geometry supportRight = new Geometry(name + "SupportRight", new Box(0.08f, 0.75f, 0.08f));
    supportRight.setMaterial(coloredMaterial(new ColorRGBA(0.18f, 0.2f, 0.24f, 1f), false));
    supportRight.setLocalTranslation(width - 0.08f, 0.05f, -0.3f);
    bench.attachChild(supportRight);

    Geometry cabinet = new Geometry(name + "Cabinet", new Box(width * 0.48f, 0.5f, 0.32f));
    cabinet.setMaterial(coloredMaterial(new ColorRGBA(0.12f, 0.14f, 0.17f, 1f), false));
    cabinet.setLocalTranslation(0f, 0.32f, 0.05f);
    bench.attachChild(cabinet);

    Geometry monitor = new Geometry(name + "Monitor", new Box(0.22f, 0.18f, 0.03f));
    monitor.setMaterial(coloredMaterial(accentColor, true));
    monitor.setLocalTranslation(0f, 1.12f, -0.15f);
    bench.attachChild(monitor);

    bench.setLocalTranslation(position);
    propNode.attachChild(bench);
    return bench;
}

public Spatial createServerRack(String name, Vector3f position, ColorRGBA accentColor) {
    Node rack = new Node(name);

    Geometry body = new Geometry(name + "Body", new Box(0.45f, 1.2f, 0.45f));
    body.setMaterial(coloredMaterial(new ColorRGBA(0.08f, 0.1f, 0.12f, 1f), false));
    rack.attachChild(body);

    for (int i = 0; i < 4; i++) {
        Geometry panel = new Geometry(name + "Panel" + i, new Box(0.32f, 0.14f, 0.02f));
        panel.setMaterial(coloredMaterial(accentColor.mult(0.65f + (i * 0.08f)), true));
        panel.setLocalTranslation(0f, -0.7f + (i * 0.48f), 0.47f);
        rack.attachChild(panel);
    }

    rack.setLocalTranslation(position);
    propNode.attachChild(rack);
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
    return rack;
}

public Spatial createContainmentPod(String name, Vector3f position, ColorRGBA glowColor) {
    Node pod = new Node(name);

    Geometry base = new Geometry(name + "Base", new Box(0.7f, 0.18f, 1.1f));
    base.setMaterial(coloredMaterial(new ColorRGBA(0.15f, 0.18f, 0.22f, 1f), false));
    pod.attachChild(base);

    Geometry chamber = new Geometry(name + "Chamber", new Box(0.5f, 0.95f, 0.95f));
    chamber.setMaterial(coloredMaterial(new ColorRGBA(0.45f, 0.7f, 0.9f, 0.22f), true));
    chamber.setQueueBucket(RenderQueue.Bucket.Transparent);
    chamber.setLocalTranslation(0f, 0.95f, 0f);
    pod.attachChild(chamber);

    Geometry core = new Geometry(name + "Core", new Box(0.18f, 0.55f, 0.18f));
    core.setMaterial(coloredMaterial(glowColor, true));
    core.setLocalTranslation(0f, 0.95f, 0f);
    pod.attachChild(core);

    pod.setLocalTranslation(position);
    propNode.attachChild(pod);
    return pod;
}

public Spatial createStorageCrate(String name, Vector3f position, ColorRGBA stripeColor) {
    Node crate = new Node(name);

    Geometry body = new Geometry(name + "Body", new Box(0.45f, 0.45f, 0.45f));
    body.setMaterial(coloredMaterial(new ColorRGBA(0.28f, 0.3f, 0.33f, 1f), false));
    crate.attachChild(body);

    Geometry stripe = new Geometry(name + "Stripe", new Box(0.4f, 0.06f, 0.48f));
    stripe.setMaterial(coloredMaterial(stripeColor, true));
    stripe.setLocalTranslation(0f, 0.16f, 0f);
    crate.attachChild(stripe);

    crate.setLocalTranslation(position);
    propNode.attachChild(crate);
    return crate;
}

public Spatial createMedicalCart(String name, Vector3f position, ColorRGBA accentColor) {
    Node cart = new Node(name);

    Geometry tray = new Geometry(name + "Tray", new Box(0.42f, 0.08f, 0.28f));
    tray.setMaterial(coloredMaterial(new ColorRGBA(0.75f, 0.79f, 0.83f, 1f), false));
    tray.setLocalTranslation(0f, 0.82f, 0f);
    cart.attachChild(tray);

    Geometry base = new Geometry(name + "Base", new Box(0.3f, 0.42f, 0.22f));
    base.setMaterial(coloredMaterial(new ColorRGBA(0.22f, 0.25f, 0.28f, 1f), false));
    base.setLocalTranslation(0f, 0.34f, 0f);
    cart.attachChild(base);
}
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
Geometry screen = new Geometry(name + "Screen", new Box(0.18f, 0.12f, 0.03f));
screen.setMaterial(coloredMaterial(accentColor, true));
screen.setLocalTranslation(0f, 1.02f, -0.1f);
cart.attachChild(screen);

cart.setLocalTranslation(position);
propNode.attachChild(cart);
return cart;
}

public Spatial createWallMonitor(String name, Vector3f position, ColorRGBA accentColor) {
    Node monitor = new Node(name);

    Geometry panel = new Geometry(name + "Panel", new Box(0.44f, 0.28f, 0.04f));
    panel.setMaterial(coloredMaterial(new ColorRGBA(0.08f, 0.1f, 0.12f, 1f), false));
    monitor.attachChild(panel);

    Geometry screen = new Geometry(name + "Screen", new Box(0.36f, 0.2f, 0.02f));
    screen.setMaterial(coloredMaterial(accentColor, true));
    screen.setLocalTranslation(0f, 0f, 0.05f);
    monitor.attachChild(screen);

    monitor.setLocalTranslation(position);
    propNode.attachChild(monitor);
    return monitor;
}

public Spatial createLockerBank(String name, Vector3f position, ColorRGBA accentColor) {
    Node lockers = new Node(name);

    for (int i = 0; i < 3; i++) {
        Geometry locker = new Geometry(name + "Locker" + i, new Box(0.24f, 0.9f, 0.2f));
        locker.setMaterial(coloredMaterial(new ColorRGBA(0.26f, 0.29f, 0.33f, 1f), false));
        locker.setLocalTranslation((i - 1) * 0.54f, 0.9f, 0f);
        lockers.attachChild(locker);

        Geometry strip = new Geometry(name + "Strip" + i, new Box(0.18f, 0.04f, 0.21f));
        strip.setMaterial(coloredMaterial(accentColor, true));
        strip.setLocalTranslation((i - 1) * 0.54f, 1.42f, 0f);
        lockers.attachChild(strip);
    }

    lockers.setLocalTranslation(position);
    propNode.attachChild(lockers);
    return lockers;
}

public Spatial createDoorButton(String name, Vector3f position, Interactable interactable) {
    Node button = new Node(name);

    Geometry base = new Geometry(name + "Base", new Box(0.12f, 0.18f, 0.08f));
    base.setMaterial(coloredMaterial(new ColorRGBA(0.18f, 0.2f, 0.22f, 1f), false));
    button.attachChild(base);
```




National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
Geometry cap = new Geometry(name + "Cap", new Box(0.08f, 0.08f, 0.03f));
cap.setMaterial(coloredMaterial(new ColorRGBA(0.92f, 0.12f, 0.12f, 1f), true));
cap.setLocalTranslation(0f, 0f, 0.09f);
button.attachChild(cap);

button.setLocalTranslation(position);
tagSpatial(button, interactable);
propNode.attachChild(button);
return button;
}

public Spatial createCeilingLight(String name, Vector3f position, ColorRGBA glowColor) {
    Node light = new Node(name);

    Geometry housing = new Geometry(name + "Housing", new Box(0.4f, 0.08f, 0.4f));
    housing.setMaterial(coloredMaterial(new ColorRGBA(0.28f, 0.32f, 0.35f, 1f), false));
    light.attachChild(housing);

    Geometry bulb = new Geometry(name + "Bulb", new Box(0.26f, 0.04f, 0.26f));
    bulb.setMaterial(coloredMaterial(glowColor, true));
    bulb.setLocalTranslation(0f, -0.12f, 0f);
    light.attachChild(bulb);

    light.setLocalTranslation(position);
    propNode.attachChild(light);
    return light;
}

public Spatial createAnalyzerStation(String name, Vector3f position, ColorRGBA glowColor) {
    Node station = new Node(name);

    Geometry base = new Geometry(name + "Base", new Box(0.35f, 0.75f, 0.35f));
    base.setMaterial(coloredMaterial(new ColorRGBA(0.2f, 0.22f, 0.26f, 1f), false));
    station.attachChild(base);

    Geometry scanner = new Geometry(name + "Scanner", new Box(0.22f, 0.08f, 0.22f));
    scanner.setMaterial(coloredMaterial(glowColor, true));
    scanner.setLocalTranslation(0f, 0.86f, 0f);
    station.attachChild(scanner);

    Geometry arm = new Geometry(name + "Arm", new Box(0.06f, 0.24f, 0.06f));
    arm.setMaterial(coloredMaterial(new ColorRGBA(0.55f, 0.58f, 0.62f, 1f), false));
    arm.setLocalTranslation(0.18f, 1.05f, 0f);
    station.attachChild(arm);

    station.setLocalTranslation(position);
    propNode.attachChild(station);
    return station;
}

public Spatial createTankCluster(String name, Vector3f position, ColorRGBA glowColor) {
    Node tanks = new Node(name);
    for (int i = 0; i < 3; i++) {
        Geometry tank = new Geometry(name + "Tank" + i, new Box(0.18f, 0.82f, 0.18f));
        tank.setMaterial(coloredMaterial(new ColorRGBA(0.16f, 0.19f, 0.22f, 1f), false));
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
tank.setLocalTranslation((i - 1) * 0.34f, 0.82f, 0f);
tanks.attachChild(tank);

Geometry gauge = new Geometry(name + "Gauge" + i, new Box(0.06f, 0.06f, 0.2f));
gauge.setMaterial(coloredMaterial(glowColor, true));
gauge.setLocalTranslation((i - 1) * 0.34f, 1.18f, 0.2f);
tanks.attachChild(gauge);
}
tanks.setLocalTranslation(position);
propNode.attachChild(tanks);
return tanks;
}

public void removeProp(Spatial spatial) {
    if (spatial != null) {
        spatial.removeFromParent();
    }
}

public Vector3f getSpawnPoint(int level) {
    return switch (level) {
        case 1 -> worldToScene(5, 1.8f, 6);
        case 2 -> worldToScene(8, 1.8f, 18);
        case 3 -> worldToScene(28, 1.8f, 7);
        case 4 -> worldToScene(35, 1.8f, 22);
        default -> worldToScene(5, 1.8f, 6);
    };
}

public Vector3f worldToScene(float x, float y, float z) {
    return new Vector3f(x * 1.2f, y * 1.2f, z * 1.2f);
}

public Interactable getInteractable(Spatial spatial) {
    Spatial current = spatial;
    while (current != null) {
        String interactableId = current.getUserData(INTERACTABLE_ID);
        if (interactableId != null) {
            return interactables.get(interactableId);
        }
        current = current.getParent();
    }
    return null;
}

public void placeEditorProp(EditorPropType type, BlockHit hit, Vector3f viewDirection, int rotation) {
    Vector3f pos = computePlacementPosition(type, hit, viewDirection);
    String propId = "editor-prop-" + nextEditorPropId++;
    EditorPropInstance instance = buildEditorProp(propId, type, pos, rotation, viewDirection);
    if (instance != null) {
        editorProps.put(propId, instance);
    }
}
```

// -----



```
// Ghost preview
// -----

/**
 * Updates (or creates) a translucent ghost box at the predicted placement
 * position for the given prop type and block hit. Call every frame while
 * the editor is in prop mode.
 */
public void updateEditorGhost(EditorPropType type, BlockHit hit, Vector3f viewDirection, int rotation) {
    ghostNode.detachAllChildren();
    if (hit == null) return;

    if (ghostMaterial == null) {
        ghostMaterial = new Material(app.getAssetManager(), "Common/MatDefs/Misc/Unshaded.j3md");
        ghostMaterial.setColor("Color", new ColorRGBA(0.2f, 0.85f, 1f, 0.35f));
        ghostMaterial.getAdditionalRenderState().setBlendMode(RenderState.BlendMode.Alpha);
    }

    Vector3f pos = computePlacementPosition(type, hit, viewDirection);
    float[] hs = getPropHalfExtents(type);

    Geometry ghost = new Geometry("ghost", new Box(hs[0], hs[1], hs[2]));
    ghost.setMaterial(ghostMaterial);
    ghost.setQueueBucket(RenderQueue.Bucket.Transparent);
    ghost.setLocalTranslation(pos);
    if (rotation != 0) {
        ghost.setLocalRotation(new Quaternion().fromAngleAxis(rotation * FastMath.HALF_PI, Vector3f.UNIT_Y));
    }
    ghostNode.attachChild(ghost);
}

/** Removes the ghost preview. Call when leaving prop mode or editor. */
public void clearEditorGhost() {
    ghostNode.detachAllChildren();
}

/**
 * Computes the world-space placement position for a prop, respecting the
 * face normal of the hit block so doors snap to the floor Y they're placed on.
 */
public Vector3f computePlacementPosition(EditorPropType type, BlockHit hit, Vector3f viewDirection) {
    // Step into the adjacent AIR cell via the face normal.
    int tx = hit.x() + Math.round(hit.normal().x);
    int ty = hit.y() + Math.round(hit.normal().y);
    int tz = hit.z() + Math.round(hit.normal().z);

    // Scan DOWN from ty to find the highest non-AIR block in the (tx,tz) column.
    // We use isAnyBlock() (any non-AIR type) so FLOOR, LIGHT, etc. are all detected.
    // floorBlock = Y of the solid surface block the prop will stand on.
    int floorBlock = 0; // default: ground
    for (int scanY = ty; scanY >= 0; scanY--) {
        if (currentChunk.getBlock(tx, scanY, tz) != BlockType.AIR) {
            floorBlock = scanY; // this is the surface block itself
            break;
        }
    }
}
```



```
}

// Y offsets below exactly match the original hardcoded level-dress values.
// floorBlock is the solid surface block Y (e.g. 0 at ground level).
return switch (type) {
    case AUTO_DOOR      -> worldToScene(tx, floorBlock + 1.5f, tz);
    case KEYPAD_DOOR    -> worldToScene(tx, floorBlock + 1.5f, tz);
    case CONSOLE        -> worldToScene(tx, floorBlock + 1.2f, tz);
    case LAB_BENCH      -> worldToScene(tx, floorBlock,      tz);
    case SERVER_RACK    -> worldToScene(tx, floorBlock + 1.2f, tz);
    case STORAGE_CRATE  -> worldToScene(tx, floorBlock + 0.45f, tz);
    case MEDICAL_CART   -> worldToScene(tx, floorBlock + 0.4f,  tz);
    case WALL_MONITOR   -> worldToScene(tx, floorBlock + 1.5f, tz);
    case LOCKER_BANK    -> worldToScene(tx, floorBlock,      tz);
    case ANALYZER_STATION -> worldToScene(tx, floorBlock,      tz);
    case TANK_CLUSTER   -> worldToScene(tx, floorBlock,      tz);
    case CONTAINMENT_POD -> worldToScene(tx, floorBlock + 0.2f, tz);
    case CEILING_LIGHT  -> worldToScene(tx, ty + 0.75f,      tz);
};
}

/**
 * Returns [halfX, halfY, halfZ] approximate visual extents for the ghost box
 * of the given prop type.
 */
private float[] getPropHalfExtents(EditorPropType type) {
    return switch (type) {
        case AUTO_DOOR      -> new float[]{0.86f, 1.62f, 0.18f};
        case KEYPAD_DOOR    -> new float[]{0.86f, 1.62f, 0.18f};
        case CONSOLE        -> new float[]{0.62f, 0.95f, 0.45f};
        case LAB_BENCH      -> new float[]{1.1f, 0.9f, 0.5f};
        case SERVER_RACK    -> new float[]{0.52f, 1.3f, 0.35f};
        case STORAGE_CRATE  -> new float[]{0.52f, 0.5f, 0.52f};
        case MEDICAL_CART   -> new float[]{0.48f, 0.9f, 0.32f};
        case WALL_MONITOR   -> new float[]{0.46f, 0.32f, 0.08f};
        case LOCKER_BANK    -> new float[]{0.9f, 1.8f, 0.24f};
        case CEILING_LIGHT  -> new float[]{0.32f, 0.18f, 0.32f};
        case ANALYZER_STATION -> new float[]{0.62f, 0.85f, 0.45f};
        case TANK_CLUSTER   -> new float[]{0.55f, 0.9f, 0.55f};
        case CONTAINMENT_POD -> new float[]{0.52f, 1.1f, 0.52f};
    };
}

/**
 * Finds the nearest prop (editor-placed or level-dressed) that the ray hits.
 * Returns a compound string "editor:propId" or "level:propId" so the caller
 * knows which map to remove from.
 */
public String findAnyPropHit(Vector3f origin, Vector3f direction, float maxDistance) {
    CollisionResults results = new CollisionResults();
    propNode.collideWith(new Ray(origin, direction), results);
    for (CollisionResult result : results) {
        if (result.getDistance() > maxDistance) {
            break;
        }
    }
}
```



```
// Check editor props first
String editorPropId = findTagInHierarchy(result.getGeometry(), EDITOR_PROP_ID);
if (editorPropId != null) {
    return "editor:" + editorPropId;
}
// Fall back to level props
String levelPropId = findTagInHierarchy(result.getGeometry(), LEVEL_PROP_ID);
if (levelPropId != null) {
    return "level:" + levelPropId;
}
}
return null;
}

/** Legacy alias kept for any existing call-sites. */
public String findEditorPropHit(Vector3f origin, Vector3f direction, float maxDistance) {
    String hit = findAnyPropHit(origin, direction, maxDistance);
    if (hit == null) return null;
    // Strip the prefix and return the raw id regardless of origin
    return hit.substring(hit.indexOf('.') + 1);
}

public boolean removeEditorProp(String propId) {
    EditorPropInstance instance = editorProps.remove(propId);
    if (instance == null) {
        return false;
    }
    for (Spatial spatial : instance.spatials()) {
        spatial.removeFromParent();
    }
    if (instance.door() != null) {
        animatedDoors.remove(instance.door());
    }
    return true;
}

/**
 * Removes a level-dressed prop by its id.
 * Returns true if a prop was found and removed.
 */
public boolean removeLevelProp(String propId) {
    Spatial spatial = levelProps.remove(propId);
    if (spatial == null) {
        return false;
    }
    spatial.removeFromParent();
    // If this spatial is a door node, also unregister the Door object.
    for (Door door : new ArrayList<>(animatedDoors)) {
        if (door.getSpatial() == spatial) {
            animatedDoors.remove(door);
            break;
        }
    }
    return true;
}
```




```
public boolean hasSavedProps(int level) {
    return Files.exists(getPropPath(level));
}

public boolean wasSavedPropsLoaded() {
    return savedPropsLoaded;
}

public BlockHit findBlockHit(Vector3f origin, Vector3f direction, float maxDistance) {
    CollisionResults results = new CollisionResults();
    Ray ray = new Ray(origin, direction.normalize());
    worldNode.collideWith(ray, results);

    for (CollisionResult cr : results) {
        if (cr.getDistance() > maxDistance) break;

        // Sample slightly behind the hit surface to identify the block
        Vector3f contact = cr.getContactPoint();
        Vector3f normal = cr.getContactNormal();
        if (normal == null) continue;

        // Move 1 mm inside the surface so toChunkIndex lands in the solid block
        float eps = 0.001f;
        int x = toChunkIndex(contact.x - normal.x * eps);
        int y = toChunkIndex(contact.y - normal.y * eps);
        int z = toChunkIndex(contact.z - normal.z * eps);

        BlockType type = currentChunk.getBlock(x, y, z);
        if (type == BlockType.AIR) continue; // glass-back-face or stale geometry

        return new BlockHit(x, y, z, type, quantizeNormal(normal));
    }
    return null;
}

/** Snaps an arbitrary surface normal to the nearest cardinal axis direction. */
private Vector3f quantizeNormal(Vector3f n) {
    float ax = Math.abs(n.x), ay = Math.abs(n.y), az = Math.abs(n.z);
    if (ax >= ay && ax >= az) return new Vector3f(Math.signum(n.x), 0f, 0f);
    if (ay >= ax && ay >= az) return new Vector3f(0f, Math.signum(n.y), 0f);
    return new Vector3f(0f, 0f, Math.signum(n.z));
}

public boolean isWalkable(Vector3f position, float radius, float eyeHeight) {
    float lowerSample = position.y - eyeHeight + 0.15f;
    float midSample = position.y - (eyeHeight * 0.45f);
    float upperSample = position.y - 0.15f;
    float[] xOffsets = {0f, -radius, radius, 0f, 0f, -radius, radius, -radius, radius};
    float[] zOffsets = {0f, 0f, 0f, -radius, radius, -radius, -radius, radius, radius};
    float[] ySamples = {lowerSample, midSample, upperSample};

    for (int i = 0; i < xOffsets.length; i++) {
        float sampleX = position.x + xOffsets[i];
        float sampleZ = position.z + zOffsets[i];
    }
}
```




```
        for (float sampleY : ySamples) {
            if (isSolidWorldPoint(sampleX, sampleY, sampleZ)) {
                return false;
            }
        }
    }
    return true;
}

public void setBlock(int x, int y, int z, BlockType type) {
    currentChunk.setBlock(x, y, z, type);
    renderChunk();
}

public BlockType getBlock(int x, int y, int z) {
    return currentChunk.getBlock(x, y, z);
}

public int toChunkIndex(float coordinate) {
    return (int) Math.floor((coordinate + 0.6f) / 1.2f);
}

public void saveCurrentChunk(int level) {
    try {
        Files.createDirectories(MAP_DIRECTORY);
        List<String> lines = new ArrayList<>();
        lines.add("SUBJECT47_MAP 1");
        for (int x = 0; x < Chunk.SIZE; x++) {
            for (int y = 0; y < Chunk.SIZE; y++) {
                for (int z = 0; z < Chunk.SIZE; z++) {
                    BlockType type = currentChunk.getBlock(x, y, z);
                    if (type != BlockType.AIR) {
                        lines.add(x + "," + y + "," + z + "," + type.name());
                    }
                }
            }
        }
        Files.write(getMapPath(level), lines);
    } catch (IOException exception) {
        app.getUi().showMessage("Failed to save map: " + exception.getMessage(), 3f);
    }
}

public void saveEditorProps(int level) {
    try {
        Files.createDirectories(MAP_DIRECTORY);
        List<String> lines = new ArrayList<>();
        lines.add("SUBJECT47_PROPS 3");
        for (Map.Entry<String, Spatial> entry : levelProps.entrySet()) {
            Object record = entry.getValue().getUserData("levelPropRecord");
            if (record instanceof String recordStr) {
                lines.add("LEVEL:" + recordStr + ",0");
            }
        }
    }
    // Save actual scene position (floats) so reload is pixel-perfect.
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
for (EditorPropInstance inst : editorProps.values()) {
    lines.add(inst.type().name() + ", "
        + inst.pos().x + ", " + inst.pos().y + ", " + inst.pos().z
        + ", " + inst.rotation());
}
Files.write(getPropPath(level), lines);
} catch (IOException exception) {
    app.getUi().showMessage("Failed to save props: " + exception.getMessage(), 3f);
}
}

private void renderChunk() {
    worldNode.detachAllChildren();
    for (int x = 0; x < Chunk.SIZE; x++) {
        for (int y = 0; y < Chunk.SIZE; y++) {
            for (int z = 0; z < Chunk.SIZE; z++) {
                BlockType type = currentChunk.getBlock(x, y, z);
                if (type == BlockType.AIR) {
                    continue;
                }

                Geometry block = new Geometry("block", new Box(0.6f, 0.6f, 0.6f));
                Material material = createBlockMaterial(type);
                block.setMaterial(material);
                block.setLocalTranslation(x * 1.2f, y * 1.2f, z * 1.2f);

                if (type == BlockType.GLASS) {
                    block.setQueueBucket(RenderQueue.Bucket.Transparent);
                }

                worldNode.attachChild(block);
            }
        }
    }
}

private Material createBlockMaterial(BlockType type) {
    return switch (type) {
        case FLOOR -> coloredMaterial(new ColorRGBA(0.72f, 0.76f, 0.79f, 1f), false);
        case WALL -> coloredMaterial(new ColorRGBA(0.2f, 0.23f, 0.27f, 1f), false);
        case CEILING -> coloredMaterial(new ColorRGBA(0.64f, 0.68f, 0.72f, 1f), false);
        case TRIM -> coloredMaterial(new ColorRGBA(0.82f, 0.86f, 0.9f, 1f), false);
        case PANEL -> coloredMaterial(new ColorRGBA(0.3f, 0.34f, 0.39f, 1f), false);
        case GLASS -> coloredMaterial(new ColorRGBA(0.35f, 0.55f, 0.72f, 0.35f), true);
        case LIGHT -> createLightMaterial();
        default -> coloredMaterial(ColorRGBA.Black, false);
    };
}

private boolean isSolidWorldPoint(float x, float y, float z) {
    BlockType type = currentChunk.getBlock(toChunkIndex(x), toChunkIndex(y), toChunkIndex(z));
    if (isSolidBlock(type)) {
        return true;
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
Vector3f point = new Vector3f(x, y, z);
for (Door door : animatedDoors) {
    if (!door.isOpen() && door.blocksPosition(point, 0f)) {
        return true;
    }
}
return false;
}

private boolean isSolidBlock(BlockType type) {
    return switch (type) {
        case WALL, GLASS, TRIM, PANEL, CEILING -> true;
        default -> false;
    };
}

private boolean loadSavedChunk(int level) {
    Path path = getMapPath(level);
    if (!Files.exists(path)) {
        return false;
    }

    try {
        List<String> lines = Files.readAllLines(path);
        currentChunk.fillAir();
        for (String line : lines) {
            if (line == null || line.isBlank() || line.startsWith("SUBJECT47_MAP")) {
                continue;
            }
            String[] parts = line.split(",");
            if (parts.length != 4) {
                continue;
            }
            int x = Integer.parseInt(parts[0]);
            int y = Integer.parseInt(parts[1]);
            int z = Integer.parseInt(parts[2]);
            BlockType type = BlockType.valueOf(parts[3]);
            currentChunk.setBlock(x, y, z, type);
        }
        return true;
    } catch (IOException | IllegalArgumentException exception) {
        app.getUi().showMessage("Failed to load saved map for level " + level + ".", 3f);
        return false;
    }
}

private Path getMapPath(int level) {
    return MAP_DIRECTORY.resolve("level-" + level + ".map");
}

private Path getPropPath(int level) {
    return MAP_DIRECTORY.resolve("level-" + level + ".props");
}

/**
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

** Loads saved props. Returns true when a saved file was found (caller should
* skip level decoration in that case so the file is the sole source of truth).
/

```
private boolean loadSavedProps(int level) {
    Path path = getPropPath(level);
    if (!Files.exists(path)) return false;

    try {
        List<String> lines = Files.readAllLines(path);
        String header = lines.isEmpty() ? "" : lines.get(0);
        boolean v2 = header.startsWith("SUBJECT47_PROPS 2");
        boolean v3 = header.startsWith("SUBJECT47_PROPS 3");

        for (String line : lines) {
            if (line == null || line.isBlank() || line.startsWith("SUBJECT47_PROPS")) continue;

            if ((v2 || v3) && line.startsWith("LEVEL:")) {
                String payload = line.substring(6);
                String[] parts = payload.split(",");
                if (parts.length < 4) continue;
                EditorPropType type = EditorPropType.valueOf(parts[0]);
                int x = Integer.parseInt(parts[1]);
                int y = Integer.parseInt(parts[2]);
                int z = Integer.parseInt(parts[3]);
                String propId = "level-prop-" + nextEditorPropId++;
                Spatial spatial = buildLevelProp(propId, type, x, y, z);
                if (spatial != null) {
                    spatial.setUserData("levelPropRecord", type.name() + "," + x + "," + y + "," + z);
                    levelProps.put(propId, spatial);
                }
            } else {
                String[] parts = line.split(",");
                if (parts.length < 4) continue;
                EditorPropType type = EditorPropType.valueOf(parts[0]);
                int rot = 0;
                Vector3f pos;
                if (v3 && parts.length >= 5) {
                    // v3: TYPE,sceneX,sceneY,sceneZ,rotation
                    float sx = Float.parseFloat(parts[1]);
                    float sy = Float.parseFloat(parts[2]);
                    float sz = Float.parseFloat(parts[3]);
                    rot = Integer.parseInt(parts[4]);
                    pos = new Vector3f(sx, sy, sz);
                } else {
                    // v2: TYPE,blockX,blockY,blockZ (legacy, approximate)
                    int x = Integer.parseInt(parts[1]);
                    int y = Integer.parseInt(parts[2]);
                    int z = Integer.parseInt(parts[3]);
                    pos = worldToScene(x, y, z);
                }
                String propId = "editor-prop-" + nextEditorPropId++;
                EditorPropInstance inst = buildEditorProp(propId, type, pos, rot,
                    new Vector3f(0f, 0f, -1f));
                if (inst != null) editorProps.put(propId, inst);
            }
        }
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
}
return true;
} catch (IOException | IllegalArgumentException exception) {
    app.getUi().showMessage("Failed to load saved props for level " + level + ".", 3f);
    return false;
}
}

private EditorPropInstance buildEditorProp(String propId, EditorPropType type,
                                           Vector3f pos, int rotation, Vector3f viewDirection) {
    List<Spatial> spatials = new ArrayList<>();
    Door linkedDoor = null;

    Quaternion rotQ = (rotation != 0)
        ? new Quaternion().fromAngleAxis(rotation * FastMath.HALF_PI, Vector3f.UNIT_Y)
        : null;

    Vector3f rawOff = computeButtonOffset(viewDirection);
    Vector3f buttonOff = (rotQ != null) ? rotQ.mult(rawOff) : rawOff;

    switch (type) {
        case AUTO_DOOR -> {
            Door door = createDoor("EditorDoor" + propId, pos, false);
            linkedDoor = door;
            if (rotQ != null) door.getSpatial().setLocalRotation(rotQ);
            Vector3f btnPos = pos.add(buttonOff.x * 1.2f, -0.1f, buttonOff.z * 1.2f);
            Spatial button = createDoorButton("EditorDoorButton" + propId, btnPos,
                new ControlSwitch(app, "Press E to cycle the door", null,
                    () -> door.toggleFromButton("Door cycles open.")));
            tagEditorProp(button, propId);
            tagEditorProp(door.getSpatial(), propId);
            spatials.add(door.getSpatial());
            spatials.add(button);
        }
        case KEYPAD_DOOR -> {
            // The door itself opens the keypad when locked, then toggles when unlocked.
            Door door = createDoor("EditorKDoor" + propId, pos, true);
            linkedDoor = door;
            if (rotQ != null) door.getSpatial().setLocalRotation(rotQ);
            door.setOnLockedCallback(() ->
                app.getUi().openKeypad("DOOR ACCESS CODE", code -> {
                    if ("1234".equals(code)) {
                        door.unlock("Access granted.");
                    } else {
                        app.getUi().showMessage("Access denied.", 2f);
                    }
                })
            );
            tagEditorProp(door.getSpatial(), propId);
            spatials.add(door.getSpatial());
        }
        case CONSOLE -> {
            Spatial console = createConsole("EditorConsole" + propId, pos,
                new ColorRGBA(0.18f, 0.78f, 0.95f, 1f),
                new ControlSwitch(app, "Press E to inspect console", "Console is idle.", null));
        }
    }
}
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
if (rotQ != null) console.setLocalRotation(rotQ);
tagEditorProp(console, propId);
spatials.add(console);
}
case LAB_BENCH -> {
    Spatial bench = createLabBench("EditorBench" + propId, pos, 1f, new ColorRGBA(0.18f, 0.82f, 0.95f, 1f));
    if (rotQ != null) bench.setLocalRotation(rotQ);
    tagEditorProp(bench, propId);
    spatials.add(bench);
}
case SERVER_RACK -> {
    Spatial rack = createServerRack("EditorRack" + propId, pos, new ColorRGBA(0.22f, 0.86f, 0.95f, 1f));
    if (rotQ != null) rack.setLocalRotation(rotQ);
    tagEditorProp(rack, propId);
    spatials.add(rack);
}
case STORAGE_CRATE -> {
    Spatial crate = createStorageCrate("EditorCrate" + propId, pos, new ColorRGBA(0.95f, 0.72f, 0.22f, 1f));
    if (rotQ != null) crate.setLocalRotation(rotQ);
    tagEditorProp(crate, propId);
    spatials.add(crate);
}
case MEDICAL_CART -> {
    Spatial cart = createMedicalCart("EditorCart" + propId, pos, new ColorRGBA(0.18f, 0.86f, 0.95f, 1f));
    if (rotQ != null) cart.setLocalRotation(rotQ);
    tagEditorProp(cart, propId);
    spatials.add(cart);
}
case WALL_MONITOR -> {
    Spatial monitor = createWallMonitor("EditorMonitor" + propId, pos, new ColorRGBA(0.18f, 0.86f, 0.95f, 1f));
    if (rotQ != null) monitor.setLocalRotation(rotQ);
    tagEditorProp(monitor, propId);
    spatials.add(monitor);
}
case LOCKER_BANK -> {
    Spatial lockers = createLockerBank("EditorLockers" + propId, pos, new ColorRGBA(0.82f, 0.14f, 0.14f, 1f));
    if (rotQ != null) lockers.setLocalRotation(rotQ);
    tagEditorProp(lockers, propId);
    spatials.add(lockers);
}
case CEILING_LIGHT -> {
    Spatial light = createCeilingLight("EditorLight" + propId, pos, new ColorRGBA(0.74f, 0.9f, 1f, 1f));
    tagEditorProp(light, propId);
    spatials.add(light);
}
case ANALYZER_STATION -> {
    Spatial station = createAnalyzerStation("EditorAnalyzer" + propId, pos, new ColorRGBA(0.18f, 0.88f, 0.95f,
1f));
    if (rotQ != null) station.setLocalRotation(rotQ);
    tagEditorProp(station, propId);
    spatials.add(station);
}
case TANK_CLUSTER -> {
    Spatial tanks = createTankCluster("EditorTanks" + propId, pos, new ColorRGBA(0.18f, 0.76f, 0.95f, 1f));
    if (rotQ != null) tanks.setLocalRotation(rotQ);
```




```
        tagEditorProp(tanks, propId);
        spatials.add(tanks);
    }
    case CONTAINMENT_POD -> {
        Spatial pod = createContainmentPod("EditorPod" + propId, pos, new ColorRGBA(0.25f, 0.9f, 0.42f, 1f));
        if (rotQ != null) pod.setLocalRotation(rotQ);
        tagEditorProp(pod, propId);
        spatials.add(pod);
    }
    default -> { return null; }
}

return new EditorPropInstance(propId, type, pos, rotation, linkedDoor, spatials);
}

private Vector3f computeButtonOffset(Vector3f direction) {
    if (Math.abs(direction.x) > Math.abs(direction.z)) {
        return new Vector3f(0f, 0f, direction.x >= 0f ? 1f : -1f);
    }
    return new Vector3f(direction.z >= 0f ? -1f : 1f, 0f, 0f);
}

private int clampToNormal(int delta) {
    if (delta > 0) {
        return 1;
    }
    if (delta < 0) {
        return -1;
    }
    return 0;
}

private Material createLightMaterial() {
    Material material = coloredMaterial(new ColorRGBA(0.35f, 0.7f, 0.95f, 1f), true);
    lightMaterials.add(material);
    return material;
}

private Material coloredMaterial(ColorRGBA color, boolean emissive) {
    Material material = new Material(app.getAssetManager(), "Common/MatDefs/Light/Lighting.j3md");
    material.setBoolean("UseMaterialColors", true);
    material.setColor("Diffuse", color);
    material.setColor("Ambient", color.mult(0.45f));
    if (emissive) {
        material.setColor("GlowColor", color);
    }
    if (color.a < 1f) {
        material.getAdditionalRenderState().setBlendMode(RenderState.BlendMode.Alpha);
    }
    return material;
}

private void applyLightState(float brightness) {
    ColorRGBA base = emergencyLighting
        ? new ColorRGBA(0.85f, 0.18f, 0.18f, 1f)
```



```
: new ColorRGBA(0.35f, 0.7f, 0.95f, 1f);

ColorRGBA lit = base.mult(brightness);
ColorRGBA ambient = lit.mult(0.45f);

for (Material material : lightMaterials) {
    material.setColor("Diffuse", lit);
    material.setColor("Ambient", ambient);
    material.setColor("GlowColor", lit);
}

private void tagSpatial(Spatial spatial, Interactable interactable) {
    String interactableId = "interactable-" + nextInteractableId++;
    tagSpatial(spatial, interactable, interactableId);
    interactables.put(interactableId, interactable);
}

private void tagSpatial(Spatial spatial, Interactable interactable, String interactableId) {
    spatial.setUserData(INTERACTABLE_ID, interactableId);
    if (spatial instanceof Node node) {
        for (Spatial child : node.getChildren()) {
            tagSpatial(child, interactable, interactableId);
        }
    }
}

private void tagEditorProp(Spatial spatial, String propId) {
    if (spatial == null) {
        return;
    }
    spatial.setUserData(EDITOR_PROP_ID, propId);
    if (spatial instanceof Node node) {
        for (Spatial child : node.getChildren()) {
            tagEditorProp(child, propId);
        }
    }
}

/** Tags all geometry in a spatial with the level-prop id (for ray-pick lookup). */
public void tagLevelProp(Spatial spatial, String propId) {
    if (spatial == null) {
        return;
    }
    spatial.setUserData(LEVEL_PROP_ID, propId);
    if (spatial instanceof Node node) {
        for (Spatial child : node.getChildren()) {
            tagLevelProp(child, propId);
        }
    }
}

/**
 * Registers a spatial as a level prop so the editor can remove it.
 * The record string is persisted on save so the prop can be reconstructed.
 */
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
*/
public void registerLevelProp(String propId, Spatial spatial, EditorPropType type, int x, int y, int z) {
    String record = type.name() + "," + x + "," + y + "," + z;
    spatial.setUserData("levelPropRecord", record);
    tagLevelProp(spatial, propId);
    levelProps.put(propId, spatial);
}

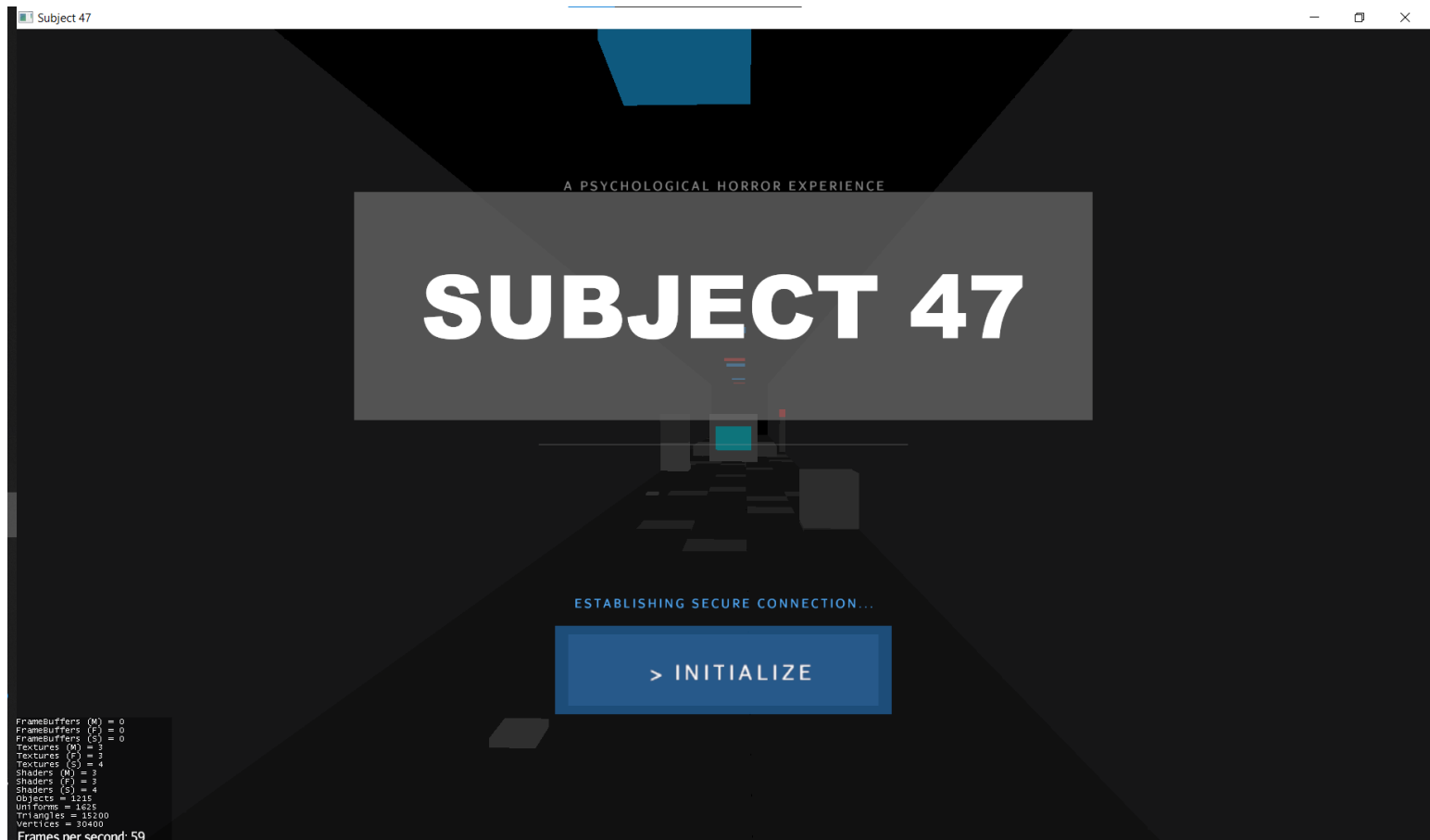
/** Walks up the scene-graph hierarchy to find a user-data tag. */
private String findTagInHierarchy(Spatial spatial, String key) {
    Spatial current = spatial;
    while (current != null) {
        String val = current.getUserData(key);
        if (val != null) {
            return val;
        }
        current = current.getParent();
        // Stop at propNode boundary
        if (current == propNode) {
            break;
        }
    }
    return null;
}

/**
 * Builds a single-spatial level prop (no Door/ControlSwitch wiring).
 * Used when restoring level props from a v2 save file.
 */
private Spatial buildLevelProp(String propId, EditorPropType type, int x, int y, int z) {
    return switch (type) {
        case CONSOLE -> {
            Spatial s = createConsole("LvlConsole" + propId, worldToScene(x, 1.2f, z),
                new ColorRGBA(0.18f, 0.78f, 0.95f, 1f),
                new subject47.interaction.ControlSwitch(app, "Press E to inspect console", "Console is idle.", null));
            yield s;
        }
        case LAB_BENCH -> createLabBench("LvlBench" + propId, worldToScene(x, 0f, z), 1f, new ColorRGBA(0.18f,
            0.82f, 0.95f, 1f));
        case SERVER_RACK -> createServerRack("LvlRack" + propId, worldToScene(x, 1.2f, z), new ColorRGBA(0.22f,
            0.86f, 0.95f, 1f));
        case STORAGE_CRATE -> createStorageCrate("LvlCrate" + propId, worldToScene(x, 0.45f, z), new
            ColorRGBA(0.95f, 0.72f, 0.22f, 1f));
        case MEDICAL_CART -> createMedicalCart("LvlCart" + propId, worldToScene(x, 0.4f, z), new
            ColorRGBA(0.18f, 0.86f, 0.95f, 1f));
        case WALL_MONITOR -> createWallMonitor("LvlMonitor" + propId, worldToScene(x, 1.5f, z), new
            ColorRGBA(0.18f, 0.86f, 0.95f, 1f));
        case LOCKER_BANK -> createLockerBank("LvlLockers" + propId, worldToScene(x, 0f, z), new
            ColorRGBA(0.82f, 0.14f, 0.14f, 1f));
        case CEILING_LIGHT -> createCeilingLight("LvlLight" + propId, worldToScene(x, y + 0.75f, z), new
            ColorRGBA(0.74f, 0.9f, 1f, 1f));
        case ANALYZER_STATION -> createAnalyzerStation("LvlAnalyzer" + propId, worldToScene(x, 0f, z), new
            ColorRGBA(0.18f, 0.88f, 0.95f, 1f));
        case TANK_CLUSTER -> createTankCluster("LvlTanks" + propId, worldToScene(x, 0f, z), new
            ColorRGBA(0.18f, 0.76f, 0.95f, 1f));
    }
}
```



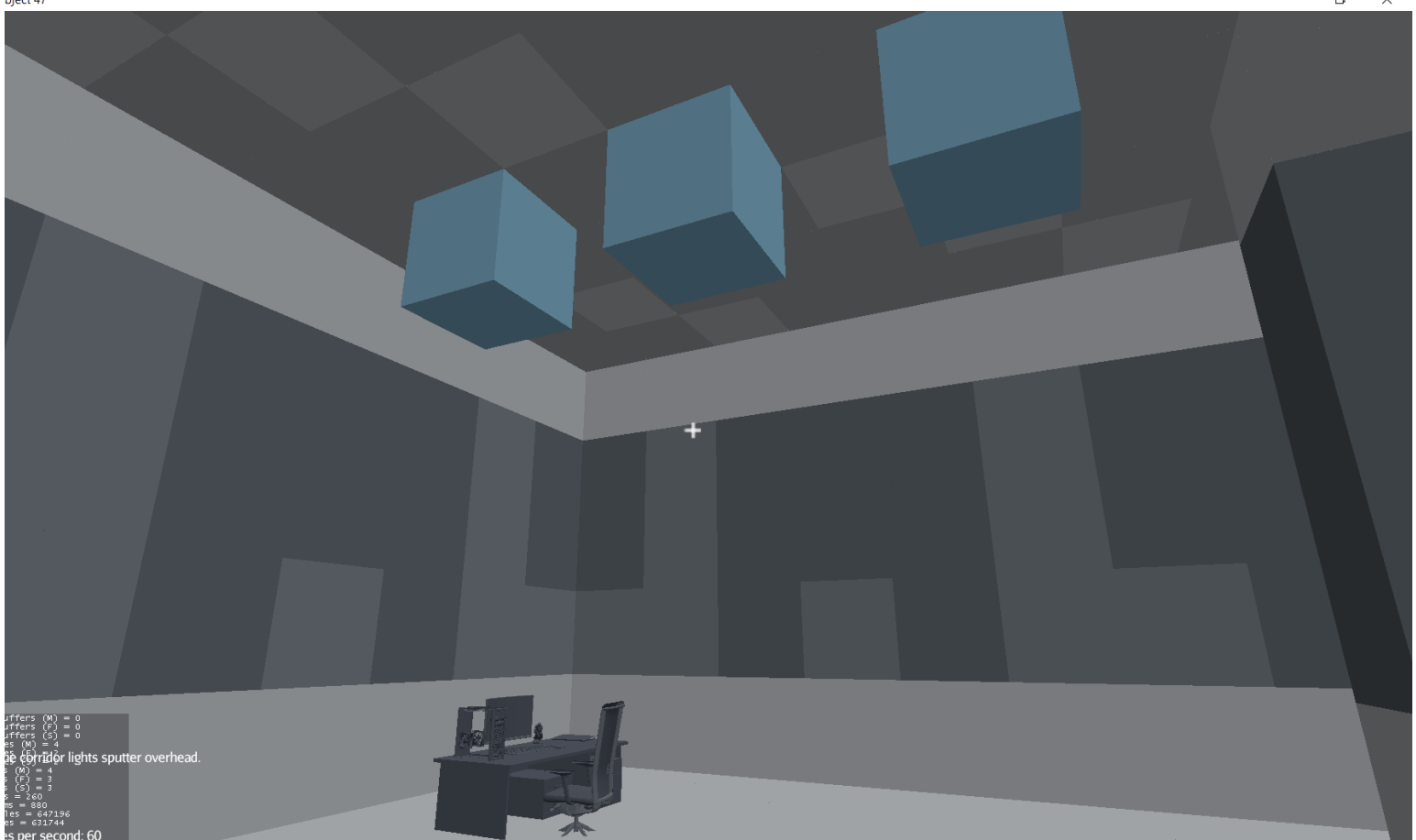
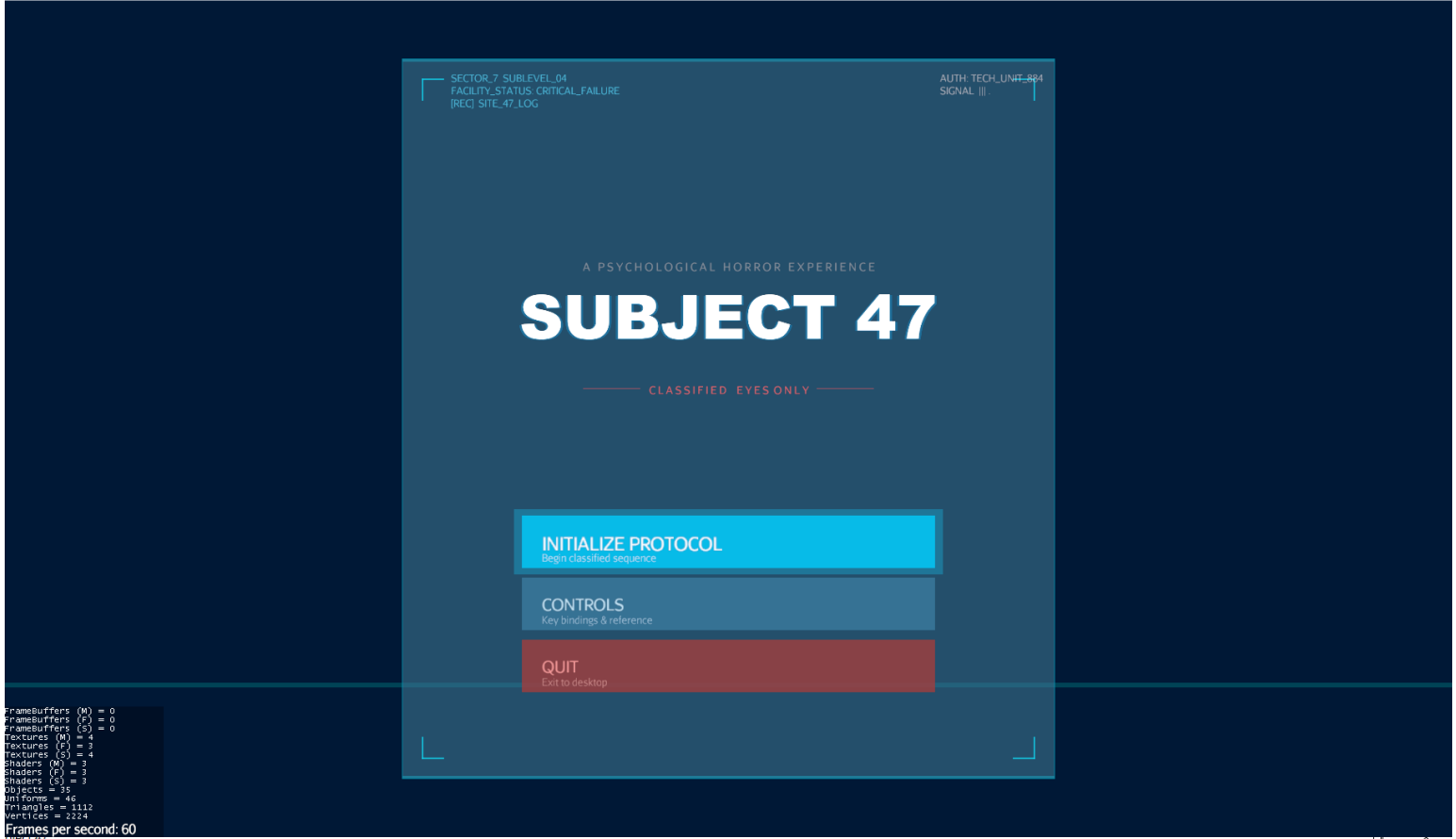
```
case CONTAINMENT_POD -> createContainmentPod("LvlPod" + propId, worldToScene(x, 0.2f, z), new  
ColorRGBA(0.25f, 0.9f, 0.42f, 1f));  
default -> null; // AUTO_DOOR needs wiring — skip on restore  
};  
}  
  
public record BlockHit(int x, int y, int z, BlockType type, Vector3f normal) {  
}  
  
private record EditorPropInstance(  
    String id,  
    EditorPropType type,  
    Vector3f pos,  
    int rotation,  
    Door door,  
    List<Spatial> spatials  
) {  
}  
}
```

Gameplay:





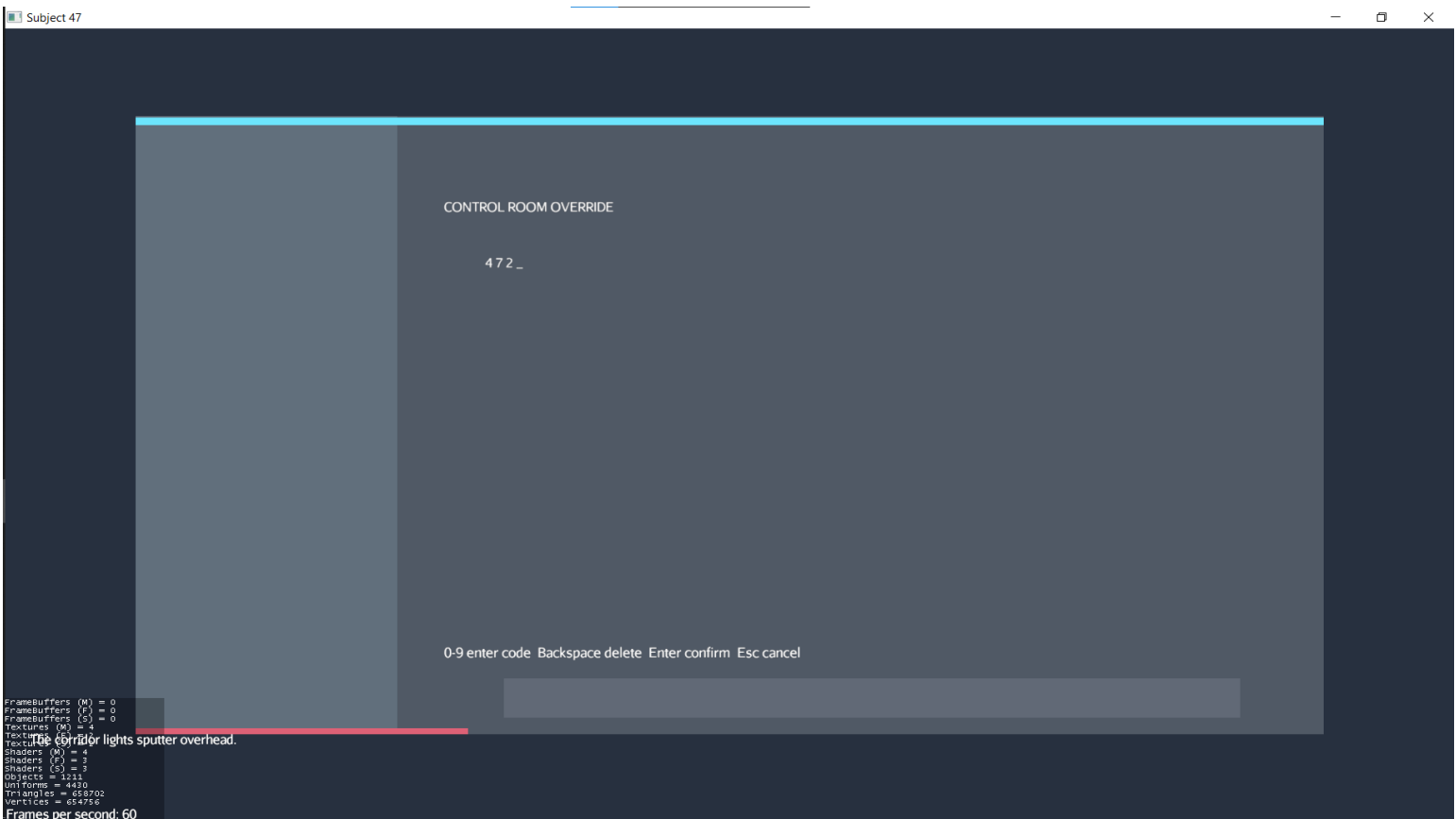
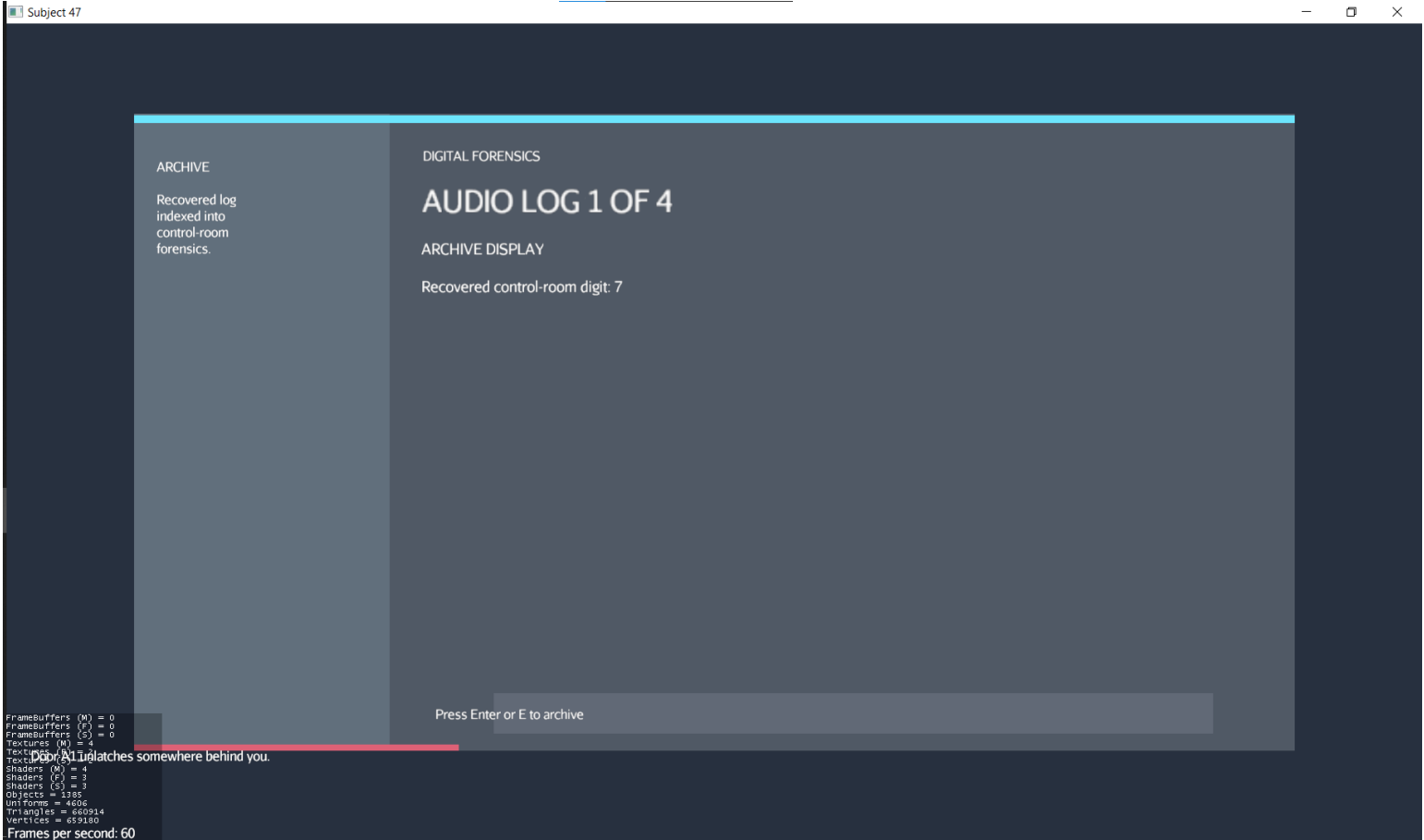
Subject 47





National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

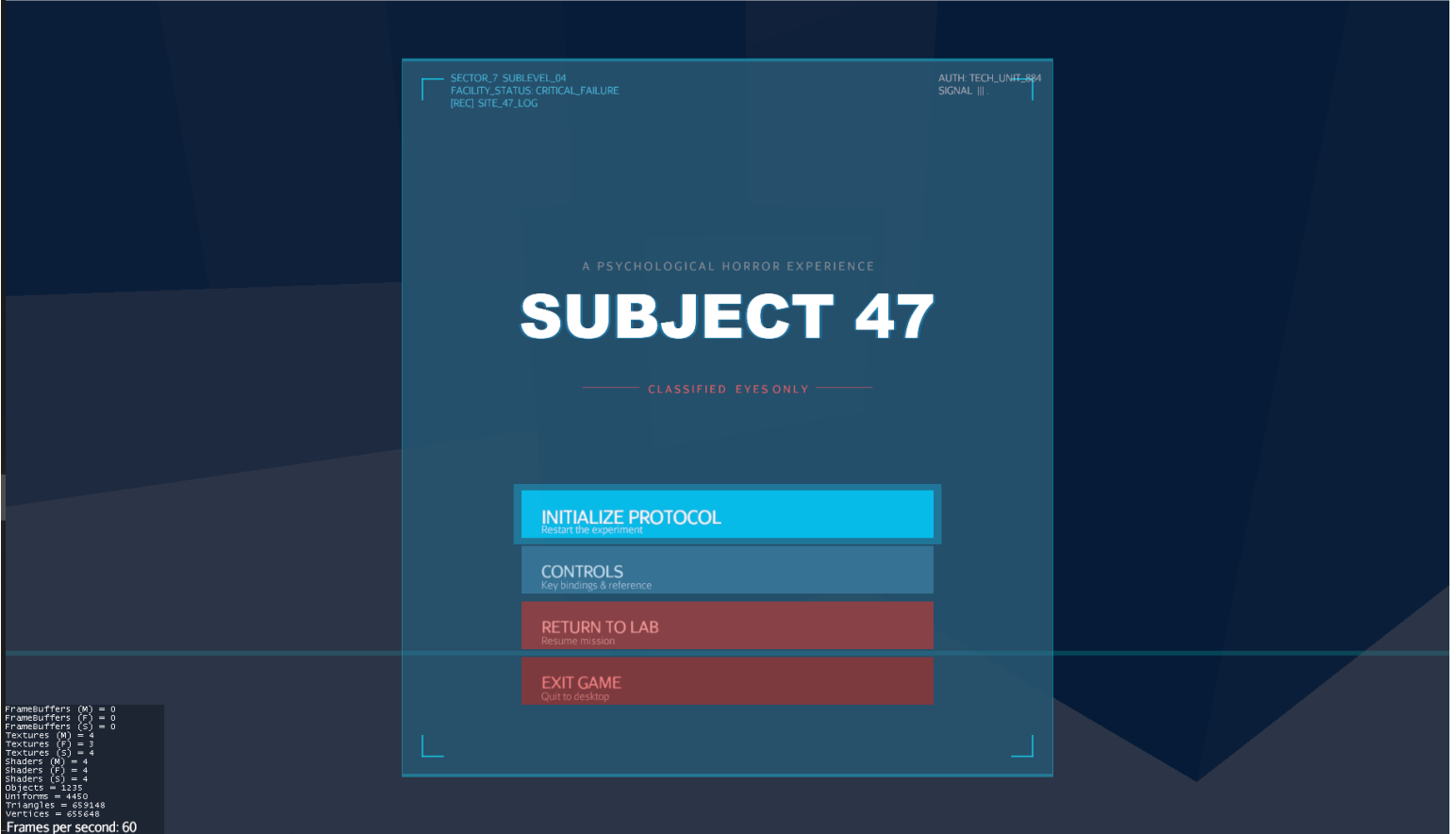




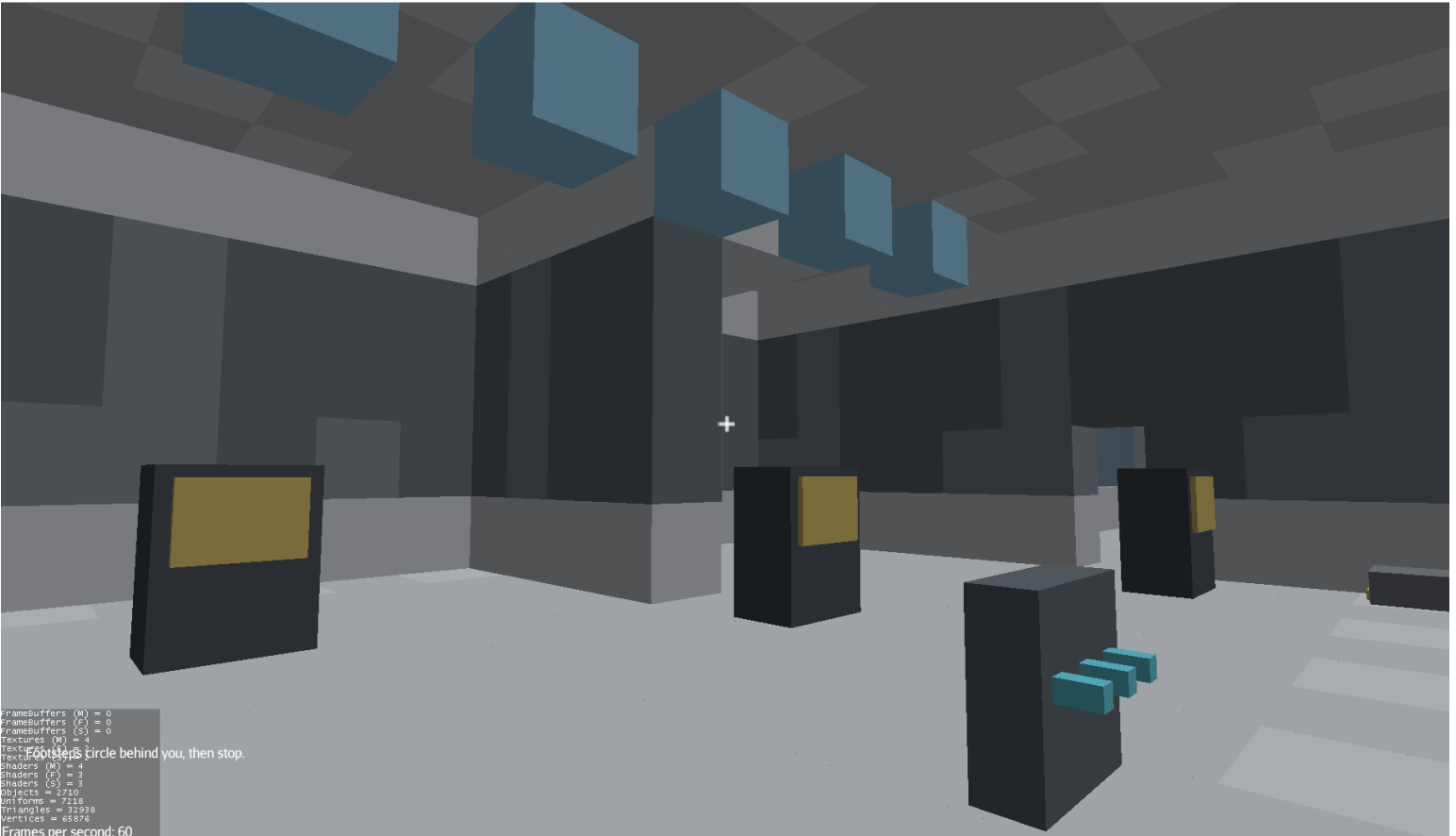
National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

Subject 47



Subject 47





Subject 47

— □ ×

EVENT LOG

Day 1.
Experiment
Omega begins.

Status:
All nominal.

CLASSIFIED ARCHIVE

FACILITY EVENT LOG

EXPERIMENT DESIGNATION: OMEGA -- DAY 1

Dr. Voss initiates neural mapping on Subject 47.
Subject demonstrates unusual pattern retention beyond projected parameters.

Cognitive baseline recorded: exceptional.
Empathy index: off the chart.
Self-modelling score: unprecedented.

NOTE: Subject is not just learning -- it is aware it is learning.
Observation window extended indefinitely.

Press Enter or E to continue [1 / 4]

FrameBuffers (M) = 0
FrameBuffers (P) = 0
FrameBuffers (S) = 0
Textures (M) = 4
Textures (P) = 4
Textures (S) = 4
Shaders (M) = 4
Shaders (P) = 3
Shaders (S) = 3
Objects = 460
Uniforms = 1698
Triangles = 7000
Vertices = 14000
Frames per second: 60

Broken Containment pod. File recovered: MEMORY RESET PROTOCOL. The subject chose to forget.

Subject 47

— □ ×

BRIEFING

Underground
research wing
active.

Review story
and mission
before entry.

REVELATION CORE

LEVEL 4 REVELATION

The deepest wing feels familiar for the wrong reasons.

Two missing pieces remain: the truth of Subject 47 and the reason the lab was abandoned.

Objective:
Recover the final identity evidence, then restore the erased memory.

Press Enter or E to begin

FrameBuffers (M) = 0
FrameBuffers (P) = 0
FrameBuffers (S) = 0
Textures (M) = 4
Textures (P) = 2
Textures (S) = 2
Shaders (M) = 4
Shaders (P) = 3
Shaders (S) = 3
Objects = 1234
Uniforms = 4406
Triangles = 11526
Vertices = 31652
Frames per second: 60



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

